

YOLO11-CARD-CPP

Generated by Doxygen 1.9.8

1 Namespace Index	1
1.1 Namespace List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 Shared Namespace Reference	7
4.1.1 Enumeration Type Documentation	7
4.1.1.1 Choice	7
4.1.2 Function Documentation	7
4.1.2.1 parseChoice()	7
5 Class Documentation	9
5.1 Detection Struct Reference	9
5.1.1 Detailed Description	9
5.1.2 Member Data Documentation	9
5.1.2.1 boundingBox	9
5.1.2.2 classConfidence	9
5.1.2.3 classId	10
5.2 Detections Struct Reference	10
5.2.1 Detailed Description	10
5.2.2 Member Data Documentation	10
5.2.2.1 boundingBoxes	10
5.2.2.2 classConfidences	10
5.2.2.3 classIds	10
5.3 MetricsCounter Struct Reference	11
5.3.1 Detailed Description	11
5.3.2 Member Data Documentation	11
5.3.2.1 fn	11
5.3.2.2 fp	11
5.3.2.3 tp	11
5.4 TerminalRawMode Struct Reference	11
5.4.1 Detailed Description	12
5.4.2 Constructor & Destructor Documentation	12
5.4.2.1 TerminalRawMode()	12
5.4.2.2 ~TerminalRawMode()	12
5.4.3 Member Data Documentation	12
5.4.3.1 oldt	12
5.5 UserData Struct Reference	12
5.5.1 Detailed Description	13

5.5.2 Constructor & Destructor Documentation	13
5.5.2.1 UserData()	13
5.5.3 Member Data Documentation	13
5.5.3.1 choice	13
5.5.3.2 data_path	13
5.6 YOLO_model Class Reference	13
5.6.1 Detailed Description	15
5.6.2 Constructor & Destructor Documentation	15
5.6.2.1 YOLO_model()	15
5.6.3 Member Function Documentation	16
5.6.3.1 clearDetections()	16
5.6.3.2 detect()	16
5.6.3.3 detectionPipeline()	20
5.6.3.4 drawBoundingBoxes() [1/2]	22
5.6.3.5 drawBoundingBoxes() [2/2]	22
5.6.3.6 filterDetectionsNMS()	23
5.6.3.7 getClassName()	24
5.6.3.8 getDataClasses()	24
5.6.3.9 getDetections()	25
5.6.3.10 getDetectionsClassNames()	25
5.6.3.11 getModelName()	26
5.6.3.12 isAvailableGPU()	26
5.6.3.13 logger()	27
5.6.3.14 mergeDetections()	28
5.6.3.15 setModelName()	28
5.6.4 Member Data Documentation	29
5.6.4.1 allocator	29
5.6.4.2 CLASS_CONFIDENCE_THRESHOLD	29
5.6.4.3 classNames	29
5.6.4.4 detections	29
5.6.4.5 env	29
5.6.4.6 modelName	29
5.6.4.7 NMS_THRESHOLD	29
5.6.4.8 session	30
5.6.4.9 sessionOptions	30
5.6.4.10 usingGPU	30
5.6.4.11 YOLO_TARGET_INPUT_SIZE	30
6 File Documentation	31
6.1 include/hermann_serain.hpp File Reference	31
6.1.1 Function Documentation	32
6.1.1.1 computeImageMetrics()	32

6.1.1.2 computeVideoMetrics()	32
6.1.1.3 initObjectsForVideoMetrics()	33
6.1.1.4 printFinalVideoMetrics()	33
6.1.1.5 readInput()	34
6.2 hermann_serain.hpp	34
6.3 include/marco_annunziata.hpp File Reference	35
6.4 marco_annunziata.hpp	36
6.5 include/shared.hpp File Reference	37
6.6 shared.hpp	38
6.7 include/sveva_turola.hpp File Reference	38
6.7.1 Function Documentation	39
6.7.1.1 cardValues()	39
6.7.1.2 drawCardGroup()	41
6.7.1.3 drawGameStatus()	42
6.7.1.4 frameCapture()	43
6.7.1.5 getDistance()	44
6.7.1.6 playOutputVideo()	45
6.7.1.7 processStream()	46
6.8 sveva_turola.hpp	48
6.9 src/finalProject.cpp File Reference	48
6.9.1 Function Documentation	49
6.9.1.1 main()	49
6.10 finalProject.cpp	50
6.11 src/hermann_serain.cpp File Reference	51
6.11.1 Function Documentation	53
6.11.1.1 calculateloU()	53
6.11.1.2 canComputeImageMetrics()	53
6.11.1.3 canComputeMetrics()	53
6.11.1.4 canComputeVideoMetrics()	53
6.11.1.5 checkPath()	54
6.11.1.6 computeImageMetrics()	54
6.11.1.7 computeVideoMetrics()	54
6.11.1.8 findMatches()	55
6.11.1.9 handleCamera()	55
6.11.1.10 handleFile()	56
6.11.1.11 initObjectsForVideoMetrics()	56
6.11.1.12 parseYoloLabels()	57
6.11.1.13 printFinalMetrics()	57
6.11.1.14 printFinalVideoMetrics()	58
6.11.1.15 printHelp()	58
6.11.1.16 readInput()	59
6.11.1.17 redrawLine()	59

6.11.1.18 splitPath()	60
6.11.1.19 updateMetricsCounters()	60
6.11.2 Variable Documentation	61
6.11.2.1 DEFAULT_IMAGE_PATH	61
6.11.2.2 DEFAULT_VIDEO_PATH	61
6.11.2.3 FLAG_IMAGE	61
6.11.2.4 FLAG_VIDEO	61
6.11.2.5 g_cumulativeVideoMetrics	61
6.11.2.6 g_lastLine	62
6.11.2.7 g_motFile	62
6.11.2.8 g_videoMetricsEnabled	62
6.12 hermann_serain.cpp	62
6.13 src/marco_annunziata.cpp File Reference	69
6.13.1 Macro Definition Documentation	69
6.13.1.1 UNUSED	69
6.14 marco_annunziata.cpp	69
6.15 src/shared.cpp File Reference	76
6.16 shared.cpp	77
6.17 src/sveva_tuola.cpp File Reference	78
6.17.1 Function Documentation	78
6.17.1.1 cardValues()	78
6.17.1.2 drawCardGroup()	80
6.17.1.3 drawGameStatus()	81
6.17.1.4 frameCapture()	83
6.17.1.5 getDistance()	84
6.17.1.6 playOutputVideo()	84
6.17.1.7 processStream()	85
6.18 sveva_tuola.cpp	87
Index	93

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Shared		7
------------------------	-------	-------------------

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Detection	9
Detections	10
MetricsCounter	11
TerminalRawMode	11
UserData	12
YOLO_model	13

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

include/hermann_serain.hpp	31
include/marco_annunziata.hpp	35
include/shared.hpp	37
include/sveva_turola.hpp	38
src/finalProject.cpp	48
src/hermann_serain.cpp	51
src/marco_annunziata.cpp	69
src/shared.cpp	76
src/sveva_turola.cpp	78

Chapter 4

Namespace Documentation

4.1 Shared Namespace Reference

Enumerations

- enum class [Choice](#) { [Camera](#) , [File](#) , [Invalid](#) , [Help](#) }

Functions

- [Choice](#) [parseChoice](#) (std::string s)

4.1.1 Enumeration Type Documentation

4.1.1.1 Choice

```
enum class Shared::Choice [strong]
```

Enumerator

Camera	
File	
Invalid	
Help	

Definition at line [10](#) of file [shared.hpp](#).

```
00010 {Camera, File, Invalid, Help};
```

4.1.2 Function Documentation

4.1.2.1 parseChoice()

```
Choice Shared::parseChoice (  
    std::string s )
```

Definition at line 8 of file [shared.cpp](#).

```
00008                                     {
00009     //NOTE: Manual Trim
00010     while (!s.empty() && std::isspace((unsigned char)s.front())) s.erase(s.begin());
00011     while (!s.empty() && std::isspace((unsigned char)s.back())) s.pop_back();
00012
00013     if (s.size() != 1) return Choice::Invalid;
00014
00015     char c = (char)std::toupper((unsigned char)s[0]);
00016     if (c == 'C' || c == 'c') return Choice::Camera;
00017     if (c == 'F' || c == 'f') return Choice::File;
00018     if (c == 'H' || c == 'h') return Choice::Help;
00019     return Choice::Invalid;
00020 }
```

Chapter 5

Class Documentation

5.1 Detection Struct Reference

```
#include <marco_annunziata.hpp>
```

Public Attributes

- `cv::Rect` [boundingBox](#) = `cv::Rect(0,0,0,0)`
- `float` [classConfidence](#) = 0
- `int` [classId](#) = -1

5.1.1 Detailed Description

Definition at line [16](#) of file [marco_annunziata.hpp](#).

5.1.2 Member Data Documentation

5.1.2.1 boundingBox

```
cv::Rect Detection::boundingBox = cv::Rect(0,0,0,0)
```

Definition at line [17](#) of file [marco_annunziata.hpp](#).

5.1.2.2 classConfidence

```
float Detection::classConfidence = 0
```

Definition at line [18](#) of file [marco_annunziata.hpp](#).

5.1.2.3 classId

```
int Detection::classId = -1
```

Definition at line 19 of file [marco_annunziata.hpp](#).

The documentation for this struct was generated from the following file:

- include/[marco_annunziata.hpp](#)

5.2 Detections Struct Reference

```
#include <marco_annunziata.hpp>
```

Public Attributes

- `std::vector< cv::Rect >` [boundingBoxes](#)
- `std::vector< float >` [classConfidences](#)
- `std::vector< int >` [classIds](#)

5.2.1 Detailed Description

Definition at line 22 of file [marco_annunziata.hpp](#).

5.2.2 Member Data Documentation

5.2.2.1 boundingBoxes

```
std::vector<cv::Rect> Detections::boundingBoxes
```

Definition at line 23 of file [marco_annunziata.hpp](#).

5.2.2.2 classConfidences

```
std::vector<float> Detections::classConfidences
```

Definition at line 24 of file [marco_annunziata.hpp](#).

5.2.2.3 classIds

```
std::vector<int> Detections::classIds
```

Definition at line 25 of file [marco_annunziata.hpp](#).

The documentation for this struct was generated from the following file:

- include/[marco_annunziata.hpp](#)

5.3 MetricsCounter Struct Reference

```
#include <hermann_serain.hpp>
```

Public Attributes

- int [tp](#) = 0
- int [fp](#) = 0
- int [fn](#) = 0

5.3.1 Detailed Description

Definition at line 27 of file [hermann_serain.hpp](#).

5.3.2 Member Data Documentation

5.3.2.1 fn

```
int MetricsCounter::fn = 0
```

Definition at line 30 of file [hermann_serain.hpp](#).

5.3.2.2 fp

```
int MetricsCounter::fp = 0
```

Definition at line 29 of file [hermann_serain.hpp](#).

5.3.2.3 tp

```
int MetricsCounter::tp = 0
```

Definition at line 28 of file [hermann_serain.hpp](#).

The documentation for this struct was generated from the following file:

- include/[hermann_serain.hpp](#)

5.4 TerminalRawMode Struct Reference

Public Member Functions

- [TerminalRawMode](#) ()
- [~TerminalRawMode](#) ()

Public Attributes

- `termios` [oldt](#)

5.4.1 Detailed Description

Definition at line 31 of file [hermann_serain.cpp](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 TerminalRawMode()

```
TerminalRawMode::TerminalRawMode ( ) [inline]
```

Definition at line 35 of file [hermann_serain.cpp](#).

```
00036     {
00037         tcgetattr(STDIN_FILENO, &oldt);
00038         termios newt = oldt;
00039
00040         newt.c_lflag &= ~(ICANON | ECHO); // no buffer, no echo
00041         tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00042     }
```

5.4.2.2 ~TerminalRawMode()

```
TerminalRawMode::~~TerminalRawMode ( ) [inline]
```

Definition at line 44 of file [hermann_serain.cpp](#).

```
00045     {
00046         tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00047     }
```

5.4.3 Member Data Documentation

5.4.3.1 oldt

```
termios TerminalRawMode::oldt
```

Definition at line 33 of file [hermann_serain.cpp](#).

The documentation for this struct was generated from the following file:

- [src/hermann_serain.cpp](#)

5.5 UserData Struct Reference

```
#include <hermann_serain.hpp>
```

Public Member Functions

- [UserData](#) ([Choice](#) _choice, string _data_path)

Public Attributes

- [Choice](#) choice
- string [data_path](#)

5.5.1 Detailed Description

Definition at line 19 of file [hermann_serain.hpp](#).

5.5.2 Constructor & Destructor Documentation

5.5.2.1 UserData()

```
UserData::UserData (
    Choice _choice,
    string _data_path ) [inline]
```

Definition at line 23 of file [hermann_serain.hpp](#).

```
00024      : choice(_choice), data\_path(_data_path) {}
```

5.5.3 Member Data Documentation

5.5.3.1 choice

```
Choice UserData::choice
```

Definition at line 20 of file [hermann_serain.hpp](#).

5.5.3.2 data_path

```
string UserData::data_path
```

Definition at line 21 of file [hermann_serain.hpp](#).

The documentation for this struct was generated from the following file:

- include/[hermann_serain.hpp](#)

5.6 YOLO_model Class Reference

```
#include <marco_annunziata.hpp>
```

Public Member Functions

- [YOLO_model](#) ()
Constructor that handles the creation of the ONNX Runtime environment.
- bool [isAvailableGPU](#) ()
Checks if GPU providers are available and updates the class with this information.
- std::vector< [Detection](#) > [detectionPipeline](#) (cv::Mat &img, bool enable_letterbox_padding=true)
High-level function for fast and accurate detection of bounding boxes on the given image.
- void [clearDetections](#) ()
- [Detections](#) [detect](#) (const cv::Mat &img, bool enable_letterbox_padding=true)
Detects bounding boxes from an image.
- void [mergeDetections](#) ([Detections](#) &dest, const [Detections](#) &source)
Merges two [Detections](#) structs together in the first struct passed.
- std::vector< std::string > [getDataClasses](#) (std::string labelsFilename="../data/model/labels.txt")
Function to call to read the class names from the labels file and save them inside [classNames](#) attribute of the class.
- std::vector< [Detection](#) > [filterDetectionsNMS](#) ([Detections](#) goodDetections)
Applies Non-Maximum Suppression to detections with low confidence.
- cv::Mat [drawBoundingBoxes](#) (cv::Mat &img, std::vector< [Detection](#) > &detections, cv::Scalar color)
Draws bounding boxes for the detected objects.
- cv::Mat [drawBoundingBoxes](#) (cv::Mat &img, cv::Scalar color)
Draws bounding boxes for the detected objects, using the detections contained in [detections](#).
- void [setModelName](#) (std::string modelName)
modelName setter
- std::string [getModelName](#) ()
modelName getter
- std::vector< [Detection](#) > [getDetections](#) ()
detections getter
- std::vector< std::string > [getDetectionsClassNames](#) ([Detections](#) detections)
Returns the class names of the corresponding id's given by the [Detections](#) struct given in input.
- std::string [getClassName](#) (int classId)
Returns the class name of the corresponding id given in input.

Static Private Member Functions

- static void [logger](#) (void *param, OrtLoggingLevel severity, const char *category, const char *logid, const char *code_location, const char *message)
Logger function used by the constructor that handles the creation of the ONNX Runtime environment.

Private Attributes

- Ort::Env [env](#)
- Ort::Session [session](#)
- Ort::SessionOptions [sessionOptions](#)
- Ort::AllocatorWithDefaultOptions [allocator](#)
- bool [usingGPU](#) = false
- const int [YOLO_TARGET_INPUT_SIZE](#) = 640
- std::vector< [Detection](#) > [detections](#)
- std::string [modelName](#) = "Yolo"
- std::vector< std::string > [classNames](#)
- const float [CLASS_CONFIDENCE_THRESHOLD](#) = 0.5
- const float [NMS_THRESHOLD](#) = 0.5

5.6.1 Detailed Description

Definition at line 28 of file [marco_annunziata.hpp](#).

5.6.2 Constructor & Destructor Documentation

5.6.2.1 YOLO_model()

```
YOLO_model::YOLO_model ( )
```

Constructor that handles the creation of the ONNX Runtime environment.

The constructor:

- make use of a logger function for logging its activities
- sets the use of the GPU if present
- sets the number of threads to use as the maximum number available
- sets the path of the model

See also

[YOLO_model::logger\(void* param, OrtLoggingLevel severity, const char* category, const char* logid, const char* code_](#)

Author

Marco Annunziata

Definition at line 27 of file [marco_annunziata.cpp](#).

```
00027         : env(ORT_LOGGING_LEVEL_VERBOSE, "YOLOModel", YOLO_model::logger, this),
    session(nullptr), sessionOptions() {
00028     if(usingGPU){
00029         try{
00030             OrtCUDAProviderOptions cudaOptions;
00031             cudaOptions.device_id = 0;
00032             cudaOptions.gpu_mem_limit = SIZE_MAX;
00033             // arena_extend_strategy: how much space to allocate at a time (arena = allocated block)
00034             // = 0 -> kNextPowerOfTwo, if the arena is filled up double its size (fast but
RAM-intensive)
00035             // = 1 -> kSameAsRequested, if the arena is filled up increase size exactly to what is
needed for the operation
00036             cudaOptions.arena_extend_strategy = 0;
00037             sessionOptions.AppendExecutionProvider_CUDA(cudaOptions);
00038             std::cout << "Using ONNX Runtime with GPU resources" << std::endl;
00039         }
00040         catch(const std::exception &e){
00041             std::cout << "CUDA not available! Using ONNX Runtime with CPU resources" << std::endl;
00042         }
00043     }
00044     unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00045     sessionOptions.SetIntraOpNumThreads(maxHardwareThreads);
00046     sessionOptions.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_EXTENDED);
00047     sessionOptions.SetLogId("YOLOModel");
00048
00049     const char* modelPath = "../data/model/YOLO11s_big_best_dynamic.onnx"; // YOLO11s_big_best_dynamic
00050     session = Ort::Session(env, modelPath, sessionOptions);
00051 }
```

5.6.3 Member Function Documentation

5.6.3.1 clearDetections()

```
void YOLO_model::clearDetections ( )
```

clears the contents of the class attribute [detections](#)

Definition at line 207 of file [marco_annunziata.cpp](#).

```
00207     {
00208         this->detections.clear();
00209     }
```

5.6.3.2 detect()

```
Detections YOLO_model::detect (
    const cv::Mat & img,
    bool enable_letterbox_padding = true )
```

Detects bounding boxes from an image.

There is a pre-processing step to prepare the image for the YOLO model (including letterbox padding, if enabled).

Attention

- 1) The function by itself doesn't clear previous detections saved inside the class.
- 2) The function returns ALL detections, even the ones with low confidence.

Parameters

<i>img</i>	A cv::Mat object containing the image
<i>enable_letterbox_padding</i>	flag to enable letterbox padding on the image before running the detection

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

[clearDetections\(\)](#)

[filterDetectionsNMS\(Detections goodDetections\)](#)

Author

Marco Annunziata

Definition at line 243 of file [marco_annunziata.cpp](#).

```
00243     {
00244         /* Note: there is small to no documentation of the C++ implementation of OnnxRuntime even from the
00245            developers themselves, so this function has been thoroughly optimized and documented
00246            such that every step is perfectly clear to everyone reading this code. */
00247         int inputHeight = img.rows;
00248         int inputWidth  = img.cols;
00249         /* It's possible to give an image as input without resizing in OnnxRuntime because it supports
00250            dynamic input, but the processing of the model is way slower.
```

```

00251         The input needs to be resized anyway for the YOLO model to have good accuracy, so
pre-processing of the image is still needed and the dynamic input feature is not used for YOLO */
00252
00253     /* YOLO: PRE-PROCESSING required for the image
00254         the YOLO model requires the image to be:
00255         1. Of dimensions multiples of 32, because we know YOLO has a CNN in its architecture:
00256             -There are 5 convolutional layers with stride=2 where the convolutions downsample
the input by a factor of 2^5=32, so the feature map has size inputSize/32
00257             -The stride of convolution replaces max pooling of typical CNNs
00258             -If the inputSize is not divisible by 32, the feature map become of size
non-integer which leads to feature misalignment
00259         2. Of overall size square (multiple of 32), typically 640x640 or 416x416
00260         3. not deformed (deformed = stretched image), since YOLO doesn't recognize deformed
objects and, even if you train it for that, the predictions are still made in fixed grid cells
00261         This means that:
00262             - If an image is smaller or bigger than 640x640, it has to be resized to the same scale of
640x640 (requirement 2)
00263             - If the image is not square, you need to resize it while keeping its aspect ratio w:h, so
that the result is not deformed (requirement 3)
00264             => the scaling factor has to be saved so that you can correctly resize the bounding box
to the original size of the image
00265             - If the image is not square, after resizing you also need to add padding to have a
resulting image of exactly 640x640 size (requirement 2)
00266         So the pre-processing of the images is composed of these steps:
00267             Case A: If the image is square, just resize it if needed and you're done
00268             Case B: If the image is not square (most of the time):
00269                 1. Resize the image to a multiple of 32 while keeping its aspect ratio, which means:
resize the longest side to 640 and adjust the shorter side following proportions w:h
00270                 2. add padding to the shorter side in order to reach the square size.
00271         Note: to guarantee the best performance, the padding has to be done in the same we YOLO was
trained for.
00272         YOLO was trained using images with letterbox padding, which means that the image is centered
and the padding is around the image.
00273         This is useful because in the uncentered padding the model would have a small bias of
detecting objects which are closer to the corner.
00274         A symmetric padding (letterbox) is more robust to this positional bias.
00275     */
00276
00277     Mat resizedImg;
00278     float resizedWidth    = -1;
00279     float resizedHeight   = -1;
00280     int xPaddedImgCorner  = 0; // It represents the upper left corner of the centered image after
letterbox padding, will be useful later to center the image
00281     int yPaddedImgCorner  = 0;
00282
00283     float imgAspectRatio  = static_cast<float>(inputWidth) / inputHeight;
00284     if(inputWidth == inputHeight){
00285         /* Case A: image is square, just resize */
00286         resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00287         resizedHeight = YOLO_TARGET_INPUT_SIZE;
00288         resize(img, resizedImg, Size(static_cast<int>(resizedWidth),
static_cast<int>(resizedHeight)));
00289     }
00290     else{
00291         /* Case B: image is not square. Resize following aspect ratio, then add letterbox padding*/
00292         if(imgAspectRatio > 1.0f){
00293             resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00294             resizedHeight = YOLO_TARGET_INPUT_SIZE / imgAspectRatio; // imgAspectRatio > 1 -> have to
divide so that the height follows the width increase/decrease
00295             resizedHeight = (static_cast<int>(resizedHeight) / 32) * 32; // Ensure dimensions of the
image are multiples of 32, as required by YOLO11 (in this case width is already multiple of 32)
00296
00297             if(enable_letterbox_padding == true) yPaddedImgCorner =
static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedHeight/2);
00298         }
00299         else{
00300             resizedHeight = YOLO_TARGET_INPUT_SIZE;
00301             resizedWidth  = YOLO_TARGET_INPUT_SIZE * imgAspectRatio; // imgAspectRatio < 1 -> have to
multiply to make the height follow the width value
00302             resizedWidth  = (static_cast<int>(resizedWidth) / 32) * 32;
00303
00304             if(enable_letterbox_padding == true) xPaddedImgCorner =
static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedWidth/2);
00305         }
00306
00307         Mat tempImg;
00308         resize(img, tempImg, Size(static_cast<int>(resizedWidth), static_cast<int>(resizedHeight)));
00309
00310         //letterbox padding
00311         if(enable_letterbox_padding == true){
00312             resizedImg = Mat::zeros(YOLO_TARGET_INPUT_SIZE, YOLO_TARGET_INPUT_SIZE, CV_8UC3);
00313             tempImg.copyTo(resizedImg(Rect(xPaddedImgCorner, yPaddedImgCorner, resizedWidth,
resizedHeight)));
00314         }
00315         else resizedImg = tempImg.clone();
00316     }
00317

```

```

00318     float resizeScalingFactor[2] = {
00319         static_cast<float>(inputWidth) / resizedWidth,
00320         static_cast<float>(inputHeight) / resizedHeight
00321     };
00322
00323     /* TENSORS
00324     OnnxRuntime needs a tensor as input.
00325     The ONNX tensor is not the mathematical tensor (although it is inspired by it).
00326     It's a multi-dimensional array of heterogeneous data types (float, int ...) which is optimized for
00327     computations.
00328     Tensors are useful because they are a convenient data structure for parallelizing GPU computations
00329     during DNN inference.*/
00330
00331     /* INPUT TENSOR
00332     * In OnnxRuntime the tensor is defined by its data (an array) and its shape (dimensions of the
00333     multi-dimensional array)
00334     * Note: ONNX has the convention to have the nput tensor shape following the NCHW order, where N =
00335     batch size, C = n°channels, H = height, W = width of the image.
00336     * For convention height comes before width because data in memory is accessed along the row:
00337     data[y][x]
00338     * You can check the shape of input tensor of your model by inspecting the .onnx file with
00339     https://netron.app/
00340     */
00341
00342     /* Preparing the parameters for the dynamic INPUT TENSOR */
00343     vector<int64_t> inputShape = {1, 3, static_cast<int>(resizedHeight),
00344         static_cast<int>(resizedWidth)}; // 4D tensor
00345     size_t inputTensorSize = 1 * 3 * resizedHeight * resizedWidth;
00346     vector<float> inputData(inputTensorSize); // the input image is flattened into a mono-dimensional
00347     array
00348
00349     // Converting image data from cv::Mat to tensor (normalization to [0,1] and converting (cv::Mat)
00350     BGR to RGB as the YOLO model requires)
00351     double normalizationFactor = 1.0 / 255.0;
00352     for (int row = 0; row < resizedHeight; row++) {
00353         for (int col = 0; col < resizedWidth; col++) {
00354             Vec3b pixel = resizedImg.at<Vec3b>(row, col);
00355             // cv::Mat -> Tensor
00356             inputData[2 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[0] *
00357             normalizationFactor; // B -> R
00358             inputData[1 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[1] *
00359             normalizationFactor; // G -> G
00360             inputData[0 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[2] *
00361             normalizationFactor; // R -> B
00362         }
00363     }
00364
00365     auto memoryInfo = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault); // Allocating
00366     the input tensor on CPU
00367
00368     // Creating the input tensor (which is a view of the data, NOT a deep copy!)
00369     Ort::Value inputOnnxTensor = Ort::Value::CreateTensor<float>(memoryInfo, inputData.data(),
00370         inputTensorSize, inputShape.data(), inputShape.size());
00371
00372     // retrieving now as string the name of the first input and output nodes
00373     // the allocations of these strings are automatically freed by the session
00374     auto inputNodeName = session.GetInputNameAllocated(0, allocator);
00375     auto outputNodeName = session.GetOutputNameAllocated(0, allocator);
00376
00377     // The API needs the array of inputs to set and the array of outputs to get (or even just a
00378     portion of output if needed).
00379     // So here you are setting the ORDER of the input tensors and the ORDER of the outputs.
00380     // This step is useful for models which have multiple input and output tensors where the order
00381     matters.
00382     // In our case we just have a 1 to 1 correspondence of input-output so the arrays passed will only
00383     have 1 item
00384     vector<const char*> inputNodeNames = {inputNodeName.get()};
00385     vector<const char*> outputNodeNames = {outputNodeName.get()};
00386
00387     // Running the inference
00388     auto outputTensors = session.Run(
00389         Ort::RunOptions{nullptr}, // e.g set a verbosity level only for this run
00390         inputNodeNames.data(), &inputOnnxTensor, inputNodeNames.size(), // input to set (=1 input
00391         image)
00392         outputNodeNames.data(), // output to take
00393         outputNodeNames.size() // actually = 1 output tensors because YOLO has only 1
00394         output layer
00395     );
00396
00397     /* OUTPUT TENSOR
00398     // YOLO11 is pre-trained by default on the COCO dataset, so the number of channels could be:
00399     // 84 = 4 + 80 (= class confidence, one for each class)
00400     // BUT, we fine-tuned the training of YOLO11 with our custom dataset, so the output layer is
00401     restructured to represent 53 classes.
00402     // This means the number of channels should be 57 = 4 (bounding box parameters) + 53 (= class
00403     confidence, one for each class)
00404     // Note: class probability = probability of having detected one class in that region

```



```

00383 //      class confidence      = confidence of having detected an object of a class in that
region, measured as class probability * objectness probability
00384 //      objectness probability = probability of having an object in that region, useful for
YOLOv5 but not for YOLO11, since there is already class confidence, so it's been removed.
00385
00386 // Note, YOLOv5 has a different representation of the output layer compared to YOLOv8/YOLO11:
00387 // YOLOv5 -> channels = 85 = 4 (bb_parameters) + 1 (objectness probability) + 80 (= class
probabilities of corresponding 80 classes of the COCO dataset)
00388 // So, in this case the objectness is separate from class confidence and each class has the class
probability instead of the class confidence.
00389 // To get class confidence of each class you have to multiply each class probability*objectness.
So in YOLOv5 the objectness probability was useful!
00390
00391 // To recap, our custom YOLO11 output tensor (3D Mat) has shape (1, 57, 8400*):
00392 // 1 - is the batch size. If =0, it can change dynamically depending
on the input of the net (not needed since in YOLO batch size = 1).
00393 // 57 (84 for default model) - is the number of channels, where the first 4 are the bounding
box parameters (x, y, width, height)
00394 // and the other ones are class probabilities for COCO'S 80
classes (There is no objectiveness parameter in YOLOv8/11)
00395 // 8400 *or lower - is the total number of regions where the predictions are made
("regions" = "grid points" = anchor locations).
00396 // *Since the ONNX model is dynamic, the output tensor can have
a lower number of detections depending on how big is the padding area.
00397 // The first and the third value become 0 when exported into ONNX format with dynamic=True,
indicating that in the dynamic model these parameters can change on runtime.
00398 // Dynamic input/output shapes are not supported by OpenCV, but they are supported by ONNX
Runtime.
00399 // In this case, OnnxRuntime is adopted instead of OpenCV more because of the superior inference
efficiency rather than the support of dynamic models.
00400 // You can also see the shape of the output layer by importing the .onnx file in
https://netron.app/
00401 // See also: https://github.com/orgs/ultralytics/discussions/17254
00402 */
00403
00404 /* Getting dynamic output tensor info (data, shape, count) */
00405 auto& outputTensor = outputTensors[0]; // In the case of YOLO11, only 1 output layer is used, so
there is only 1 output tensor (outputTensors[0])
00406 auto tensorInfo = outputTensor.GetTensorTypeInfoAndShapeInfo();
00407 auto tensorCount = tensorInfo.GetElementCount(); UNUSED(tensorCount);
00408 auto tensorShape = tensorInfo.GetShape();
00409 float* outputData = outputTensor.GetTensorMutableData<float>(); // the output values are flattened
into a 1D array
00410 // YOLO11: outputData[0..4...] = (x, y, width, height, confidence, classConfidence1,
classConfidence2, ...)
00411
00412 int batchSize = static_cast<int>(tensorShape[0]); UNUSED(batchSize);
00413 int numChannels = static_cast<int>(tensorShape[1]);
00414 int numDetections = static_cast<int>(tensorShape[2]); // The number of detections will vary based
on input size and padding area
00415
00416 // NOTE: The output tensor is a matrix like this, if the batch size =1: (neglecting the first four
parameters used for the bounding box)
00417 //      class1_detection1      class1_detection2      ...      class1_detectionN
00418 //      ....
00419 //      class57_detection1      class57_detectionN
00420
00421 Detections goodDetections;
00422 vector<int> classIds;
00423 vector<float> detectedClassConfidences;
00424 vector<Rect> boundingBoxes;
00425
00426 /* Declaring things used in the post-process phase */
00427 Mat classScores;
00428 Point classId;
00429 double maxClassScore;
00430
00431 // I want to iterate through the output as a flat 1D array, so that i can use a pointer instead of
copying values
00432 // When doing this, I need the output to have detections as rows and classes as columns so that I
can evaluate the bounding box of each detection.
00433 //      class1_detection1      class1_detection2      ...      class1_detectionN
TRANSPOSE      detection1_class1      detection1_class2      ...      detection1_class57
00434 //      ....
00435 //      class57_detection1      class57_detectionN
detectionN_class1      detectionN_class57
00436 // This is why a Mat is constructed from the pointer outputData and its transposition t() is taken
00437 Mat outputDataMat= Mat(numChannels, numDetections, CV_32F, outputData).t(); // -> now the data is
in the order we needed! [Det1Class1 Det1Class2 ... Det1Class57 Det2Class1 ...]
00438 // -> This Mat constructor with the fourth paramter is defined as:
00439 // Mat(int rows, int cols, int type, void* data, size_t step=AUTO_STEP);
00440 // The fourth parameter is just a pointer to the data, it doesn't need to copy the values!
00441 // So this constructor just wraps pointer outputData in a matrix header, resulting a lot more
efficient.
00442
00443 for(int i = 0; i < numDetections; i++){

```

```

00444         float* currentDetection = outputDataMat.ptr<float>(i); // 1 detection corresponds to a 1D
        array of the 57 elements of the detection
00445         float* firstClassScore = currentDetection + 4;
00446         classScores = Mat(1, numChannels-4, CV_32FC1, firstClassScore); // using efficient constructor
00447         // Also, minMaxLoc accepts as input a Mat and as output a Point, so this is why classScores
        and classId are defined as Mat and Point.
00448
00449         minMaxLoc(classScores, 0, &maxClassScore, 0, &classId);
00450
00451         if(maxClassScore > CLASS_CONFIDENCE_THRESHOLD){
00452             detectedClassConfidences.push_back(maxClassScore);
00453             classIds.push_back(classId.x);
00454
00455             float xCenter = currentDetection[0];
00456             float yCenter = currentDetection[1];
00457             float boxWidth = currentDetection[2];
00458             float boxHeight = currentDetection[3];
00459
00460             /* removing coordinates translation caused by the padding */
00461             if(enable_letterbox_padding == true){
00462                 xCenter -= xPaddedImgCorner;
00463                 yCenter -= yPaddedImgCorner;
00464             }
00465
00466             /* computing the bounding box of the detected object and scaling it to the original image
        size */
00467             int xPosScaled = int((xCenter - 0.5f*boxWidth) * resizeScalingFactor[0]);
00468             int yPosScaled = int((yCenter - 0.5f*boxHeight) * resizeScalingFactor[1]);
00469             int bWScaled = int(boxWidth * resizeScalingFactor[0]);
00470             int bHScaled = int(boxHeight * resizeScalingFactor[1]);
00471
00472             Rect box = Rect(xPosScaled, yPosScaled, bWScaled, bHScaled);
00473
00474             /* Clamping bounding box to image boundaries */
00475             box.x = max(0, box.x);
00476             box.y = max(0, box.y);
00477             box.width = min(box.width, img.cols - box.x);
00478             box.height = min(box.height, img.rows - box.y);
00479
00480             boundingBoxes.push_back(box);
00481         }
00482     }
00483
00484     goodDetections.boundingBoxes = boundingBoxes;
00485     goodDetections.classConfidences = detectedClassConfidences;
00486     goodDetections.classIds = classIds;
00487
00488     return goodDetections;
00489     /* Non-Maxima Suppression and construction of the final array of detections */
00490 }

```

5.6.3.3 detectionPipeline()

```

std::vector< Detection > YOLO_model::detectionPipeline (
    cv::Mat & img,
    bool enable_letterbox_padding = true )

```

High-level function for fast and accurate detection of bounding boxes on the given image.

The pipeline consists of:

- clearing the previous model detections
- checking if the GPU is available
- using a multi-threaded tiled approach for detecting bounding boxes in big images (which are images with one dimension exceeds 1.5*640 pixels)
- directly starting the detection for images small enough (with all dimensions within 1.5*640 pixels)
- filtering the detections using Non-Maximum Suppression (NMS)

Parameters

<i>img</i>	A cv::Mat object containing the image
<i>enable_letterbox_padding</i>	flag to enable letterbox padding on the image before running the detection

Returns

a std::vector of [Detection](#) structs with all good detections found in the image.

See also

[detect](#)(const Mat &img, bool enable_letterbox_padding)
[mergeDetections](#)(Detections& dest, const Detections& source)
[filterDetectionsNMS](#)(Detections goodDetections)

Author

Marco Annunziata

Definition at line 134 of file [marco_annunziata.cpp](#).

```

00134
00135     int slice_size = 640;
00136     float overlap_ratio = 0.2;
00137     int step = slice_size * (1 - overlap_ratio);
00138
00139     // removes previous detections, in case of multiple subsequent detections
00140     clearDetections();
00141
00142     this->isAvailableGPU();
00143
00144     if(img.cols > 1.5*slice_size || img.rows > 1.5*slice_size){
00145         Detections mergedDetections;
00146         /* Tiled approach if image is too big (makes cards easier to detect for the model) */
00147         if(this->usingGPU){
00148             Detections tileDetections;
00149             for(int y = 0; y <= img.rows - slice_size; y += step){
00150                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00151                     cv::Rect roi(x, y, slice_size, slice_size);
00152                     cv::Mat tile = img(roi);
00153                     tileDetections = this->detect(tile, enable_letterbox_padding);
00154                     /* proceeding to map the box coordinates found in the tile to the whole image,
00155                      * then adding the detections of the specific tile to the detections of the overall
00156                      image
00157                      */
00158                     for(auto &box : tileDetections.boundingBoxes){
00159                         box.x += x;
00160                         box.y += y;
00161                     }
00162                     mergeDetections(mergedDetections, tileDetections);
00163                 }
00164             }
00165             /* Detection on the whole image improves result for large images having really large cards
00166             */
00167             mergeDetections(mergedDetections, this->detect(img, enable_letterbox_padding));
00168             this->detections = filterDetectionsNMS(mergedDetections);
00169         }
00170         else{
00171             unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00172             auto activeThreads = std::make_shared<std::atomic<int>>(0);
00173             std::vector<std::future<Detections>> futures;
00174
00175             for(int y = 0; y <= img.rows - slice_size; y += step){
00176                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00177                     while((*activeThreads) >= (int) maxHardwareThreads){
00178                         std::this_thread::yield();
00179                     }
00180                     (*activeThreads)++;
00181                     cv::Rect roi(x, y, slice_size, slice_size);
00182                     cv::Mat tile = img(roi).clone();
00183                     futures.push_back(std::async(std::launch::async, [this, tile,
00184                     enable_letterbox_padding, x, y, activeThreads](){
00185                         Detections tileDetections = this->detect(tile, enable_letterbox_padding);
00186                         /* proceeding to map the box coordinates found in the tile to the whole image,
00187                          * then adding the detections of the specific tile to the detections of the
00188                          overall image */

```

```

00187         for(auto &box : tileDetections.boundingBoxes){
00188             box.x += x;
00189             box.y += y;
00190         }
00191         (*activeThreads)--;
00192         return tileDetections;
00193     }));
00194     }
00195 }
00196 /* Detect on the whole image, collect detections and merge them, then filter with NMS
00197 suppression */
00197 futures.push_back(std::async(std::launch::async, [this, img, enable_letterbox_padding]() {
00198     return this->detect(img, enable_letterbox_padding); }));
00198     for(auto& future : futures) mergeDetections(mergedDetections, future.get());
00199     this->detections = filterDetectionsNMS(mergedDetections);
00200 }
00201 }
00202 else this->detections = filterDetectionsNMS(this->detect(img, enable_letterbox_padding));
00203 return this->detections;
00204 }

```

5.6.3.4 drawBoundingBoxes() [1/2]

```

cv::Mat YOLO_model::drawBoundingBoxes (
    cv::Mat & img,
    cv::Scalar color )

```

Draws bounding boxes for the detected objects, using the detections contained in [detections](#).

The bounding box is scaled depending on the input image size, using empirical values.

Parameters

<i>img</i>	a integer representing the id of the detected bounding box
<i>color</i>	a integer representing the id of the detected bounding box

Returns

A cv::Mat object containing the image with bounding box drawn, if present.

See also

`drawBoundingBoxes(Mat &img, std::vector<Detection> &detections, Scalar color)`

Author

Marco Annunziata

Definition at line 589 of file [marco_annunziata.cpp](#).

```

00589 {
00590     return drawBoundingBoxes(img, this->detections, color);
00591 }

```

5.6.3.5 drawBoundingBoxes() [2/2]

```

cv::Mat YOLO_model::drawBoundingBoxes (
    cv::Mat & img,
    std::vector< Detection > & detections,
    cv::Scalar color )

```

Draws bounding boxes for the detected objects.

The bounding box is scaled depending on the input image size, using empirical values.

Parameters

<i>img</i>	a integer representing the id of the detected bounding box
<i>detections</i>	a vector of Detection structs containing the detections
<i>color</i>	a integer representing the id of the detected bounding box

Returns

A cv::Mat object containing the image with bounding box drawn, if present.

See also

[drawBoundingBoxes\(Mat &img, Scalar color\)](#)

Author

Marco Annunziata

Definition at line 565 of file [marco_annunziata.cpp](#).

```

00565                                     {
00566     Mat resultImg = img.clone();
00567     for (auto detection : detections)
00568     {
00569         int thickness = max(1, int(max(img.rows, img.cols) / 640));
00570         rectangle(resultImg, detection.boundingBox, color, 2 * thickness);
00571         string label = this->getClassName(detection.classId) + " " +
00572             to_string(static_cast<int>(detection.classConfidence * 100)) + "%";
00572         putText(resultImg, label, Point(detection.boundingBox.x, detection.boundingBox.y - 5 *
00573             thickness), FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 * thickness);
00573     }
00574     // std::string windowTitle = getModelName() + " - " + std::to_string(detections.size()) + "
00574     detections";
00575     // namedWindow(windowTitle, WINDOW_NORMAL);
00576     // imshow(windowTitle, resultImg);
00577     // waitKey(0);
00578     return resultImg;
00579 }
```

5.6.3.6 filterDetectionsNMS()

```

std::vector< Detection > YOLO_model::filterDetectionsNMS (
    Detections goodDetections )
```

Applies Non-Maximum Suppression to detections with low confidence.

Parameters

<i>goodDetections</i>	a Detections struct containing the detections
-----------------------	---

Returns

A std::vector of [Detection](#) structs containing only the detections with confidence higher than [CLASS_CONFIDENCE_THRESHOLD](#)

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

Author

Marco Annunziata

Definition at line 498 of file [marco_annunziata.cpp](#).

```

00498                                     {
00499     vector<int> resultNMS;
00500     cv::dnn::NMSBoxes(goodDetections.boundingBoxes, goodDetections.classConfidences,
00501                       CLASS_CONFIDENCE_THRESHOLD, NMS_THRESHOLD, resultNMS);
00502     std::vector<Detection> finalDetections;
00503     for(auto finalBoxIndex : resultNMS){
00504         Detection finalDetection;
00505         finalDetection.classId = goodDetections.classIds[finalBoxIndex];
00506         finalDetection.classConfidence = goodDetections.classConfidences[finalBoxIndex];
00507         finalDetection.boundingBox = goodDetections.boundingBoxes[finalBoxIndex];
00508         finalDetections.push_back(finalDetection);
00509     }
00510     return finalDetections;
00511 }

```

5.6.3.7 getClassName()

```

std::string YOLO_model::getClassName (
    int classId )

```

Returns the class name of the corresponding id given in input.

In order to find names, [classNames](#) is used, meaning it must not be empty.

Attention

Function to call only when class names have already been read from the file, meaning `getDataClasses(string labelsFilename)` has to be called first

Parameters

<i>classId</i>	a integer representing the id of the detected bounding box
----------------	--

Returns

A string containing the corresponding name

Author

Marco Annunziata

Definition at line 552 of file [marco_annunziata.cpp](#).

```

00552                                     {
00553     return this->classNames[classId];
00554 }

```

5.6.3.8 getDataClasses()

```

vector< string > YOLO_model::getDataClasses (
    std::string labelsFilename = "../data/model/labels.txt" )

```

Function to call to read the class names from the labels file and save them inside [classNames](#) attribute of the class.

Parameters

<i>labelsFilename</i>	a string with the path of the labels file
-----------------------	---

Returns

A std::vector of strings containing the labels saved in [classNames](#)

See also

[getDetectionsClassNames\(Detections detections\)](#)

Author

Marco Annunziata

Definition at line 519 of file [marco_annunziata.cpp](#).

```
00519
00520     vector<string> dataClasses;
00521     ifstream ifs(labelsFilename);
00522     string line;
00523     while (getline(ifs, line))
00524         dataClasses.push_back(line);
00525     this->classNames = dataClasses;
00526     return dataClasses;
00527 }
```

5.6.3.9 getDetections()

```
std::vector< Detection > YOLO_model::getDetections ( )
```

[detections](#) getter

Author

Marco Annunziata

Definition at line 610 of file [marco_annunziata.cpp](#).

```
00610
00611     return this->detections;
00612 }
```

5.6.3.10 getDetectionsClassNames()

```
vector< string > YOLO_model::getDetectionsClassNames (
    Detections detections )
```

Returns the class names of the corresponding id's given by the [Detections](#) struct given in input.

In order to find names, [classNames](#) is used, meaning it must not be empty.

Attention

Function to call only when class names have already been read from the file, meaning [getDataClasses\(string labelsFilename\)](#) has to be called first

Parameters

<i>detections</i>	a Detections struct containing all detections
-------------------	---

Returns

A std::vector of strings containing the corresponding names

See also

[getDataClasses\(string labelsFilename\)](#)

Author

Marco Annunziata

Definition at line 537 of file [marco_annunziata.cpp](#).

```
00537
00538     std::vector<string> detectionsClassNames;
00539     for(int id: detections.classIds){
00540         detectionsClassNames.push_back(this->classNames[id]);
00541     }
00542     return detectionsClassNames;
00543 }
```

5.6.3.11 getModelName()

```
string YOLO_model::getModelName ( )
```

[modelName](#) getter

Author

Marco Annunziata

Definition at line 603 of file [marco_annunziata.cpp](#).

```
00603
00604     return this->modelName;
00605 }
```

5.6.3.12 isAvailableGPU()

```
bool YOLO_model::isAvailableGPU ( )
```

Checks if GPU providers are available and updates the class with this information.

Returns

a boolean which is true if a GPU provider is available.

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

Definition at line 104 of file [marco_annunziata.cpp](#).

```
00104
00105     // Ottieni la lista dei provider compilati nel tuo ONNX Runtime
00106     std::vector<std::string> providers = Ort::GetAvailableProviders();
00107
00108     for (const auto& p : providers) {
00109         if (p == "CUDAXecutionProvider" || p == "DmlExecutionProvider") {
00110             this->usingGPU = true;
00111             return true;
00112         }
00113     }
00114     this->usingGPU = false;
00115     return false;
00116 }
```


5.6.3.13 logger()

```
void YOLO_model::logger (
    void * param,
    OrtLoggingLevel severity,
    const char * category,
    const char * logid,
    const char * code_location,
    const char * message ) [static], [private]
```

Logger function used by the constructor that handles the creation of the ONNX Runtime environment.

Parameters

<i>param</i>	
<i>severity</i>	
<i>category</i>	
<i>logid</i>	
<i>code_location</i>	
<i>message</i>	

See also

[YOLO_model::YOLO_model\(\)](#)

Author

Marco Annunziata

Definition at line 64 of file [marco_annunziata.cpp](#).

```
00065     {
00066         static std::mutex logMutex;
00067         static std::ofstream logFile("./onnxruntime.log", std::ios::out | std::ios::app);
00068
00069         const char* severityStr = "";
00070         switch (severity) {
00071             case ORT_LOGGING_LEVEL_VERBOSE: severityStr = "VERBOSE"; break;
00072             case ORT_LOGGING_LEVEL_INFO:    severityStr = "INFO";    break;
00073             case ORT_LOGGING_LEVEL_WARNING: severityStr = "WARNING"; break;
00074             case ORT_LOGGING_LEVEL_ERROR:   severityStr = "ERROR";   break;
00075             case ORT_LOGGING_LEVEL_FATAL:   severityStr = "FATAL";   break;
00076             default:                         severityStr = "UNKNOWN"; break;
00077         }
00078
00079         // concatenating the output into a single string buffer to write into the log file
00080         std::ostringstream os;
00081         os << "[" << (logid ? logid : "no-logid") << "]" << " "
00082            << severityStr << " | " << (category ? category : "") << " @ "
00083            << (code_location ? code_location : "") << "\n" << " "
00084            << (message ? message : "") << "\n";
00085
00086         std::lock_guard<std::mutex> lock(logMutex);
00087
00088         if (severity <= ORT_LOGGING_LEVEL_INFO) {
00089             // INFO and VERBOSE go to file
00090             if (logFile) {
00091                 logFile << os.str();
00092                 logFile.flush();
00093             }
00094         } else {
00095             // WARNING, ERROR, FATAL go to stderr
00096             std::cerr << os.str();
00097         }
00098     }
```

5.6.3.14 mergeDetections()

```
void YOLO_model::mergeDetections (
    Detections & dest,
    const Detections & source )
```

Merges two [Detections](#) structs together in the first struct passed.

Useful for merging detections of different tiles of the same image.

Parameters

<i>dest</i>	pointer to the Detections struct used as destination
<i>source</i>	pointer to the Detections struct to merge

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

Author

Marco Annunziata

Definition at line 218 of file [marco_annunziata.cpp](#).

```
00218                                     {
00219     if(source.boundingBoxes.empty()) return;
00220
00221     size_t newSize = dest.boundingBoxes.size() + source.boundingBoxes.size();
00222
00223     dest.boundingBoxes.reserve(newSize);
00224     dest.classConfidences.reserve(newSize);
00225     dest.classIds.reserve(newSize);
00226
00227     dest.boundingBoxes.insert(dest.boundingBoxes.end(), source.boundingBoxes.begin(),
00228     source.boundingBoxes.end());
00228     dest.classConfidences.insert(dest.classConfidences.end(), source.classConfidences.begin(),
00229     source.classConfidences.end());
00229     dest.classIds.insert(dest.classIds.end(), source.classIds.begin(), source.classIds.end());
00230
00231 }
```

5.6.3.15 setModelName()

```
void YOLO_model::setModelName (
    std::string modelName )
```

[modelName](#) setter

Author

Marco Annunziata

Definition at line 596 of file [marco_annunziata.cpp](#).

```
00596                                     {
00597     this->modelName = modelName;
00598 }
```

5.6.4 Member Data Documentation

5.6.4.1 allocator

```
Ort::AllocatorWithDefaultOptions YOLO_model::allocator [private]
```

Definition at line 34 of file [marco_annunziata.hpp](#).

5.6.4.2 CLASS_CONFIDENCE_THRESHOLD

```
const float YOLO_model::CLASS_CONFIDENCE_THRESHOLD = 0.5 [private]
```

Definition at line 40 of file [marco_annunziata.hpp](#).

5.6.4.3 classNames

```
std::vector<std::string> YOLO_model::classNames [private]
```

Definition at line 39 of file [marco_annunziata.hpp](#).

5.6.4.4 detections

```
std::vector<Detection> YOLO_model::detections [private]
```

Definition at line 37 of file [marco_annunziata.hpp](#).

5.6.4.5 env

```
Ort::Env YOLO_model::env [private]
```

Definition at line 30 of file [marco_annunziata.hpp](#).

5.6.4.6 modelName

```
std::string YOLO_model::modelName = "Yolo" [private]
```

Definition at line 38 of file [marco_annunziata.hpp](#).

5.6.4.7 NMS_THRESHOLD

```
const float YOLO_model::NMS_THRESHOLD = 0.5 [private]
```

Definition at line 41 of file [marco_annunziata.hpp](#).

5.6.4.8 session

```
Ort::Session YOLO_model::session [private]
```

Definition at line 32 of file [marco_annunziata.hpp](#).

5.6.4.9 sessionOptions

```
Ort::SessionOptions YOLO_model::sessionOptions [private]
```

Definition at line 33 of file [marco_annunziata.hpp](#).

5.6.4.10 usingGPU

```
bool YOLO_model::usingGPU = false [private]
```

Definition at line 35 of file [marco_annunziata.hpp](#).

5.6.4.11 YOLO_TARGET_INPUT_SIZE

```
const int YOLO_model::YOLO_TARGET_INPUT_SIZE = 640 [private]
```

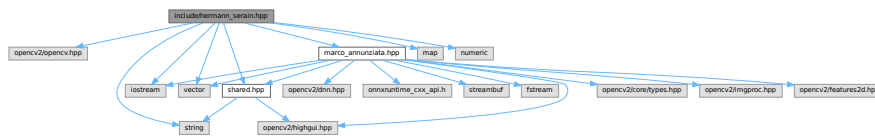
Definition at line 36 of file [marco_annunziata.hpp](#).

The documentation for this class was generated from the following files:

- include/[marco_annunziata.hpp](#)
- src/[marco_annunziata.cpp](#)

File Documentation

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <numeric>
#include "shared.hpp"
#include "marco_annunziata.hpp"
Include dependency graph for hermann_serain.hpp:
```



```

graph BT
    A[include/hermann_serain.hpp]
    B[src/finalProject.cpp]
    C[src/hermann_serain.cpp]
    D[src/sveva_turola.cpp]
    B --> A
    C --> A
    D --> A
  
```

- struct **UserData**
- struct **MetricsCounter**

Functions

- [UserData readInput](#) (int argc, char **argv)
- void [computeImageMetrics](#) (string imagePath, std::vector< [Detection](#) > predictedDetections)
- void [initObjectsForVideoMetrics](#) (string videoPath)
- void [computeVideoMetrics](#) (std::vector< [Detection](#) > predictedDetections, int frameCount)
- void [printFinalVideoMetrics](#) ()

6.1.1 Function Documentation

6.1.1.1 computeImageMetrics()

```
void computeImageMetrics (
    string imagePath,
    std::vector< Detection > predictedDetections )
```

Definition at line 437 of file [hermann_serain.cpp](#).

```
00437                                     {
00438     if (!canComputeImageMetrics(imagePath)) {
00439         return;
00440     }
00441
00442     fs::path p(imagePath);
00443     string filename = p.stem().string();
00444     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00445
00446     cv::Mat img = cv::imread(imagePath);
00447     if (img.empty()) {
00448         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00449         return;
00450     }
00451
00452     // Extraction of ground truth using the helper method
00453     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00454
00455     if (!actualDetections.empty()) {
00456         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00457         printFinalMetrics(results);
00458     }
00459 }
```

6.1.1.2 computeVideoMetrics()

```
void computeVideoMetrics (
    std::vector< Detection > predictedDetections,
    int frameCount )
```

Definition at line 487 of file [hermann_serain.cpp](#).

```
00487                                     {
00488     if (!g_videoMetricsEnabled || !g_motFile.is_open()) return;
00489
00490     std::vector<Detection> actualDetections;
00491     string line;
00492
00493     // If we have a cached row from the previous frame, we start from that one
00494     if (!g_lastLine.empty()) {
00495         line = g_lastLine;
00496         g_lastLine = "";
00497     } else if (!getline(g_motFile, line)) {
00498         return; // End of file
00499     }
00500
00501     while (true) {
00502         stringstream ss(line);
00503         string token;
00504         vector<string> tokens;
00505
00506         // Fast line parsing (CSV)
00507         while (getline(ss, token, ',')) tokens.push_back(token);
```

```

00508
00509         if (tokens.size() >= 8) {
00510             int currentLineFrame = stoi(tokens[0]);
00511
00512             if (currentLineFrame == frameCount) {
00513                 Detection d;
00514                 d.boundingBox = cv::Rect(stof(tokens[2]), stof(tokens[3]), stof(tokens[4]),
stof(tokens[5]));
00515                 d.classId = stoi(tokens[7]) - 1;
00516                 d.classConfidence = 1.0f;
00517                 actualDetections.push_back(d);
00518             }
00519             else if (currentLineFrame > frameCount) {
00520                 g_lastLine = line;
00521                 break;
00522             }
00523         }
00524
00525         if (!getline(g_motFile, line)) break;
00526     }
00527     MetricsCounter frameResults = updateMetricsCounters(predictedDetections, actualDetections);
00528
00529     g_cumulativeVideoMetrics.tp += frameResults.tp;
00530     g_cumulativeVideoMetrics.fp += frameResults.fp;
00531     g_cumulativeVideoMetrics.fn += frameResults.fn;
00532 }

```

6.1.1.3 initObjectsForVideoMetrics()

```

void initObjectsForVideoMetrics (
    string videoPath )

```

Definition at line 469 of file [hermann_serain.cpp](#).

```

00469 {
00470     if (g_motFile.is_open()) g_motFile.close();
00471     g_videoMetricsEnabled = false;
00472     g_cumulativeVideoMetrics = MetricsCounter();
00473     g_lastLine = "";
00474
00475     if (!canComputeVideoMetrics(videoPath)) return;
00476
00477     fs::path p(videoPath);
00478     string labelPath = "../data/test/ground_truths/videos/" + p.stem().string() + ".txt";
00479
00480     g_motFile.open(labelPath);
00481     if (g_motFile.is_open()) {
00482         g_videoMetricsEnabled = true;
00483         cout << "[METRICS] FMOT file opened for sequential reading." << endl;
00484     }
00485 }

```

6.1.1.4 printFinalVideoMetrics()

```

void printFinalVideoMetrics ( )

```

Definition at line 535 of file [hermann_serain.cpp](#).

```

00535 {
00536     if (!g_videoMetricsEnabled) {
00537         cout << "[METRICS] No data accumulated for this video." << endl;
00538         return;
00539     }
00540
00541     cout << "\n--- FINAL CUMULATIVE VIDEO STATISTICS ---" << endl;
00542     printFinalMetrics(g_cumulativeVideoMetrics);
00543
00544     if (g_motFile.is_open()) {
00545         g_motFile.close();
00546     }
00547
00548     g_videoMetricsEnabled = false;
00549     g_lastLine = "";
00550     g_cumulativeVideoMetrics.tp = 0;
00551     g_cumulativeVideoMetrics.fp = 0;
00552     g_cumulativeVideoMetrics.fn = 0;
00553
00554     cout << "[INFO] Metrics state reset for the next session." << endl;
00555 }

```

6.1.1.5 readInput()

```
UserData readInput (
    int argc,
    char ** argv )
```

Definition at line 63 of file [hermann_serain.cpp](#).

```
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG_VIDEO) {
00073                 result.data_path = DEFAULT_VIDEO_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG_IMAGE) {
00077                 result.data_path = DEFAULT_IMAGE_PATH;
00078                 result.choice = Choice::File;
00079             }
00080         }
00081     }
00082
00083     if(result.choice != Choice::Invalid)
00084         return result;
00085
00086     //NOTE: I'm gonna read the user input from now on
00087     string choiceStr;
00088
00089     // NOTE: Let's keep asking if the input is invalid
00090     while (result.choice == Choice::Invalid)
00091     {
00092         cout << "-----\n";
00093         cout << " C = Camera | F = File | H = Help\n";
00094         cout << "-----\n";
00095         cout << "Write your choice: ";
00096
00097         cin >> choiceStr;
00098         result.choice = parseChoice(choiceStr);
00099
00100         switch (result.choice)
00101         {
00102             case Choice::Camera:
00103             {
00104                 handleCamera();
00105             }
00106             break;
00107
00108             case Choice::File:
00109             {
00110                 result.data_path = handleFile();
00111             }
00112             break;
00113
00114             case Choice::Help:
00115             {
00116                 printHelp();
00117                 result.choice = Choice::Invalid; //NOTE: continuing loop
00118             }
00119             break;
00120
00121             case Choice::Invalid:
00122             {
00123                 cout << "Usage: enter C or F (or H for help)!\n";
00124             }
00125         }
00126     }
00127
00128     return result;
00129 }
```

6.2 hermann_serain.hpp

[Go to the documentation of this file.](#)


```

00001 #ifndef HERMANN_SERAIN_HPP
00002 #define HERMANN_SERAIN_HPP
00003
00004 #include <opencv2/opencv.hpp>
00005 #include <iostream>
00006 #include <vector>
00007 #include <map>
00008 #include <string>
00009 #include <numeric>
00010
00011 #include "shared.hpp"
00012 #include "marco_annunziata.hpp"
00013
00014 using namespace cv;
00015 using namespace std;
00016 using namespace Shared;
00017
00018 //NOTE: Contains the user choices
00019 struct UserData {
00020     Choice choice;
00021     string data_path;
00022
00023     UserData(Choice _choice, string _data_path)
00024         : choice(_choice), data_path(_data_path) {}
00025 };
00026
00027 struct MetricsCounter {
00028     int tp = 0; // True Positives
00029     int fp = 0; // False Positives
00030     int fn = 0; // False Negatives
00031 };
00032
00033 //NOTE: Public methods
00034 UserData readInput(int argc, char** argv);
00035 void computeImageMetrics(string imagePath, std::vector<Detection> predictedDetections);
00036 void initObjectsForVideoMetrics(string videoPath);
00037 void computeVideoMetrics(std::vector<Detection> predictedDetections, int frameCount);
00038 void printFinalVideoMetrics();
00039
00040 #endif

```

6.3 include/marco_annunziata.hpp File Reference

```

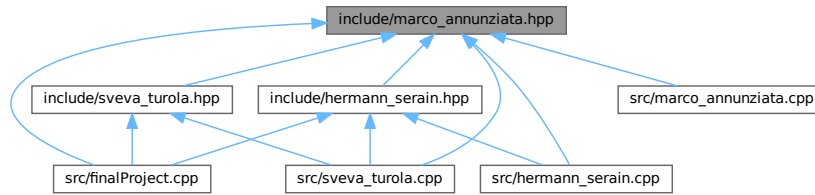
#include <opencv2/core/types.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/features2d.hpp>
#include <opencv2/dnn.hpp>
#include <onnxruntime_cxx_api.h>
#include "shared.hpp"
#include <iostream>
#include <streambuf>
#include <vector>
#include <fstream>

```

Include dependency graph for marco_annunziata.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Detection](#)
- struct [Detections](#)
- class [YOLO_model](#)

6.4 marco_annunziata.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef MARCO_ANNUNZIATA_HPP
00002 #define MARCO_ANNUNZIATA_HPP
00003
00004 #include <opencv2/core/types.hpp>
00005 #include <opencv2/highgui.hpp>
00006 #include <opencv2/imgproc.hpp>
00007 #include <opencv2/features2d.hpp>
00008 #include <opencv2/dnn.hpp>
00009 #include <onnxruntime_cxx_api.h>
00010 #include "shared.hpp"
00011 #include <iostream>
00012 #include <streambuf>
00013 #include <vector>
00014 #include <fstream>
00015
00016 struct Detection{
00017     cv::Rect boundingBox = cv::Rect(0,0,0,0);
00018     float classConfidence = 0;
00019     int classId = -1;
00020 };
00021
00022 struct Detections{
00023     std::vector<cv::Rect> boundingBoxes;
00024     std::vector<float> classConfidences;
00025     std::vector<int> classIds;
00026 };
00027
00028 class YOLO_model{
00029     private:
00030         Ort::Env env;
00031         static void logger(void* param, OrtLoggingLevel severity, const char*
category, const char* logid, const char* code_location, const char* message);
00032         Ort::Session session;
00033         Ort::SessionOptions sessionOptions;
00034         Ort::AllocatorWithDefaultOptions allocator;
00035         bool usingGPU = false;
00036         const int YOLO_TARGET_INPUT_SIZE = 640; // MUST be multiple of 32. See
YOLO_model::detectObjects implementation.
00037         std::vector<Detection> detections;
00038         std::string modelName = "Yolo";
00039         std::vector<std::string> classNames;
00040         const float CLASS_CONFIDENCE_THRESHOLD = 0.5;
00041         const float NMS_THRESHOLD = 0.5;
00042     public:
00043         YOLO_model();
00044         bool isAvailableGPU();
00045         std::vector<Detection> detectionPipeline(cv::Mat &img, bool enable_letterbox_padding=true);
00046         void clearDetections();

```

```

00047     Detections          detect(const cv::Mat &img, bool enable_letterbox_padding=true);
00048     void                mergeDetections(Detections& dest, const Detections& source);
00049     std::vector<std::string> getDataClasses(std::string
labelsFilename="../data/model/labels.txt");
00050     std::vector<Detection> filterDetectionsNMS(Detections goodDetections);
00051     cv::Mat              drawBoundingBoxes(cv::Mat &img, std::vector<Detection> &detections,
cv::Scalar color);
00052     cv::Mat              drawBoundingBoxes(cv::Mat &img, cv::Scalar color);
00053     void                setModelName(std::string modelName);
00054     std::string          getModelName();
00055     std::vector<Detection> getDetections();
00056     std::vector<std::string> getDetectionsClassNames(Detections detections);
00057     std::string          getClassName(int classId);
00058 };
00059
00060 #endif

```

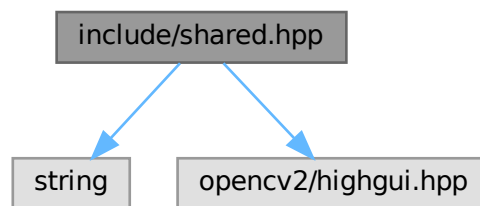
6.5 include/shared.hpp File Reference

```

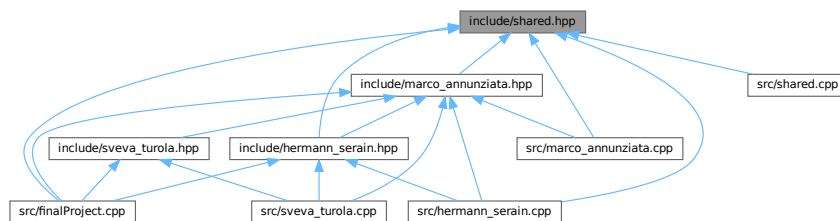
#include <string>
#include <opencv2/highgui.hpp>

```

Include dependency graph for shared.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [Shared](#)

Enumerations

- enum class [Shared::Choice](#) { [Shared::Camera](#) , [Shared::File](#) , [Shared::Invalid](#) , [Shared::Help](#) }

Functions

- int [frameCapture](#) (string data_path, string labels_path)
Initializes the video source and manages the frame extraction process.
- int [processStream](#) (string data_path, VideoCapture cap, VideoWriter out, Mat frame, int savedCount, string labels_path)
Manages the main video processing loop, including detection and frame saving.
- void [playOutputVideo](#) (string path, double fps)
Opens and reproduces the processed video file.
- double [getDistance](#) (const Rect &r1, const Rect &r2)
Calculates the distance between the centers of two rectangles.
- Mat [cardValues](#) (vector< [Detection](#) > detections, [YOLO_model](#) &model, Mat &frame)
Processes detections to group card symbols and apply the Hi-Lo counting system.
- void [drawCardGroup](#) (Mat &img, const vector< [Detection](#) > &list, Scalar color, bool filled, [YOLO_model](#) &model)
Draws a group of detected cards on the image.
- void [drawGameStatus](#) (Mat &frame, const vector< [Detection](#) > &green, const vector< [Detection](#) > &red)
Calculates and displays the blackjack running count and game status.

6.7.1 Function Documentation

6.7.1.1 cardValues()

```
Mat cardValues (
    vector< Detection > detections,
    YOLO\_model & model,
    Mat & frame )
```

Processes detections to group card symbols and apply the Hi-Lo counting system.

This function analyzes the detections and performs several steps:

- Groups symbols of the same class that are physically close into a single "full card" box.
- Ensures each card box meets a minimum size and remains centered.
- Assigns a color to each card based on the Hi-Lo system.
- Creates a visual overlay on the frame with colored boxes and the game status.

Parameters

<i>detections</i>	is a vector of all objects found by the YOLO model.
<i>model</i>	is the reference to the YOLO model.
<i>frame</i>	is the current video frame to be processed.

Returns

Returns a new Mat frame with all visual detections and the running count drawn on it.

Author

Sveva Turola

Definition at line 267 of file sveva_turola.cpp.

```

00267
00268     vector<Detection> green, blue, red;
00269
00270     // groups the detections by value
00271     map<int, vector<Detection> > classGroups;
00272     for(const Detection& d : detections){
00273         classGroups[d.classId].push_back(d);
00274     }
00275
00276     // iterates on each group of classGroups
00277     for(auto& [id, dets] : classGroups){
00278         // used to mark the symbols already processed
00279         vector<bool> processed(dets.size(), false);
00280
00281         for (size_t i = 0; i < dets.size(); i++){
00282             if (processed[i]){
00283                 continue;
00284             }
00285
00286             // takes the first detection not already processed
00287             Detection fullCard = dets[i];
00288             processed[i] = true;
00289
00290             // searches among the other symbols of the same group
00291             for (size_t j = i + 1; j < dets.size(); j++){
00292                 // sets a dynamic threshold based on the dimension of the diagonal
00293                 float diag = sqrt(pow(fullCard.boundingBox.width, 2) +
pow(fullCard.boundingBox.height, 2));
00294                 float threshold = min(diag * 4.5f, 350.0f);
00295
00296                 // if it finds another identical and very close symbol, then they belong to the same
00297                 card if (!processed[j] && getDistance(dets[i].boundingBox, dets[j].boundingBox) <
threshold){
00298                     // draws the bounding box of the entire card based on the coordinates of the
00299                     detections int minX = min(fullCard.boundingBox.x, dets[j].boundingBox.x);
00300                     int minY = min(fullCard.boundingBox.y, dets[j].boundingBox.y);
00301                     int maxX = max(fullCard.boundingBox.x + fullCard.boundingBox.width,
dets[j].boundingBox.x + dets[j].boundingBox.width);
00302                     int maxY = max(fullCard.boundingBox.y + fullCard.boundingBox.height,
dets[j].boundingBox.y + dets[j].boundingBox.height);
00303
00304                     Rect mergedBox(minX, minY, maxX - minX, maxY - minY);
00305
00306                     // normalization of the dimension of the bounding box
00307                     int minSize = 100;
00308
00309                     // used when the width of the bounding box is less than minSize
00310                     if (mergedBox.width < minSize) {
00311                         int diff = minSize - mergedBox.width;
00312                         // updates the x to the left to preserve the center
00313                         mergedBox.x -= diff / 2;
00314                         mergedBox.width = minSize;
00315                     }
00316
00317                     // used when the height of the bounding box is less than minSize
00318                     if (mergedBox.height < minSize) {
00319                         int diff = minSize - mergedBox.height;
00320                         // updates the y up to preserve the center
00321                         mergedBox.y -= diff / 2;
00322                         mergedBox.height = minSize;
00323                     }
00324
00325                     // updates the bounding box of fullCard
00326                     fullCard.boundingBox = mergedBox;
00327                     processed[j] = true;
00328                 }
00329             }
00330
00331             // Hi-Lo system
00332             char cardNumber = model.getClassName(id)[0];
00333
00334             if(cardNumber >= '2' && cardNumber <= '6'){
00335                 green.push_back(fullCard);
00336             } else if(cardNumber >= '7' && cardNumber <= '9'){
00337                 blue.push_back(fullCard);
00338             } else {
00339                 red.push_back(fullCard);
00340             }
00341         }

```

```

00342     }
00343
00344     Mat outputFrame = frame.clone();
00345
00346     // draws original bounding boxes using white
00347     if(!detections.empty()){
00348         Mat detectionOverlay = outputFrame.clone();
00349         detectionOverlay = model.drawBoundingBoxes(detectionOverlay, detections, Scalar(255, 255,
00350 255));
00351
00352         addWeighted(detectionOverlay, 0.7, outputFrame, 0.3, 0, outputFrame);
00353     }
00354
00355     // draws the bounding box for the complete card
00356     drawCardGroup(outputFrame, green, Scalar(0, 255, 0), false, model);
00357     drawCardGroup(outputFrame, blue, Scalar(255, 0, 0), false, model);
00358     drawCardGroup(outputFrame, red, Scalar(0, 0, 255), false, model);
00359
00360     // fills the bounding box with the respective color
00361     Mat overlay = outputFrame.clone();
00362
00363     drawCardGroup(overlay, green, Scalar(0, 255, 0), true, model);
00364     drawCardGroup(overlay, blue, Scalar(255, 0, 0), true, model);
00365     drawCardGroup(overlay, red, Scalar(0, 0, 255), true, model);
00366
00367     addWeighted(overlay, 0.3, outputFrame, 0.7, 0, outputFrame);
00368
00369     // computes and shows the running count
00370     drawGameStatus(outputFrame, green, red);
00371
00372     return outputFrame;
00373 }

```

6.7.1.2 drawCardGroup()

```

void drawCardGroup (
    Mat & img,
    const vector< Detection > & list,
    Scalar color,
    bool filled,
    YOLO_model & model )

```

Draws a group of detected cards on the image.

This function draws bounding boxes and labels for a list of detections. It automatically adjusts the thickness of lines and the size of the text based on the image resolution. The labels are centered and placed below each box.

Parameters

<i>img</i>	is the image where the bounding boxes will be drawn.
<i>list</i>	is the list of card detections to show.
<i>color</i>	is the color used for the boxes and text.
<i>filled</i>	is a boolean and if is true, the boxes will be filled with color, otherwise only the outline is drawn.
<i>model</i>	is the reference to the YOLO model.

Author

Sveva Turola

Definition at line 386 of file [sveva_turola.cpp](#).

```

00386 {
00387     // dynamic thickness
00388     int thickness = max(1, int(max(img.rows, img.cols) / 640));
00389
00390     // defines the thickness of the box: filled or thick line

```

```

00391     int boxThickness;
00392     if(filled == true){
00393         boxThickness = FILLED;
00394     } else {
00395         boxThickness = 2 * thickness;
00396     }
00397
00398     // iterates through all detections in list
00399     for(size_t i = 0; i < list.size(); i++) {
00400         Rect box = list[i].boundingBox;
00401         rectangle(img, box, color, boxThickness);
00402
00403         string label = model.getClassName(list[i].classId);
00404
00405         double fontScale = 0.5 * thickness;
00406         int fontThickness = max(1, int(1.5 * thickness));
00407         int baseLine = 0;
00408
00409         // measures text dimension
00410         Size textSize = getTextSize(label, FONT_HERSHEY_SIMPLEX, fontScale, fontThickness, &baseLine);
00411
00412         // defines text position: centered w.r.t. x and below the bottom edge
00413         int textX = box.x + (box.width - textSize.width) / 2;
00414         int textY = box.y + box.height + textSize.height + (10 * thickness);
00415
00416         Point textPosition(textX, textY);
00417
00418         putText(img, label, textPosition, FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 *
00419                 thickness);
00420     }

```

6.7.1.3 drawGameStatus()

```

void drawGameStatus (
    Mat & frame,
    const vector< Detection > & green,
    const vector< Detection > & red )

```

Calculates and displays the blackjack running count and game status.

This function computes the "Running Count" by comparing green (low value) and red (high value) cards. It draws a semi-transparent dashboard on the frame showing the current count and color-coded status (Green for player advantage, Red for house advantage). If the count is high (+2 or more), it displays an additional alert message to monitor the game.

Parameters

<i>frame</i>	is the video frame where the status will be drawn.
<i>green</i>	is the vector of detections for low-value cards.
<i>red</i>	is the vector of detections for high-value cards.

Author

Sveva Turola

Definition at line 433 of file [sveva_turola.cpp](#).

```

00433     {
00434         // computes running count
00435         int runningCount = (int)green.size() - (int)red.size();
00436
00437         // setup to visualize the running count
00438         int xPos = 20;
00439         int yPos = 20;
00440         int width = 300;
00441         int height = 70;
00442
00443         // draws the background for the running count

```



```

00444     Mat overlay;
00445     frame.copyTo(overlay);
00446     rectangle(overlay, Rect(xPos, yPos, width, height), Scalar(0, 0, 0), FILLED);
00447     addWeighted(overlay, 0.5, frame, 0.5, 0, frame);
00448
00449     // draws the border for the background
00450     rectangle(frame, Rect(xPos, yPos, width, height), Scalar(255, 255, 255), 2);
00451
00452     // defines the color and the string
00453     Scalar countColor = Scalar(255, 255, 255); // using white for running count = 0
00454     string prefix = "";
00455     string statusText = "Neutral";
00456
00457     if (runningCount > 0) {
00458         // using green for player's advantage
00459         countColor = Scalar(0, 255, 0);
00460         prefix = "+";
00461         statusText = "Player Advantage";
00462     } else if (runningCount < 0) {
00463         // using red for house's advantage
00464         countColor = Scalar(0, 0, 255);
00465         statusText = "House Advantage";
00466     }
00467
00468     string countStr = "Count: " + prefix + to_string(runningCount);
00469
00470     // writes running count
00471     putText(frame, countStr, Point(xPos + 20, yPos + 45), FONT_HERSHEY_SIMPLEX, 1.2, countColor, 2);
00472
00473     if (runningCount >= 2) {
00474         // in this case, the count is high and this indicates an advantage for the players
00475         int alertY = yPos + height + 10;
00476
00477         // defines the alert rectangle
00478         Rect alertBox;
00479         alertBox.x = xPos;
00480         alertBox.y = alertY;
00481         alertBox.width = width;
00482         alertBox.height = 50;
00483
00484         Mat overlay;
00485         frame.copyTo(overlay);
00486
00487         rectangle(overlay, alertBox, Scalar(0, 0, 255), FILLED);
00488
00489         addWeighted(overlay, 0.7, frame, 0.3, 0, frame);
00490
00491         Point textPosition;
00492         textPosition.x = alertBox.x + 10;
00493         textPosition.y = alertBox.y + 35;
00494
00495         // warns to keep an eye on the players' moves
00496         putText(frame, "Monitor players' moves!", textPosition, FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255,
255, 255), 1);
00497     }
00498 }

```

6.7.1.4 frameCapture()

```

int frameCapture (
    string data_path,
    string labels_path )

```

Initializes the video source and manages the frame extraction process.

This function sets up the environment for video analysis. It creates a fresh output directory for frames, removing any previous content. Depending on the input, it opens either a camera (if "0") or a video file. For video files, it also initializes specific objects for tracking metrics before starting the stream processing pipeline.

Parameters

<i>data_path</i>	is the path to the video file or "0" to use the camera.
<i>labels_path</i>	is the path to the file containing class labels.

Returns

Returns 0 on success, or -1 if the video source fails to open.

Author

Sveva Turola

Definition at line 35 of file [sveva_turola.cpp](#).

```

00035                                     {
00036     fs::path p(data_path);
00037     string name = p.stem().string() + "_frames/";
00038     string outputDir = "../output/" + name; //output directory
00039     Mat frame;
00040     int savedCount = 0;
00041     VideoCapture cap;
00042     VideoWriter out;
00043
00044     // creation of output directory: at every run, creates a new directory, in order to have only the
frames for that run
00045     if (fs::exists(outputDir)){
00046         fs::remove_all(outputDir);
00047     }
00048     fs::create_directory(outputDir);
00049
00050     if(data_path == "0"){
00051         // if camera was selected
00052         cap.open(0);
00053
00054         if (!cap.isOpened()) {
00055             cerr << "Error opening camera!" << endl;
00056             return -1;
00057         }
00058
00059         cout << "Loading camera video..." << endl;
00060         savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00061     } else {
00062         // if video file was selected
00063         cap.open(data_path);
00064
00065         if (!cap.isOpened()) {
00066             cerr << "Error opening video!" << endl;
00067             return -1;
00068         }
00069
00070         cout << "Loading video..." << endl;
00071         initObjectsForVideoMetrics(data_path);
00072         savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00073         printFinalVideoMetrics();
00074     }
00075
00076     cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00077     cout << "Video saved in output directory" << endl;
00078
00079     return 0;
00080 }
```

6.7.1.5 getDistance()

```

double getDistance (
    const Rect & r1,
    const Rect & r2 )
```

Calculates the distance between the centers of two rectangles.

This function finds the center point (x, y) for both input rectangles. It then calculates the Euclidean distance between these two centers.

Parameters

<i>r1</i>	is the first rectangle (Rect object).
<i>r2</i>	is the second rectangle (Rect object).

Returns

Returns the distance between the two centers as a double.

Author

Sveva Turola

Definition at line 244 of file [sveva_turola.cpp](#).

```
00244                                     {
00245     // computes center x and center y for both rectangles
00246     double cx1 = r1.x + (r1.width / 2.0);
00247     double cy1 = r1.y + (r1.height / 2.0);
00248     double cx2 = r2.x + (r2.width / 2.0);
00249     double cy2 = r2.y + (r2.height / 2.0);
00250
00251     // returns euclidean distance
00252     return sqrt(pow(cx2 - cx1, 2) + pow(cy2 - cy1, 2));
00253 }
```

6.7.1.6 playOutputVideo()

```
void playOutputVideo (
    string path,
    double fps )
```

Opens and reproduces the processed video file.

This function opens the video located at the given path and displays it in a fullscreen window. It calculates the correct playback speed based on the original FPS. Users can control the playback using the keyboard:

- Press 'q' to quit the video.
- Press 'p' to pause the video.

Parameters

<i>path</i>	is the file path of the video to be played.
<i>fps</i>	is the number of frames per second used to set the playback speed.

Author

Sveva Turola

Definition at line 195 of file [sveva_turola.cpp](#).

```
00195                                     {
00196     // opens the output video detections.mp4
00197     VideoCapture playback;
00198     playback.open(path);
00199
00200     if (!playback.isOpened()) {
00201         cerr << "Error in opening " << path << endl;
00202         return;
00203     }
00204
00205     namedWindow("Final video", WINDOW_NORMAL);
00206     setWindowProperty("Final video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00207
00208     Mat playFrame;
00209
00210     cout << "Reproduction of the output video. Press 'q' to close it or 'p' to pause it." << endl;
00211     while (playback.read(playFrame)) {
00212         // shows the output video
```

```

00213         imshow("Final video", playFrame);
00214
00215         // computes the delay for the playback of the video
00216         int delay = 30;
00217
00218         if(fps > 0){
00219             delay = 1000 / (int) fps;
00220         }
00221
00222         int key = waitKey(delay);
00223         // used to stop the playback of the output video
00224         if (key == 'q') {
00225             break;
00226         } else if (key == 'p'){
00227             // used to pause the playback of the output video
00228             waitKey(0);
00229         }
00230     }
00231
00232     playback.release();
00233     destroyWindow("Final video");
00234 }

```

6.7.1.7 processStream()

```

int processStream (
    string data_path,
    VideoCapture cap,
    VideoWriter out,
    Mat frame,
    int savedCount,
    string labels_path )

```

Manages the main video processing loop, including detection and frame saving.

This function initializes the YOLO11s model and processes the video frame by frame. It performs several key tasks:

- Detects objects at fixed intervals.
- Computes video metrics.
- Identifies card values and draws boxes around them.
- Saves frames as images only when the number of detected cards changes.
- Saves all processed frames into a final MP4 video file.

Parameters

<i>cap</i>	is the VideoCapture object already opened in frameCapture.
<i>out</i>	is the VideoWriter object used to save the output video.
<i>frame</i>	is a Mat object used to store each frame during reading.
<i>savedCount</i>	is the initial number of saved frames.
<i>labels_path</i>	is the path to the configuration file for class names.

Returns

Returns the total number of saved frames, or -1 if the initial frame is empty or the output video cannot be open.

Author

Sveva Turola

Definition at line 99 of file [sveva_turola.cpp](#).

```

00099 {
00100     // initialization of YOLO model
00101     YOLO_model model;
00102     model.setModelName("YOLO11s");
00103     vector<string> dataClasses = model.getDataClasses(labels_path);
00104
00105     // setup of video variables
00106     int frameCount = 0;
00107     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00108     int fps = cap.get(CAP_PROP_FPS);
00109
00110     vector<Detection> detections;
00111
00112     // checks if input is the camera
00113     bool isCamera = (cap.get(CAP_PROP_FRAME_COUNT) <= 0);
00114
00115     cap.read(frame);
00116     if (frame.empty()){
00117         return -1;
00118     }
00119
00120     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00121     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00122
00123     // opens output video
00124     fs::path p(data_path);
00125     string name = p.stem().string();
00126     out.open("../output/" + name + "_detections.mp4", codec, fps, Size(frame_width, frame_height),
00127 true);
00128     if(!out.isOpened()){
00129         cerr << "Could not open the output video file for write\n";
00130         return -1;
00131     }
00132
00133     int cardCount = -1;
00134
00135     cout << "Starting video analysis... Results will be available once the computation is complete." <<
endl;
00136
00137     while (true){
00138         cap.read(frame);
00139
00140         if (frame.empty()){
00141             break;
00142         }
00143
00144         savedCount++;
00145         int key = waitKey(1) & 0xFF;
00146
00147         if (frameCount % (fps/2) == 0){
00148             detections = model.detectionPipeline(frame);
00149             computeVideoMetrics(detections, frameCount+1);
00150         }
00151
00152         frameCount++;
00153         Mat outputFrame = cardValues(detections, model, frame);
00154
00155         int currentCardCount = detections.size();
00156
00157         // saving only relevant frames, that are the frames in which the number of detections changes
00158         if(currentCardCount != cardCount){
00159             string filename = "../output/" + name + "_frames/frame_" + to_string(savedCount) + ".jpg";
00160             imwrite(filename, outputFrame);
00161             cardCount = currentCardCount;
00162         }
00163
00164         // adds each frame to the video
00165         out.write(outputFrame);
00166
00167         if(isCamera == true){
00168             // stops the camera
00169             if (key == 'q') {
00170                 cout << "Closed camera!\n";
00171                 break;
00172             }
00173         }
00174     }
00175
00176     cap.release();
00177     out.release();

```

```

00177     destroyAllWindows();
00178
00179     // calls the method to show the video
00180     playOutputVideo("../output/" + name + "_detections.mp4"), fps);
00181
00182     return savedCount;
00183 }

```

6.8 sveva_turola.hpp

Go to the documentation of this file.

```

00001 #ifndef SVEVA_TUROLA_HPP
00002 #define SVEVA_TUROLA_HPP
00003
00004 #include <opencv2/features2d.hpp>
00005 #include <opencv2/highgui.hpp>
00006 #include <iostream>
00007 #include <string>
00008 #include <vector>
00009 #include <opencv2/core.hpp>
00010 #include <opencv2/imgproc.hpp>
00011 #include <opencv2/calib3d.hpp>
00012
00013 #include "marco_annunziata.hpp"
00014
00015 using namespace std;
00016 using namespace cv;
00017
00018 int frameCapture(string data_path, string labels_path);
00019 int processStream(string data_path, VideoCapture cap, VideoWriter out, Mat frame, int savedCount,
    string labels_path);
00020 void playOutputVideo(string path, double fps);
00021 double getDistance(const Rect& r1, const Rect& r2);
00022 Mat cardValues(vector<Detection> detections, YOLO_model &model, Mat &frame);
00023 void drawCardGroup(Mat& img, const vector<Detection>& list, Scalar color, bool filled, YOLO_model&
    model);
00024 void drawGameStatus(Mat& frame, const vector<Detection>& green, const vector<Detection>& red);
00025
00026 #endif

```

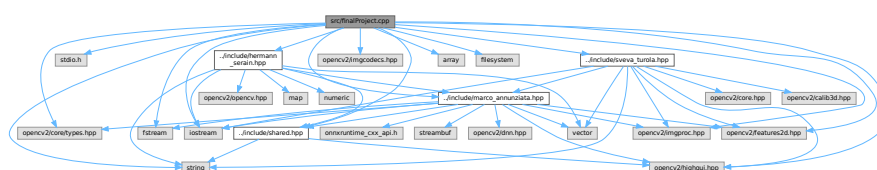
6.9 src/finalProject.cpp File Reference

```

#include <opencv2/core/types.hpp>
#include <stdio.h>
#include <iostream>
#include <string>
#include <fstream>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/imgcodecs.hpp>
#include <opencv2/features2d.hpp>
#include <array>
#include <filesystem>
#include "../include/shared.hpp"
#include "../include/marco_annunziata.hpp"
#include "../include/hermann_serain.hpp"
#include "../include/sveva_turola.hpp"

```

Include dependency graph for finalProject.cpp:



Functions

- int [main](#) (int argc, char **argv)

6.9.1 Function Documentation

6.9.1.1 main()

```
int main (
    int argc,
    char ** argv )
```

Definition at line 25 of file [finalProject.cpp](#).

```
00025     {
00026         string choice = "X";
00027         string data_path;
00028         string labels_path = "../data/model/labels.txt";
00029         enum class fileCategories {IMAGE, VIDEO, UNKNOWN};
00030         auto fileType = fileCategories::UNKNOWN;
00031         string allowedImgType[] = {".png", ".jpg", ".jpeg"};
00032         string allowedVidType[] = {".mp4"};
00033         YOLO_model model;
00034         model.setModelName("YOLO11s");
00035         UserData userData = readInput(argc, argv);
00036
00037         switch (userData.choice)
00038         {
00039             case Choice::Camera:
00040             {
00041                 frameCapture("0", labels_path);
00042             }
00043             break;
00044
00045             case Choice::File:
00046             {
00047                 /* check if file is image */
00048                 for(auto type : allowedImgType){
00049                     if(userData.data_path.rfind(type) != string::npos){
00050                         fileType = fileCategories::IMAGE;
00051                         Mat frame = imread(userData.data_path, cv::IMREAD_COLOR);
00052
00053                         vector<string> dataClasses = model.getDataClasses(labels_path);
00054                         cout << "GPU available? " << (model.isAvailableGPU() ? "YES" : "NO") << endl;
00055                         model.detectionPipeline(frame);
00056                         auto detections = model.getDetections();
00057                         Mat resultImg = cardValues(detections, model, frame);
00058                         string outputDir = "../output/";
00059                         if (!fs::exists(outputDir)) {
00060                             fs::create_directory(outputDir);
00061                         }
00062                         fs::path p(userData.data_path);
00063                         string filename = outputDir + p.stem().string() + "_detections.jpg";
00064                         imwrite(filename, resultImg);
00065                         computeImageMetrics(userData.data_path, detections);
00066                         std::string windowTitle = model.getModelName() + " - " +
00067                             std::to_string(detections.size()) + " detections";
00067                         namedWindow(windowTitle, WINDOW_NORMAL);
00068                         imshow(windowTitle, resultImg);
00069                         waitKey(0);
00070                         break;
00071                     }
00072                 }
00073
00074                 /* check if file is video */
00075                 if(fileType == fileCategories::UNKNOWN){
00076                     for(auto type: allowedVidType){
00077                         if(userData.data_path.rfind(type) != string::npos){
00078                             fileType = fileCategories::VIDEO;
00079                             cout << "Using ONNX Runtime GPU? " << (model.isAvailableGPU() ? "YES" : "NO") <<
00080                             endl;
00081                             frameCapture(userData.data_path, labels_path);
00082                             break;
00083                         }
00084                     }
00085
00086                     /* invalid file */
00087                     if(fileType == fileCategories::UNKNOWN){
```

```

00088             cerr << "The image or video provided has an invalid file type. Please retry with a
different one.\n";
00089             return -1;
00090         }
00091     }
00092 }
00093 }
00094 break;
00095
00096 case Choice::Invalid:
00097 case Choice::Help:
00098 {
00099     //TODO: Verifica se questa parte va bene, di norma in questo caso non può ricaderci
l'utente
00100     throw std::logic_error("Invalid operation: ...");
00101 }
00102 }
00103
00104 // TODO: batch processing
00105 if(labels_path.rfind(".txt") == string::npos){
00106     cerr << "The label file provided has an invalid file type. Please provide a .txt file.\n";
00107     return -1;
00108 }
00109
00110 // destroyAllWindows();
00111 return 0;
00112 }

```

6.10 finalProject.cpp

[Go to the documentation of this file.](#)

```

00001 //includes
00002 #include <opencv2/core/types.hpp>
00003 #include <stdio.h>
00004 #include <iostream>
00005 #include <string>
00006 #include <fstream>
00007 #include <opencv2/highgui.hpp>
00008 #include <opencv2/imgproc.hpp>
00009 #include <opencv2/imgcodecs.hpp>
00010 #include <opencv2/features2d.hpp>
00011 #include <array>
00012 #include <filesystem>
00013
00014 //libs
00015 #include "../include/shared.hpp"
00016 #include "../include/marco_annunziata.hpp"
00017 #include "../include/hermann_serain.hpp"
00018 #include "../include/sveva_turolo.hpp"
00019
00020 using namespace std;
00021 using namespace cv;
00022 using namespace Shared;
00023 namespace fs = std::filesystem;
00024
00025 int main(int argc, char** argv){
00026     string choice = "X";
00027     string data_path;
00028     string labels_path = "../data/model/labels.txt";
00029     enum class fileCategories {IMAGE, VIDEO, UNKNOWN};
00030     auto fileType = fileCategories::UNKNOWN;
00031     string allowedImgType[] = {".png", ".jpg", ".jpeg"};
00032     string allowedVidType[] = {".mp4"};
00033     YOLO_model model;
00034     model.setModelName("YOLO11s");
00035     UserData userData = readInput(argc, argv);
00036
00037     switch (userData.choice)
00038     {
00039         case Choice::Camera:
00040         {
00041             frameCapture("0", labels_path);
00042         }
00043         break;
00044
00045         case Choice::File:
00046         {
00047             /* check if file is image */
00048             for(auto type : allowedImgType){
00049                 if(userData.data_path.rfind(type) != string::npos){
00050                     fileType = fileCategories::IMAGE;

```



```

00051         Mat frame = imread(userData.data_path, cv::IMREAD_COLOR);
00052
00053         vector<string> dataClasses = model.getDataClasses(labels_path);
00054         cout << "GPU available? " << (model.isAvailableGPU() ? "YES" : "NO") << endl;
00055         model.detectionPipeline(frame);
00056         auto detections = model.getDetections();
00057         Mat resultImg = cardValues(detections, model, frame);
00058         string outputDir = "../output/";
00059         if (!fs::exists(outputDir)) {
00060             fs::create_directory(outputDir);
00061         }
00062         fs::path p(userData.data_path);
00063         string filename = outputDir + p.stem().string() + "_detections.jpg";
00064         imwrite(filename, resultImg);
00065         computeImageMetrics(userData.data_path, detections);
00066         std::string windowTitle = model.getModelName() + " - " +
std::to_string(detections.size()) + " detections";
00067         namedWindow(windowTitle, WINDOW_NORMAL);
00068         imshow(windowTitle, resultImg);
00069         waitKey(0);
00070         break;
00071     }
00072 }
00073
00074     /* check if file is video */
00075     if(fileType == fileCategories::UNKNOWN){
00076         for(auto type: allowedVidType){
00077             if(userData.data_path.rfind(type) != string::npos){
00078                 fileType = fileCategories::VIDEO;
00079                 cout << "Using ONNX Runtime GPU? " << (model.isAvailableGPU() ? "YES" : "NO") <<
endl;
00080                 frameCapture(userData.data_path, labels_path);
00081                 break;
00082             }
00083         }
00084     }
00085
00086     /* invalid file */
00087     if(fileType == fileCategories::UNKNOWN){
00088         cerr << "The image or video provided has an invalid file type. Please retry with a
different one.\n";
00089         return -1;
00090     }
00091 }
00092
00093 }
00094 break;
00095
00096 case Choice::Invalid:
00097 case Choice::Help:
00098 {
00099     //TODO: Verifica se questa parte va bene, di norma in questo caso non può ricaderci
l'utente
00100     throw std::logic_error("Invalid operation: ...");
00101 }
00102 }
00103
00104 // TODO: batch processing
00105 if(labels_path.rfind(".txt") == string::npos){
00106     cerr << "The label file provided has an invalid file type. Please provide a .txt file.\n";
00107     return -1;
00108 }
00109
00110 // destroyAllWindows();
00111 return 0;
00112 }

```

6.11 src/hermann_serain.cpp File Reference

```

#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <numeric>
#include <fstream>
#include <sstream>

```


6.11.1 Function Documentation

6.11.1.1 calculateIoU()

```
float calculateIoU (
    const cv::Rect & box1,
    const cv::Rect & box2 )
```

Definition at line 304 of file [hermann_serain.cpp](#).

```
00304                                     {
00305     int x1 = std::max(box1.x, box2.x);
00306     int y1 = std::max(box1.y, box2.y);
00307     int x2 = std::min(box1.x + box1.width, box2.x + box2.width);
00308     int y2 = std::min(box1.y + box1.height, box2.y + box2.height);
00309
00310     int intersectionArea = std::max(0, x2 - x1) * std::max(0, y2 - y1);
00311     int unionArea = box1.area() + box2.area() - intersectionArea;
00312
00313     if (unionArea == 0) return 0.0f;
00314     return static_cast<float>(intersectionArea) / unionArea;
00315 }
```

6.11.1.2 canComputeImageMetrics()

```
bool canComputeImageMetrics (
    string imgPath )
```

Definition at line 295 of file [hermann_serain.cpp](#).

```
00295                                     {
00296     return canComputeMetrics(imgPath, "images");
00297 }
```

6.11.1.3 canComputeMetrics()

```
bool canComputeMetrics (
    string filePath,
    string directoryName )
```

Definition at line 279 of file [hermann_serain.cpp](#).

```
00279                                     {
00280     fs::path p(filePath);
00281     string filename = p.stem().string();
00282
00283     // Construct the path dynamically based on directoryName
00284     string labelPath = "../data/test/ground_truths/" + directoryName + "/" + filename + ".txt";
00285
00286     if (checkPath(labelPath)) {
00287         cout << "[METRICS] Ground truth found in " << directoryName << " for: " << filename << ".
Proceeding with calculation." << endl;
00288         return true;
00289     }
00290
00291     cout << "[INFO] No ground truth labels found at: " << labelPath << ". Skipping metrics." << endl;
00292     return false;
00293 }
```

6.11.1.4 canComputeVideoMetrics()

```
bool canComputeVideoMetrics (
    string videoPath )
```

Definition at line 299 of file [hermann_serain.cpp](#).

```
00299                                     {
00300     return canComputeMetrics(videoPath, "videos");
00301 }
```

6.11.1.5 checkPath()

```
bool checkPath (
    const string & fullPath )
```

Definition at line 268 of file [hermann_serain.cpp](#).

```
00268     {
00269     try {
00270         if (fs::exists(fullPath) && fs::is_regular_file(fullPath)) {
00271             return true;
00272         }
00273     } catch (const fs::filesystem_error& e) {
00274         cerr << "[ERROR] Filesystem access error: " << e.what() << endl;
00275     }
00276     return false;
00277 }
```

6.11.1.6 computeImageMetrics()

```
void computeImageMetrics (
    string imagePath,
    std::vector< Detection > predictedDetections )
```

Definition at line 437 of file [hermann_serain.cpp](#).

```
00437     {
00438     if (!canComputeImageMetrics(imagePath)) {
00439         return;
00440     }
00441
00442     fs::path p(imagePath);
00443     string filename = p.stem().string();
00444     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00445
00446     cv::Mat img = cv::imread(imagePath);
00447     if (img.empty()) {
00448         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00449         return;
00450     }
00451
00452     // Extraction of ground truth using the helper method
00453     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00454
00455     if (!actualDetections.empty()) {
00456         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00457         printFinalMetrics(results);
00458     }
00459 }
```

6.11.1.7 computeVideoMetrics()

```
void computeVideoMetrics (
    std::vector< Detection > predictedDetections,
    int frameCount )
```

Definition at line 487 of file [hermann_serain.cpp](#).

```
00487     {
00488     if (!g_videoMetricsEnabled || !g_motFile.is_open()) return;
00489
00490     std::vector<Detection> actualDetections;
00491     string line;
00492
00493     // If we have a cached row from the previous frame, we start from that one
00494     if (!g_lastLine.empty()) {
00495         line = g_lastLine;
00496         g_lastLine = "";
00497     } else if (!getline(g_motFile, line)) {
00498         return; // End of file
00499     }
00500
00501     while (true) {
00502         stringstream ss(line);
```

```

00503         string token;
00504         vector<string> tokens;
00505
00506         // Fast line parsing (CSV)
00507         while (getline(ss, token, ',')) tokens.push_back(token);
00508
00509         if (tokens.size() >= 8) {
00510             int currentLineFrame = stoi(tokens[0]);
00511
00512             if (currentLineFrame == frameCount) {
00513                 Detection d;
00514                 d.boundingBox = cv::Rect(stof(tokens[2]), stof(tokens[3]), stof(tokens[4]),
stof(tokens[5]));
00515                 d.classId = stoi(tokens[7]) - 1;
00516                 d.classConfidence = 1.0f;
00517                 actualDetections.push_back(d);
00518             }
00519             else if (currentLineFrame > frameCount) {
00520                 g_lastLine = line;
00521                 break;
00522             }
00523         }
00524
00525         if (!getline(g_motFile, line)) break;
00526     }
00527     MetricsCounter frameResults = updateMetricsCounters(predictedDetections, actualDetections);
00528
00529     g_cumulativeVideoMetrics.tp += frameResults.tp;
00530     g_cumulativeVideoMetrics.fp += frameResults.fp;
00531     g_cumulativeVideoMetrics.fn += frameResults.fn;
00532 }

```

6.11.1.8 findMatches()

```

vector< string > findMatches (
    const string & directory,
    const string & partial )

```

Definition at line 242 of file [hermann_serain.cpp](#).

```

00243 {
00244     vector<string> matches;
00245
00246     if (!fs::exists(directory))
00247         return matches;
00248
00249     for (const auto& entry : fs::directory_iterator(directory))
00250     {
00251         string name = entry.path().filename().string();
00252
00253         if (name.rfind(partial, 0) == 0)
00254             matches.push_back(name);
00255     }
00256
00257     return matches;
00258 }

```

6.11.1.9 handleCamera()

```

void handleCamera ( )

```

Definition at line 154 of file [hermann_serain.cpp](#).

```

00155 {
00156     cout << "Selected camera (press q to close the camera)\n";
00157 }

```

6.11.1.10 handleFile()

```
string handleFile ( )
```

Definition at line 159 of file [hermann_serain.cpp](#).

```
00160 {
00161     TerminalRawMode rawMode; // enable raw terminal
00162
00163     string currentPath;
00164     cout << "Insert file path: ";
00165     cout.flush();
00166
00167     while (true)
00168     {
00169         char ch;
00170         read(STDIN_FILENO, &ch, 1);
00171
00172         // ENTER
00173         if (ch == '\n')
00174         {
00175             cout << endl;
00176             return currentPath;
00177         }
00178
00179         // BACKSPACE
00180         if (ch == 127)
00181         {
00182             if (!currentPath.empty())
00183             {
00184                 currentPath.pop_back();
00185                 redrawLine(currentPath);
00186             }
00187             continue;
00188         }
00189
00190         // TAB
00191         if (ch == '\t')
00192         {
00193             string dir, partial;
00194             splitPath(currentPath, dir, partial);
00195
00196             auto matches = findMatches(dir, partial);
00197
00198             if (matches.size() == 1)
00199             {
00200                 currentPath.erase(currentPath.size() - partial.size());
00201                 currentPath += matches[0];
00202             }
00203             else if (matches.size() > 1)
00204             {
00205                 cout << "\n";
00206                 for (const auto& m : matches)
00207                     cout << " " << m << "\n";
00208             }
00209
00210             redrawLine(currentPath);
00211             continue;
00212         }
00213
00214         // Normal char
00215         if (isprint(ch))
00216         {
00217             currentPath.push_back(ch);
00218             cout << ch;
00219             cout.flush();
00220         }
00221     }
00222 }
```

6.11.1.11 initObjectsForVideoMetrics()

```
void initObjectsForVideoMetrics (
    string videoPath )
```

Definition at line 469 of file [hermann_serain.cpp](#).

```
00469 {
00470     if (g_motFile.is_open()) g_motFile.close();
00471     g_videoMetricsEnabled = false;
```

```

00472     g_cumulativeVideoMetrics = MetricsCounter();
00473     g_lastLine = "";
00474
00475     if (!canComputeVideoMetrics(videoPath)) return;
00476
00477     fs::path p(videoPath);
00478     string labelPath = "../data/test/ground_truths/videos/" + p.stem().string() + ".txt";
00479
00480     g_motFile.open(labelPath);
00481     if (g_motFile.is_open()) {
00482         g_videoMetricsEnabled = true;
00483         cout << "[METRICS] FMOT file opened for sequential reading." << endl;
00484     }
00485 }

```

6.11.1.12 parseYoloLabels()

```

std::vector< Detection > parseYoloLabels (
    string labelPath,
    int imgW,
    int imgH )

```

Definition at line 397 of file [hermann_serain.cpp](#).

```

00397
00398     std::vector<Detection> actualDetections;
00399     ifstream file(labelPath);
00400     string line;
00401
00402     try {
00403         while (getline(file, line)) {
00404             if (line.empty()) continue;
00405
00406             stringstream ss(line);
00407             int classId;
00408             float x_center, y_center, w, h;
00409
00410             if (!(ss >> classId >> x_center >> y_center >> w >> h)) {
00411                 throw std::runtime_error("Invalid YOLO format in file: " + labelPath);
00412             }
00413
00414             //NOTE: Converting bounding box width and height from percentage to actual values
00415             int pixelW = static_cast<int>(w * imgW);
00416             int pixelH = static_cast<int>(h * imgH);
00417             //NOTE: Moving the bounding box center to the left top corner.
00418             int pixelX = static_cast<int>((x_center * imgW) - (pixelW / 2.0f));
00419             int pixelY = static_cast<int>((y_center * imgH) - (pixelH / 2.0f));
00420
00421             Detection gt;
00422             gt.classId = classId - 1;
00423             gt.boundingBox = cv::Rect(pixelX, pixelY, pixelW, pixelH);
00424             gt.classConfidence = 1.0f;
00425
00426             actualDetections.push_back(gt);
00427         }
00428     } catch (const std::exception& e) {
00429         cerr << "[ERROR] Reading image ground truth: " << e.what() << endl;
00430         file.close();
00431         return {}; // Return empty vector on error
00432     }
00433     file.close();
00434     return actualDetections;
00435 }

```

6.11.1.13 printFinalMetrics()

```

void printFinalMetrics (
    const MetricsCounter & total )

```

Definition at line 376 of file [hermann_serain.cpp](#).

```

00376
00377     if (total.tp + total.fp == 0 || total.tp + total.fn == 0) {
00378         cout << "[METRICS] Warning: Not enough data to compute Precision/Recall." << endl;
00379         return;
00380     }

```

```

00381
00382     float precision = static_cast<float>(total.tp) / (total.tp + total.fp);
00383     float recall = static_cast<float>(total.tp) / (total.tp + total.fn);
00384     float f1Score = (precision + recall > 0) ? (2.0f * (precision * recall) / (precision + recall)) :
0.0f;
00385
00386     cout << "\n--- Final Performance Metrics ---" << endl;
00387     cout << "Total True Positives (TP): " << total.tp << endl;
00388     cout << "Total False Positives (FP): " << total.fp << endl;
00389     cout << "Total False Negatives (FN): " << total.fn << endl;
00390     cout << "-----" << endl;
00391     cout << "Precision: " << precision << endl;
00392     cout << "Recall: " << recall << endl;
00393     cout << "F1-Score: " << f1Score << endl;
00394     cout << "-----\n" << endl;
00395 }

```

6.11.1.14 printFinalVideoMetrics()

```
void printFinalVideoMetrics ( )
```

Definition at line 535 of file [hermann_serain.cpp](#).

```

00535     {
00536         if (!g_videoMetricsEnabled) {
00537             cout << "[METRICS] No data accumulated for this video." << endl;
00538             return;
00539         }
00540
00541         cout << "\n--- FINAL CUMULATIVE VIDEO STATISTICS ---" << endl;
00542         printFinalMetrics(g_cumulativeVideoMetrics);
00543
00544         if (g_motFile.is_open()) {
00545             g_motFile.close();
00546         }
00547
00548         g_videoMetricsEnabled = false;
00549         g_lastLine = "";
00550         g_cumulativeVideoMetrics.tp = 0;
00551         g_cumulativeVideoMetrics.fp = 0;
00552         g_cumulativeVideoMetrics.fn = 0;
00553
00554         cout << "[INFO] Metrics state reset for the next session." << endl;
00555     }

```

6.11.1.15 printHelp()

```
void printHelp ( )
```

Definition at line 133 of file [hermann_serain.cpp](#).

```

00134 {
00135     cout << "\n";
00136     cout << "===== \n";
00137     cout << "          APPLICATION HELP PAGE          \n";
00138     cout << "===== \n";
00139     cout << "\n";
00140     cout << "Commands: \n";
00141     cout << "  C          Use camera as input \n";
00142     cout << "  F          Use file as input \n";
00143     cout << "  H / -h     Show this help page \n";
00144     cout << "\n";
00145     cout << "Details: \n";
00146     cout << "  - Camera closes by pressing 'q' \n";
00147     cout << "  - File path must not contain spaces \n";
00148     cout << "  - TAB completion not implemented yet \n";
00149     cout << "\n";
00150     cout << "===== \n";
00151     cout << endl;
00152 }

```


6.11.1.16 readInput()

```
UserData readInput (
    int argc,
    char ** argv )
```

Definition at line 63 of file [hermann_serain.cpp](#).

```
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG_VIDEO) {
00073                 result.data_path = DEFAULT_VIDEO_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG_IMAGE) {
00077                 result.data_path = DEFAULT_IMAGE_PATH;
00078                 result.choice = Choice::File;
00079             }
00080         }
00081     }
00082
00083     if(result.choice != Choice::Invalid)
00084         return result;
00085
00086     //NOTE: I'm gonna read the user input from now on
00087     string choiceStr;
00088
00089     // NOTE: Let's keep asking if the input is invalid
00090     while (result.choice == Choice::Invalid)
00091     {
00092         cout << "-----\n";
00093         cout << " C = Camera | F = File | H = Help\n";
00094         cout << "-----\n";
00095         cout << "Write your choice: ";
00096
00097         cin >> choiceStr;
00098         result.choice = parseChoice(choiceStr);
00099
00100         switch (result.choice)
00101         {
00102             case Choice::Camera:
00103             {
00104                 handleCamera();
00105             }
00106             break;
00107
00108             case Choice::File:
00109             {
00110                 result.data_path = handleFile();
00111             }
00112             break;
00113
00114             case Choice::Help:
00115             {
00116                 printHelp();
00117                 result.choice = Choice::Invalid; //NOTE: continuing loop
00118             }
00119             break;
00120
00121             case Choice::Invalid:
00122             {
00123                 cout << "Usage: enter C or F (or H for help)!\n";
00124             }
00125         }
00126     }
00127
00128     return result;
00129 }
```

6.11.1.17 redrawLine()

```
void redrawLine (
    const string & text )
```

Definition at line 260 of file [hermann_serain.cpp](#).

```
00261 {
00262     cout << "\rInsert file path: " << text;
00263     cout << "\033[K"; // delete at the row end
00264     cout.flush();
00265 }
```

6.11.1.18 splitPath()

```
void splitPath (
    const string & fullPath,
    string & directory,
    string & partial )
```

Definition at line 226 of file [hermann_serain.cpp](#).

```
00227 {
00228     size_t pos = fullPath.find_last_of('/');
00229
00230     if (pos == string::npos)
00231     {
00232         directory = ".";
00233         partial = fullPath;
00234     }
00235     else
00236     {
00237         directory = fullPath.substr(0, pos);
00238         partial = fullPath.substr(pos + 1);
00239     }
00240 }
```

6.11.1.19 updateMetricsCounters()

```
MetricsCounter updateMetricsCounters (
    std::vector< Detection > predictedDetections,
    std::vector< Detection > actualDetections )
```

Definition at line 318 of file [hermann_serain.cpp](#).

```
00318 {
00319     const float IOU_THRESHOLD = 0.5f;
00320     MetricsCounter frameResults;
00321
00322     // Copy of ground truth to manage matching and removals
00323     std::vector<Detection> remainingGT = actualDetections;
00324
00325     for (const auto& pred : predictedDetections) {
00326         int bestMatchIdx = -1;
00327         float maxIoU = 0.0f;
00328
00329         // Search for the best Ground Truth match (Same Class + Highest IoU)
00330         for (int i = 0; i < remainingGT.size(); ++i) {
00331             if (pred.classId == remainingGT[i].classId) {
00332                 float currentIoU = calculateIoU(pred.boundingBox, remainingGT[i].boundingBox);
00333                 if (currentIoU > maxIoU) {
00334                     maxIoU = currentIoU;
00335                     bestMatchIdx = i;
00336                 }
00337             }
00338         }
00339
00340         // Validate the match against the threshold
00341         if (bestMatchIdx != -1 && maxIoU >= IOU_THRESHOLD) {
00342             frameResults.tp++;
00343
00344             //WARNING: CHECK CORRECTNESS WITH DEBUGGING
00345
00346             /* CARD CONSOLIDATION LOGIC
00347              Per your requirement: remove the matched GT and other GTs of the
00348              same class within the surrounding "card-sized" region.
00349             */
00350             cv::Rect matchedRect = remainingGT[bestMatchIdx].boundingBox;
00351             int matchedClass = remainingGT[bestMatchIdx].classId;
00352 }
```

```

00353         // Remove the exact match first
00354         remainingGT.erase(remainingGT.begin() + bestMatchIdx);
00355
00356         // Remove nearby boxes of the same card (approx. same region)
00357         remainingGT.erase(
00358             std::remove_if(remainingGT.begin(), remainingGT.end(),
00359                 [&](const Detection& d) {
00360                     return (d.classId == matchedClass && calculateIoU(matchedRect, d.boundingBox)
00361 > 0.1f);
00362             })),
00363             remainingGT.end()
00364         );
00365     } else {
00366         frameResults.fp++; //NOTE: No valid ground truths found for this prediction
00367     }
00368 }
00369
00370 // Remaining ground truths are false negatives
00371 frameResults.fn = static_cast<int>(remainingGT.size());
00372
00373 return frameResults;
00374 }

```

6.11.2 Variable Documentation

6.11.2.1 DEFAULT_IMAGE_PATH

```
const std::string DEFAULT_IMAGE_PATH = "../data/test/images/default_image.jpg"
```

Definition at line 26 of file [hermann_serain.cpp](#).

6.11.2.2 DEFAULT_VIDEO_PATH

```
const std::string DEFAULT_VIDEO_PATH = "../data/test/videos/default_video.mp4"
```

Definition at line 25 of file [hermann_serain.cpp](#).

6.11.2.3 FLAG_IMAGE

```
const std::string FLAG_IMAGE = "--default-image"
```

Definition at line 28 of file [hermann_serain.cpp](#).

6.11.2.4 FLAG_VIDEO

```
const std::string FLAG_VIDEO = "--default-video"
```

Definition at line 27 of file [hermann_serain.cpp](#).

6.11.2.5 g_cumulativeVideoMetrics

```
MetricsCounter g_cumulativeVideoMetrics
```

Definition at line 466 of file [hermann_serain.cpp](#).

6.11.2.6 g_lastLine

```
std::string g_lastLine
```

Definition at line 467 of file [hermann_serain.cpp](#).

6.11.2.7 g_motFile

```
std::ifstream g_motFile
```

Definition at line 464 of file [hermann_serain.cpp](#).

6.11.2.8 g_videoMetricsEnabled

```
bool g_videoMetricsEnabled = false
```

Definition at line 465 of file [hermann_serain.cpp](#).

6.12 hermann_serain.cpp

[Go to the documentation of this file.](#)

```
00001 #include <opencv2/opencv.hpp>
00002 #include <iostream>
00003 #include <vector>
00004 #include <map>
00005 #include <string>
00006 #include <numeric>
00007 #include <fstream>
00008 #include <sstream>
00009 #include <array>
00010 #include <stdexcept>
00011 #include <filesystem>
00012 #include <termios.h>
00013 #include <unistd.h>
00014
00015 #include "../include/hermann_serain.hpp"
00016 #include "../include/marco_annunziata.hpp"
00017 #include "../include/shared.hpp"
00018
00019 namespace fs = std::filesystem;
00020 using namespace cv;
00021 using namespace std;
00022 using namespace Shared;
00023
00024 //User parameters consts
00025 const std::string DEFAULT_VIDEO_PATH = "../data/test/videos/default_video.mp4";
00026 const std::string DEFAULT_IMAGE_PATH = "../data/test/images/default_image.jpg";
00027 const std::string FLAG_VIDEO = "--default-video";
00028 const std::string FLAG_IMAGE = "--default-image";
00029
00030 //Tab utilities
00031 struct TerminalRawMode
00032 {
00033     termios oldt;
00034
00035     TerminalRawMode()
00036     {
00037         tcgetattr(STDIN_FILENO, &oldt);
00038         termios newt = oldt;
00039
00040         newt.c_lflag &= ~(ICANON | ECHO); // no buffer, no echo
00041         tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00042     }
00043
00044     ~TerminalRawMode()
00045     {
00046         tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00047     }
00048 }
```

```

00047     }
00048 };
00049
00050 void splitPath(const string& fullPath, string& directory, string& partial);
00051 vector<string> findMatches(const string& directory, const string& partial);
00052 void redrawLine(const string& text);
00053
00054
00055 //Help page methods
00056 void printHelp();
00057 void handleCamera();
00058 string handleFile();
00059
00060
00061 //WARNING: THIS PROGRAM USE THE TERMINAL API AND PROCESS THE TERMINAL IN RAW MODE,
00062 //IF THE TERMINAL DOESN'T WORK AS BEFORE, TYPE "reset" BLINDLY
00063 UserData readInput(int argc, char** argv)
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG_VIDEO) {
00073                 result.data_path = DEFAULT_VIDEO_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG_IMAGE) {
00077                 result.data_path = DEFAULT_IMAGE_PATH;
00078                 result.choice = Choice::File;
00079             }
00080         }
00081     }
00082
00083     if(result.choice != Choice::Invalid)
00084         return result;
00085
00086     //NOTE: I'm gonna read the user input from now on
00087     string choiceStr;
00088
00089     // NOTE: Let's keep asking if the input is invalid
00090     while (result.choice == Choice::Invalid)
00091     {
00092         cout << "-----\n";
00093         cout << " C = Camera | F = File | H = Help\n";
00094         cout << "-----\n";
00095         cout << "Write your choice: ";
00096
00097         cin >> choiceStr;
00098         result.choice = parseChoice(choiceStr);
00099
00100         switch (result.choice)
00101         {
00102             case Choice::Camera:
00103             {
00104                 handleCamera();
00105             }
00106             break;
00107
00108             case Choice::File:
00109             {
00110                 result.data_path = handleFile();
00111             }
00112             break;
00113
00114             case Choice::Help:
00115             {
00116                 printHelp();
00117                 result.choice = Choice::Invalid; //NOTE: continuing loop
00118             }
00119             break;
00120
00121             case Choice::Invalid:
00122             {
00123                 cout << "Usage: enter C or F (or H for help)!\n";
00124             }
00125         }
00126     }
00127
00128     return result;
00129 }
00130
00131
00132 //NOTE: Print the help page
00133 void printHelp()

```

```

00134 {
00135     cout << "\n";
00136     cout << "===== \n";
00137     cout << "          APPLICATION HELP PAGE          \n";
00138     cout << "===== \n";
00139     cout << "\n";
00140     cout << "Commands:\n";
00141     cout << "  C          Use camera as input\n";
00142     cout << "  F          Use file as input\n";
00143     cout << "  H / -h      Show this help page\n";
00144     cout << "\n";
00145     cout << "Details:\n";
00146     cout << "  - Camera closes by pressing 'q'\n";
00147     cout << "  - File path must not contain spaces\n";
00148     cout << "  - TAB completion not implemented yet\n";
00149     cout << "\n";
00150     cout << "===== \n";
00151     cout << endl;
00152 }
00153
00154 void handleCamera()
00155 {
00156     cout << "Selected camera (press q to close the camera)\n";
00157 }
00158
00159 string handleFile()
00160 {
00161     TerminalRawMode rawMode; // enable raw terminal
00162
00163     string currentPath;
00164     cout << "Insert file path: ";
00165     cout.flush();
00166
00167     while (true)
00168     {
00169         char ch;
00170         read(STDIN_FILENO, &ch, 1);
00171
00172         // ENTER
00173         if (ch == '\n')
00174         {
00175             cout << endl;
00176             return currentPath;
00177         }
00178
00179         // BACKSPACE
00180         if (ch == 127)
00181         {
00182             if (!currentPath.empty())
00183             {
00184                 currentPath.pop_back();
00185                 redrawLine(currentPath);
00186             }
00187             continue;
00188         }
00189
00190         // TAB
00191         if (ch == '\t')
00192         {
00193             string dir, partial;
00194             splitPath(currentPath, dir, partial);
00195
00196             auto matches = findMatches(dir, partial);
00197
00198             if (matches.size() == 1)
00199             {
00200                 currentPath.erase(currentPath.size() - partial.size());
00201                 currentPath += matches[0];
00202             }
00203             else if (matches.size() > 1)
00204             {
00205                 cout << "\n";
00206                 for (const auto& m : matches)
00207                     cout << "  " << m << "\n";
00208             }
00209
00210             redrawLine(currentPath);
00211             continue;
00212         }
00213
00214         // Normal char
00215         if (isprint(ch))
00216         {
00217             currentPath.push_back(ch);
00218             cout << ch;
00219             cout.flush();
00220         }

```

```

00221     }
00222 }
00223
00224
00225 //Print utilities implementations
00226 void splitPath(const string& fullPath, string& directory, string& partial)
00227 {
00228     size_t pos = fullPath.find_last_of('/');
00229
00230     if (pos == string::npos)
00231     {
00232         directory = ".";
00233         partial = fullPath;
00234     }
00235     else
00236     {
00237         directory = fullPath.substr(0, pos);
00238         partial = fullPath.substr(pos + 1);
00239     }
00240 }
00241
00242 vector<string> findMatches(const string& directory, const string& partial)
00243 {
00244     vector<string> matches;
00245
00246     if (!fs::exists(directory))
00247         return matches;
00248
00249     for (const auto& entry : fs::directory_iterator(directory))
00250     {
00251         string name = entry.path().filename().string();
00252
00253         if (name.rfind(partial, 0) == 0)
00254             matches.push_back(name);
00255     }
00256
00257     return matches;
00258 }
00259
00260 void redrawLine(const string& text)
00261 {
00262     cout << "\rInsert file path: " << text;
00263     cout << "\033[K"; // delete at the row end
00264     cout.flush();
00265 }
00266
00267 //NOTE: Metrics
00268 bool checkPath(const string& fullPath) {
00269     try {
00270         if (fs::exists(fullPath) && fs::is_regular_file(fullPath)) {
00271             return true;
00272         }
00273     } catch (const fs::filesystem_error& e) {
00274         cerr << "[ERROR] Filesystem access error: " << e.what() << endl;
00275     }
00276     return false;
00277 }
00278
00279 bool canComputeMetrics(string filePath, string directoryName) {
00280     fs::path p(filePath);
00281     string filename = p.stem().string();
00282
00283     // Construct the path dynamically based on directoryName
00284     string labelPath = "../data/test/ground_truths/" + directoryName + "/" + filename + ".txt";
00285
00286     if (checkPath(labelPath)) {
00287         cout << "[METRICS] Ground truth found in " << directoryName << " for: " << filename << ".
Proceeding with calculation." << endl;
00288         return true;
00289     }
00290
00291     cout << "[INFO] No ground truth labels found at: " << labelPath << ". Skipping metrics." << endl;
00292     return false;
00293 }
00294
00295 bool canComputeImageMetrics(string imgPath) {
00296     return canComputeMetrics(imgPath, "images");
00297 }
00298
00299 bool canComputeVideoMetrics(string videoPath) {
00300     return canComputeMetrics(videoPath, "videos");
00301 }
00302
00303 //NOTE: Helper method to compute the IoU
00304 float calculateIoU(const cv::Rect& box1, const cv::Rect& box2) {
00305     int x1 = std::max(box1.x, box2.x);
00306     int y1 = std::max(box1.y, box2.y);

```

```

00307     int x2 = std::min(box1.x + box1.width, box2.x + box2.width);
00308     int y2 = std::min(box1.y + box1.height, box2.y + box2.height);
00309
00310     int intersectionArea = std::max(0, x2 - x1) * std::max(0, y2 - y1);
00311     int unionArea = box1.area() + box2.area() - intersectionArea;
00312
00313     if (unionArea == 0) return 0.0f;
00314     return static_cast<float>(intersectionArea) / unionArea;
00315 }
00316
00317 //NOTE: Method used to compute the Detections for both images and videos
00318 MetricsCounter updateMetricsCounters(std::vector<Detection> predictedDetections,
std::vector<Detection> actualDetections) {
00319     const float IOU_THRESHOLD = 0.5f;
00320     MetricsCounter frameResults;
00321
00322     // Copy of ground truth to manage matching and removals
00323     std::vector<Detection> remainingGT = actualDetections;
00324
00325     for (const auto& pred : predictedDetections) {
00326         int bestMatchIdx = -1;
00327         float maxIoU = 0.0f;
00328
00329         // Search for the best Ground Truth match (Same Class + Highest IoU)
00330         for (int i = 0; i < remainingGT.size(); ++i) {
00331             if (pred.classId == remainingGT[i].classId) {
00332                 float currentIoU = calculateIoU(pred.boundingBox, remainingGT[i].boundingBox);
00333                 if (currentIoU > maxIoU) {
00334                     maxIoU = currentIoU;
00335                     bestMatchIdx = i;
00336                 }
00337             }
00338         }
00339
00340         // Validate the match against the threshold
00341         if (bestMatchIdx != -1 && maxIoU >= IOU_THRESHOLD) {
00342             frameResults.tp++;
00343
00344             //WARNING: CHECK CORRECTNESS WITH DEBUGGING
00345
00346             /* CARD CONSOLIDATION LOGIC
00347              Per your requirement: remove the matched GT and other GTs of the
00348              same class within the surrounding "card-sized" region.
00349             */
00350             cv::Rect matchedRect = remainingGT[bestMatchIdx].boundingBox;
00351             int matchedClass = remainingGT[bestMatchIdx].classId;
00352
00353             // Remove the exact match first
00354             remainingGT.erase(remainingGT.begin() + bestMatchIdx);
00355
00356             // Remove nearby boxes of the same card (approx. same region)
00357             remainingGT.erase(
00358                 std::remove_if(remainingGT.begin(), remainingGT.end(),
00359                     [&](const Detection& d) {
00360                         return (d.classId == matchedClass && calculateIoU(matchedRect, d.boundingBox)
00361 > 0.1f);
00362                     })),
00363                 remainingGT.end()
00364             );
00365         } else {
00366             frameResults.fp++; //NOTE: No valid ground truths found for this prediction
00367         }
00368     }
00369
00370     // Remaining ground truths are false negatives
00371     frameResults.fn = static_cast<int>(remainingGT.size());
00372
00373     return frameResults;
00374 }
00375
00376 void printFinalMetrics(const MetricsCounter& total) {
00377     if (total.tp + total.fp == 0 || total.tp + total.fn == 0) {
00378         cout << "[METRICS] Warning: Not enough data to compute Precision/Recall." << endl;
00379         return;
00380     }
00381
00382     float precision = static_cast<float>(total.tp) / (total.tp + total.fp);
00383     float recall = static_cast<float>(total.tp) / (total.tp + total.fn);
00384     float f1Score = (precision + recall > 0) ? (2.0f * (precision * recall) / (precision + recall)) :
0.0f;
00385
00386     cout << "\n--- Final Performance Metrics ---" << endl;
00387     cout << "Total True Positives (TP): " << total.tp << endl;
00388     cout << "Total False Positives (FP): " << total.fp << endl;
00389     cout << "Total False Negatives (FN): " << total.fn << endl;
00390     cout << "-----" << endl;

```



```

00391     cout << "Precision: " << precision << endl;
00392     cout << "Recall:      " << recall << endl;
00393     cout << "F1-Score:   " << f1Score << endl;
00394     cout << "-----\n" << endl;
00395 }
00396
00397 std::vector<Detection> parseYoloLabels(string labelPath, int imgW, int imgH) {
00398     std::vector<Detection> actualDetections;
00399     ifstream file(labelPath);
00400     string line;
00401
00402     try {
00403         while (getline(file, line)) {
00404             if (line.empty()) continue;
00405
00406             stringstream ss(line);
00407             int classId;
00408             float x_center, y_center, w, h;
00409
00410             if (!(ss >> classId >> x_center >> y_center >> w >> h)) {
00411                 throw std::runtime_error("Invalid YOLO format in file: " + labelPath);
00412             }
00413
00414             //NOTE: Converting bounding box width and height from percentage to actual values
00415             int pixelW = static_cast<int>(w * imgW);
00416             int pixelH = static_cast<int>(h * imgH);
00417             //NOTE: Moving the bounding box center to the left top corner.
00418             int pixelX = static_cast<int>((x_center * imgW) - (pixelW / 2.0f));
00419             int pixelY = static_cast<int>((y_center * imgH) - (pixelH / 2.0f));
00420
00421             Detection gt;
00422             gt.classId = classId - 1;
00423             gt.boundingBox = cv::Rect(pixelX, pixelY, pixelW, pixelH);
00424             gt.classConfidence = 1.0f;
00425
00426             actualDetections.push_back(gt);
00427         }
00428     } catch (const std::exception& e) {
00429         cerr << "[ERROR] Reading image ground truth: " << e.what() << endl;
00430         file.close();
00431         return {}; // Return empty vector on error
00432     }
00433     file.close();
00434     return actualDetections;
00435 }
00436
00437 void computeImageMetrics(string imagePath, std::vector<Detection> predictedDetections) {
00438     if (!canComputeImageMetrics(imagePath)) {
00439         return;
00440     }
00441
00442     fs::path p(imagePath);
00443     string filename = p.stem().string();
00444     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00445
00446     cv::Mat img = cv::imread(imagePath);
00447     if (img.empty()) {
00448         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00449         return;
00450     }
00451
00452     // Extraction of ground truth using the helper method
00453     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00454
00455     if (!actualDetections.empty()) {
00456         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00457         printFinalMetrics(results);
00458     }
00459 }
00460
00461
00462 //NOTE: Compute Video Metrics
00463
00464 std::ifstream g_motFile;
00465 bool g_videoMetricsEnabled = false;
00466 MetricsCounter g_cumulativeVideoMetrics;
00467 std::string g_lastLine;
00468
00469 void initObjectsForVideoMetrics(string videoPath) {
00470     if (g_motFile.is_open()) g_motFile.close();
00471     g_videoMetricsEnabled = false;
00472     g_cumulativeVideoMetrics = MetricsCounter();
00473     g_lastLine = "";
00474
00475     if (!canComputeVideoMetrics(videoPath)) return;
00476
00477     fs::path p(videoPath);

```

```

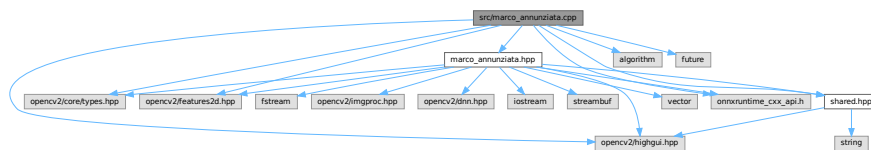
00478     string labelPath = "../data/test/ground_truths/videos/" + p.stem().string() + ".txt";
00479
00480     g_motFile.open(labelPath);
00481     if (g_motFile.is_open()) {
00482         g_videoMetricsEnabled = true;
00483         cout << "[METRICS] FMOT file opened for sequential reading." << endl;
00484     }
00485 }
00486
00487 void computeVideoMetrics(std::vector<Detection> predictedDetections, int frameCount) {
00488     if (!g_videoMetricsEnabled || !g_motFile.is_open()) return;
00489
00490     std::vector<Detection> actualDetections;
00491     string line;
00492
00493     // If we have a cached row from the previous frame, we start from that one
00494     if (!g_lastLine.empty()) {
00495         line = g_lastLine;
00496         g_lastLine = "";
00497     } else if (!getline(g_motFile, line)) {
00498         return; // End of file
00499     }
00500
00501     while (true) {
00502         stringstream ss(line);
00503         string token;
00504         vector<string> tokens;
00505
00506         // Fast line parsing (CSV)
00507         while (getline(ss, token, ',')) tokens.push_back(token);
00508
00509         if (tokens.size() >= 8) {
00510             int currentLineFrame = stoi(tokens[0]);
00511
00512             if (currentLineFrame == frameCount) {
00513                 Detection d;
00514                 d.boundingBox = cv::Rect(stof(tokens[2]), stof(tokens[3]), stof(tokens[4]),
00515                     stof(tokens[5]));
00516                 d.classId = stoi(tokens[7]) - 1;
00517                 d.classConfidence = 1.0f;
00518                 actualDetections.push_back(d);
00519             } else if (currentLineFrame > frameCount) {
00520                 g_lastLine = line;
00521                 break;
00522             }
00523         }
00524
00525         if (!getline(g_motFile, line)) break;
00526     }
00527     MetricsCounter frameResults = updateMetricsCounters(predictedDetections, actualDetections);
00528
00529     g_cumulativeVideoMetrics.tp += frameResults.tp;
00530     g_cumulativeVideoMetrics.fp += frameResults.fp;
00531     g_cumulativeVideoMetrics.fn += frameResults.fn;
00532 }
00533
00534 void printFinalVideoMetrics() {
00535     if (!g_videoMetricsEnabled) {
00536         cout << "[METRICS] No data accumulated for this video." << endl;
00537         return;
00538     }
00539
00540     cout << "\n--- FINAL CUMULATIVE VIDEO STATISTICS ---" << endl;
00541     printFinalMetrics(g_cumulativeVideoMetrics);
00542
00543     if (g_motFile.is_open()) {
00544         g_motFile.close();
00545     }
00546
00547     g_videoMetricsEnabled = false;
00548     g_lastLine = "";
00549     g_cumulativeVideoMetrics.tp = 0;
00550     g_cumulativeVideoMetrics.fp = 0;
00551     g_cumulativeVideoMetrics.fn = 0;
00552
00553     cout << "[INFO] Metrics state reset for the next session." << endl;
00554 }

```

6.13 src/marco_annunziata.cpp File Reference

```
#include "marco_annunziata.hpp"
#include "shared.hpp"
#include <opencv2/core/types.hpp>
#include <opencv2/features2d.hpp>
#include <opencv2/highgui.hpp>
#include <onnxruntime_cxx_api.h>
#include <algorithm>
#include <future>
```

Include dependency graph for marco_annunziata.cpp:



Macros

- #define `UNUSED(x)` `(void)(x)`

6.13.1 Macro Definition Documentation

6.13.1.1 UNUSED

```
#define UNUSED(
    x ) (void)(x)
```

Definition at line 12 of file `marco_annunziata.cpp`.

6.14 marco_annunziata.cpp

[Go to the documentation of this file.](#)

```
00001 // AUTHOR: Marco Annunziata. https://github.com/Remdoox
00002 #include "marco_annunziata.hpp"
00003 #include "shared.hpp"
00004 #include <opencv2/core/types.hpp>
00005 #include <opencv2/features2d.hpp>
00006 #include <opencv2/highgui.hpp>
00007 #include <onnxruntime_cxx_api.h>
00008 #include <algorithm>
00009 #include <future>
00010
00011 // A couple of variables could be useful to keep so I use this in order to avoid g++ warnings
00012 #define UNUSED(x) (void)(x)
00013
00014 using namespace std;
00015 using namespace cv;
00016 using namespace cv::dnn;
00017
00027 YOLO_model::YOLO_model(): env(ORT_LOGGING_LEVEL_VERBOSE, "YOLOModel", YOLO_model::logger, this),
    session(nullptr), sessionOptions() {
00028     if (usingGPU) {
00029         try {
00030             OrtCUDAProviderOptions cudaOptions;
```

```

00031         cudaOptions.device_id = 0;
00032         cudaOptions.gpu_mem_limit = SIZE_MAX;
00033         // arena_extend_strategy: how much space to allocate at a time (arena = allocated block)
00034         // = 0 -> kNextPowerOfTwo, if the arena is filled up double its size (fast but
RAM-intensive)
00035         // = 1 -> kSameAsRequested, if the arena is filled up increase size exactly to what is
needed for the operation
00036         cudaOptions.arena_extend_strategy = 0;
00037         sessionOptions.AppendExecutionProvider_CUDA(cudaOptions);
00038         std::cout << "Using ONNX Runtime with GPU resources" << std::endl;
00039     }
00040     catch(const std::exception &e){
00041         std::cout << "CUDA not available! Using ONNX Runtime with CPU resources" << std::endl;
00042     }
00043 }
00044 unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00045 sessionOptions.SetIntraOpNumThreads(maxHardwareThreads);
00046 sessionOptions.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_EXTENDED);
00047 sessionOptions.SetLogId("YOLOModel");
00048
00049 const char* modelPath = "../data/model/YOLO11s_big_best_dynamic.onnx"; // YOLO11s_big_best_dynamic
00050 session = Ort::Session(env, modelPath, sessionOptions);
00051 }
00052
00053 void YOLO_model::logger(void* param, OrtLoggingLevel severity, const char* category, const char*
logid, const char* code_location,
00065 const char* message){
00066     static std::mutex logMutex;
00067     static std::ofstream logFile("../onnxruntime.log", std::ios::out | std::ios::app);
00068
00069     const char* severityStr = "";
00070     switch (severity) {
00071         case ORT_LOGGING_LEVEL_VERBOSE: severityStr = "VERBOSE"; break;
00072         case ORT_LOGGING_LEVEL_INFO: severityStr = "INFO"; break;
00073         case ORT_LOGGING_LEVEL_WARNING: severityStr = "WARNING"; break;
00074         case ORT_LOGGING_LEVEL_ERROR: severityStr = "ERROR"; break;
00075         case ORT_LOGGING_LEVEL_FATAL: severityStr = "FATAL"; break;
00076         default: severityStr = "UNKNOWN"; break;
00077     }
00078
00079     // concatenating the output into a single string buffer to write into the log file
00080     std::ostringstream os;
00081     os << "[" << (logid ? logid : "no-logid") << "]" << " "
00082     << severityStr << " | " << (category ? category : "") << " @ "
00083     << (code_location ? code_location : "") << "\n "
00084     << (message ? message : "") << "\n";
00085
00086     std::lock_guard<std::mutex> lock(logMutex);
00087
00088     if (severity <= ORT_LOGGING_LEVEL_INFO) {
00089         // INFO and VERBOSE go to file
00090         if (logFile) {
00091             logFile << os.str();
00092             logFile.flush();
00093         }
00094     } else {
00095         // WARNING, ERROR, FATAL go to stderr
00096         std::cerr << os.str();
00097     }
00098 }
00099
00104 bool YOLO_model::isAvailableGPU(){
00105     // Ottieni la lista dei provider compilati nel tuo ONNX Runtime
00106     std::vector<std::string> providers = Ort::GetAvailableProviders();
00107
00108     for (const auto& p : providers) {
00109         if (p == "CUDAExecutionProvider" || p == "DmlExecutionProvider") {
00110             this->usingGPU = true;
00111             return true;
00112         }
00113     }
00114     this->usingGPU = false;
00115     return false;
00116 }
00117
00118 std::vector<Detection> YOLO_model::detectionPipeline(Mat &img, bool enable_letterbox_padding){
00135     int slice_size = 640;
00136     float overlap_ratio = 0.2;
00137     int step = slice_size * (1 - overlap_ratio);
00138
00139     // removes previous detections, in case of multiple subsequent detections
00140     clearDetections();
00141
00142     this->isAvailableGPU();
00143

```

```

00144     if(img.cols > 1.5*slice_size || img.rows > 1.5*slice_size){
00145         Detections mergedDetections;
00146         /* Tiled approach if image is too big (makes cards easier to detect for the model) */
00147         if(this->usingGPU){
00148             Detections tileDetections;
00149             for(int y = 0; y <= img.rows - slice_size; y += step){
00150                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00151                     cv::Rect roi(x, y, slice_size, slice_size);
00152                     cv::Mat tile = img(roi);
00153                     tileDetections = this->detect(tile, enable_letterbox_padding);
00154                     /* proceeding to map the box coordinates found in the tile to the whole image,
00155                     * then adding the detections of the specific tile to the detections of the overall
00156                     image
00157                     */
00158                     for(auto &box : tileDetections.boundingBoxes){
00159                         box.x += x;
00160                         box.y += y;
00161                     }
00162                     mergeDetections(mergedDetections, tileDetections);
00163                 }
00164             }
00165             /* Detection on the whole image improves result for large images having really large cards
00166             */
00167             mergeDetections(mergedDetections, this->detect(img, enable_letterbox_padding));
00168             this->detections = filterDetectionsNMS(mergedDetections);
00169         }
00170         else{
00171             unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00172             auto activeThreads = std::make_shared<std::atomic<int>>(0);
00173             std::vector<std::future<Detections>> futures;
00174             for(int y = 0; y <= img.rows - slice_size; y += step){
00175                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00176                     while((*activeThreads) >= (int) maxHardwareThreads){
00177                         std::this_thread::yield();
00178                     }
00179                     (*activeThreads)++;
00180                     cv::Rect roi(x, y, slice_size, slice_size);
00181                     cv::Mat tile = img(roi).clone();
00182                     futures.push_back(std::async(std::launch::async, [this, tile,
00183                     enable_letterbox_padding, x, y, activeThreads](){
00184                         Detections tileDetections = this->detect(tile, enable_letterbox_padding);
00185                         /* proceeding to map the box coordinates found in the tile to the whole image,
00186                         * then adding the detections of the specific tile to the detections of the
00187                         overall image */
00188                         for(auto &box : tileDetections.boundingBoxes){
00189                             box.x += x;
00190                             box.y += y;
00191                         }
00192                         (*activeThreads)--;
00193                         return tileDetections;
00194                     }));
00195                 }
00196             }
00197             /* Detect on the whole image, collect detections and merge them, then filter with NMS
00198             suppression */
00199             futures.push_back(std::async(std::launch::async, [this, img, enable_letterbox_padding](){
00200                 return this->detect(img, enable_letterbox_padding);
00201             }));
00202             for(auto& future : futures) mergeDetections(mergedDetections, future.get());
00203             this->detections = filterDetectionsNMS(mergedDetections);
00204         }
00205     }
00206     else this->detections = filterDetectionsNMS(this->detect(img, enable_letterbox_padding));
00207     return this->detections;
00208 }
00209 void YOLO_model::clearDetections(){
00210     this->detections.clear();
00211 }
00212 void YOLO_model::mergeDetections(Detections& dest, const Detections& source){
00213     if(source.boundingBoxes.empty()) return;
00214     size_t newSize = dest.boundingBoxes.size() + source.boundingBoxes.size();
00215     dest.boundingBoxes.reserve(newSize);
00216     dest.classConfidences.reserve(newSize);
00217     dest.classIds.reserve(newSize);
00218     dest.boundingBoxes.insert(dest.boundingBoxes.end(), source.boundingBoxes.begin(),
00219     source.boundingBoxes.end());
00220     dest.classConfidences.insert(dest.classConfidences.end(), source.classConfidences.begin(),
00221     source.classConfidences.end());
00222     dest.classIds.insert(dest.classIds.end(), source.classIds.begin(), source.classIds.end());
00223 }
00224 
```

```

00231 }
00232
00243 Detections YOLO_model::detect(const Mat &img, bool enable_letterbox_padding){
00244     /* Note: there is small to no documentation of the C++ implementation of OnnxRuntime even from the
        developers themselves, so this function has been throughly optimized and documented
        such that every step is perfectly clear to everyone reading this code. */
00245
00246     int inputHeight = img.rows;
00247     int inputWidth = img.cols;
00248
00249     /* It's possible to give an image as input without resizing in OnnxRuntime because it supports
        dynamic input, but the processing of the model is way slower.
        The input needs to be resized anyway for the YOLO model to have good accuracy, so
        pre-processing of the image is still needed and the dynamic input feature is not used for YOLO */
00250
00251     /* YOLO: PRE-PROCESSING required for the image
        the YOLO model requires the image to be:
        1. Of dimensions multiples of 32, because we know YOLO has a CNN in its architecture:
        -There are 5 convolutional layers with stride=2 where the convolutions downsample
        the input by a factor of 2^5=32, so the feature map has size inputSize/32
        -The stride of convolution replaces max pooling of typical CNNs
        -If the inputSize is not divisible by 32, the feature map become of size
        non-integer which leads to feature misalignment
        2. Of overall size square (multiple of 32), typically 640x640 or 416x416
        3. not deformed (deformed = stretched image), since YOLO doesn't recognize deformed
        objects and, even if you train it for that, the predictions are still made in fixed grid cells
        This means that:
        - If an image is smaller or bigger than 640x640, it has to be resized to the same scale of
        640x640 (requirement 2)
        - If the image is not square, you need to resize it while keeping its aspect ratio w:h, so
        that the result is not deformed (requirement 3)
        => the scaling factor has to be saved so that you can correctly resize the bounding box
        to the original size of the image
        - If the image is not square, after resizing you also need to add padding to have a
        resulting image of exactly 640x640 size (requirement 2)
        So the pre-processing of the images is composed of these steps:
        Case A: If the image is square, just resize it if needed and you're done
        Case B: If the image is not square (most of the time):
        1. Resize the image to a multiple of 32 while keeping its aspect ratio, which means:
        resize the longest side to 640 and adjust the shorter side following proportions w:h
        2. add padding to the shorter side in order to reach the square size.
        Note: to guarantee the best performance, the padding has to be done in the same we YOLO was
        trained for.
        YOLO was trained using images with letterbox padding, which means that the image is centered
        and the padding is around the image.
        This is useful because in the uncentered padding the model would have a small bias of
        detecting objects which are closer to the corner.
        A symmetric padding (letterbox) is more robust to this positional bias.
        */
00274     Mat resizedImg;
00275     float resizedWidth = -1;
00276     float resizedHeight = -1;
00277     int xPaddedImgCorner = 0; // It represents the upper left corner of the centered image after
        letterbox padding, will be useful later to center the image
00280     int yPaddedImgCorner = 0;
00281
00282     float imgAspectRatio = static_cast<float>(inputWidth) / inputHeight;
00283     if(inputWidth == inputHeight){
00284         /* Case A: image is square, just resize */
00285         resizedWidth = YOLO_TARGET_INPUT_SIZE;
00286         resizedHeight = YOLO_TARGET_INPUT_SIZE;
00287         resize(img, resizedImg, Size(static_cast<int>(resizedWidth),
        static_cast<int>(resizedHeight)));
00288     }
00289     else{
00290         /* Case B: image is not square. Resize following aspect ratio, then add letterbox padding*/
00291         if(imgAspectRatio > 1.0f){
00292             resizedWidth = YOLO_TARGET_INPUT_SIZE;
00293             resizedHeight = YOLO_TARGET_INPUT_SIZE / imgAspectRatio; // imgAspectRatio > 1 -> have to
        divide so that the height follows the width increase/decrease
00295             resizedHeight = (static_cast<int>(resizedHeight) / 32) * 32; // Ensure dimensions of the
        image are multiples of 32, as required by YOLO11 (in this case width is already multiple of 32)
00296
00297             if(enable_letterbox_padding == true) yPaddedImgCorner =
        static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedHeight/2);
00298         }
00299         else{
00300             resizedHeight = YOLO_TARGET_INPUT_SIZE;
00301             resizedWidth = YOLO_TARGET_INPUT_SIZE * imgAspectRatio; // imgAspectRatio < 1 -> have to
        multiply to make the height follow the width value
00302             resizedWidth = (static_cast<int>(resizedWidth) / 32) * 32;
00303
00304             if(enable_letterbox_padding == true) xPaddedImgCorner =
        static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedWidth/2);
00305         }
00306     }

```

```

00307         Mat tempImg;
00308         resize(img, tempImg, Size(static_cast<int>(resizedWidth), static_cast<int>(resizedHeight)));
00309
00310         //letterbox padding
00311         if(enable_letterbox_padding == true){
00312             resizedImg = Mat::zeros(YOLO_TARGET_INPUT_SIZE, YOLO_TARGET_INPUT_SIZE, CV_8UC3);
00313             tempImg.copyTo(resizedImg(Rect(xPaddedImgCorner, yPaddedImgCorner, resizedWidth,
00314             resizedHeight)));
00315         }
00316         else resizedImg = tempImg.clone();
00317     }
00318     float resizeScalingFactor[2] = {
00319         static_cast<float>(inputWidth) / resizedWidth,
00320         static_cast<float>(inputHeight) / resizedHeight
00321     };
00322
00323     /* TENSORS
00324     OnnxRuntime needs a tensor as input.
00325     The ONNX tensor is not the mathematical tensor (although it is inspired by it).
00326     It's a multi-dimensional array of heterogeneous data types (float, int ...) which is optimized for
00327     computations.
00328     Tensors are useful because they are a convenient data structure for parallelizing GPU computations
00329     during DNN inference.*/
00330
00331     /* INPUT TENSOR
00332     * In OnnxRuntime the tensor is defined by its data (an array) and its shape (dimensions of the
00333     multi-dimensional array)
00334     * Note: ONNX has the convention to have the nput tensor shape following the NCHW order, where N =
00335     batch size, C = n°channels, H = height, W = width of the image.
00336     * For convention height comes before width because data in memory is accessed along the row:
00337     data[y][x]
00338     * You can check the shape of input tensor of your model by inspecting the .onnx file with
00339     https://netron.app/
00340     */
00341
00342     /* Preparing the parameters for the dynamic INPUT TENSOR */
00343     vector<int64_t> inputShape = {1, 3, static_cast<int>(resizedHeight),
00344     static_cast<int>(resizedWidth)}; // 4D tensor
00345     size_t inputTensorSize = 1 * 3 * resizedHeight * resizedWidth;
00346     vector<float> inputData(inputTensorSize); // the input image is flattened into a mono-dimensional
00347     array
00348
00349     // Converting image data from cv::Mat to tensor (normalization to [0,1] and converting (cv::Mat)
00350     BGR to RGB as the YOLO model requires)
00351     double normalizationFactor = 1.0 / 255.0;
00352     for (int row = 0; row < resizedHeight; row++) {
00353         for (int col = 0; col < resizedWidth; col++) {
00354             Vec3b pixel = resizedImg.at<Vec3b>(row, col);
00355             // cv::Mat -> Tensor
00356             inputData[2 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[0] *
00357             normalizationFactor; // B -> R
00358             inputData[1 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[1] *
00359             normalizationFactor; // G -> G
00360             inputData[0 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[2] *
00361             normalizationFactor; // R -> B
00362         }
00363     }
00364
00365     auto memoryInfo = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault); // Allocating
00366     the input tensor on CPU
00367
00368     // Creating the input tensor (which is a view of the data, NOT a deep copy!)
00369     Ort::Value inputOnnxTensor = Ort::Value::CreateTensor<float>(memoryInfo, inputData.data(),
00370     inputTensorSize, inputShape.data(), inputShape.size());
00371
00372     // retrieving now as string the name of the first input and output nodes
00373     // the allocations of these strings are automatically freed by the session
00374     auto inputNodeName = session.GetInputNameAllocated(0, allocator);
00375     auto outputNodeName = session.GetOutputNameAllocated(0, allocator);
00376
00377     // The API needs the array of inputs to set and the array of outputs to get (or even just a
00378     portion of output if needed).
00379     // So here you are setting the ORDER of the input tensors and the ORDER of the outputs.
00380     // This step is useful for models which have multiple input and output tensors where the order
00381     matters.
00382     // In our case we just have a 1 to 1 correspondence of input-output so the arrays passed will only
00383     have 1 item
00384     vector<const char*> inputNodeNames = {inputNodeName.get()};
00385     vector<const char*> outputNodeNames = {outputNodeName.get()};
00386
00387     // Running the inference
00388     auto outputTensors = session.Run(
00389         Ort::RunOptions{nullptr}, // e.g set a verbosity level only for this run
00390         inputNodeNames.data(), &inputOnnxTensor, inputNodeNames.size(), // input to set (=1 input
00391         image)
00392         outputNodeNames.data(), // output to take

```

```

00374         outputNodeNames.size() // actually = 1 output tensors because YOLO has only 1
output layer
00375     );
00376
00377     /* OUTPUT TENSOR
00378     // YOLO11 is pre-trained by default on the COCO dataset, so the number of channels should be:
00379     // 84 = 4 + 80 (= class confidence, one for each class)
00380     // BUT, we fine-tuned the training of YOLO11 with our custom dataset, so the output layer is
restructured to represent 53 classes.
00381     // This means the number of channels should be 57 = 4 (bounding box parameters) + 53 (= class
confidence, one for each class)
00382     // Note: class probability = probability of having detected one class in that region
00383     // class confidence = confidence of having detected an object of a class in that
region, measured as class probability * objectness probability
00384     // objectness probability = probability of having an object in that region, useful for
YOLOv5 but not for YOLO11, since there is already class confidence, so it's been removed.
00385
00386     // Note, YOLOv5 has a different representation of the output layer compared to YOLOv8/YOLO11:
00387     // YOLOv5 -> channels = 85 = 4 (bb_parameters) + 1 (objectness probability) + 80 (= class
probabilities of corresponding 80 classes of the COCO dataset)
00388     // So, in this case the objectness is separate from class confidence and each class has the class
probability instead of the class confidence.
00389     // To get class confidence of each class you have to multiply each class probability*objectness.
So in YOLOv5 the objectness probability was useful!
00390
00391     // To recap, our custom YOLO11 output tensor (3D Mat) has shape (1, 57, 8400*):
00392     // 1 - is the batch size. If =0, it can change dynamically depending
on the input of the net (not needed since in YOLO batch size = 1).
00393     // 57 (84 for default model) - is the number of channels, where the first 4 are the bounding
box parameters (x, y, width, height)
00394     // and the other ones are class probabilities for COCO'S 80
classes (There is no objectiveness parameter in YOLOv8/11)
00395     // 8400 *or lower - is the total number of regions where the predictions are made
("regions" = "grid points" = anchor locations).
00396     // *Since the ONNX model is dynamic, the output tensor can have
a lower number of detections depending on how big is the padding area.
00397     // The first and the third value become 0 when exported into ONNX format with dynamic=True,
indicating that in the dynamic model these parameters can change on runtime.
00398     // Dynamic input/output shapes are not supported by OpenCV, but they are supported by ONNX
Runtime.
00399     // In this case, OnnxRuntime is adopted instead of OpenCV more because of the superior inference
efficiency rather than the support of dynamic models.
00400     // You can also see the shape of the output layer by importing the .onnx file in
https://netron.app/
00401     // See also: https://github.com/orgs/ultralytics/discussions/17254
00402     */
00403
00404     /* Getting dynamic output tensor info (data, shape, count) */
00405     auto& outputTensor = outputTensors[0]; // In the case of YOLO11, only 1 output layer is used, so
there is only 1 output tensor (outputTensors[0])
00406     auto tensorInfo = outputTensor.GetTensorTypeAndShapeInfo();
00407     auto tensorCount = tensorInfo.GetElementCount(); UNUSED(tensorCount);
00408     auto tensorShape = tensorInfo.GetShape();
00409     float* outputData = outputTensor.GetTensorMutableData<float>(); // the output values are flattened
into a 1D array
00410     // YOLO11: outputData[0..4...] = (x, y, width, height, confidence, classConfidence1,
classConfidence2, ...)
00411
00412     int batchSize = static_cast<int>(tensorShape[0]); UNUSED(batchSize);
00413     int numChannels = static_cast<int>(tensorShape[1]);
00414     int numDetections = static_cast<int>(tensorShape[2]); // The number of detections will vary based
on input size and padding area
00415
00416     // NOTE: The output tensor is a matrix like this, if the batch size =1: (neglecting the first four
parameters used for the bounding box)
00417     // class1_detection1      class1_detection2      ...      class1_detectionN
00418     // .....
00419     // class57_detection1      class57_detectionN
00420
00421     Detections goodDetections;
00422     vector<int> classIds;
00423     vector<float> detectedClassConfidences;
00424     vector<Rect> boundingBoxes;
00425
00426     /* Declaring things used in the post-process phase */
00427     Mat classScores;
00428     Point classId;
00429     double maxClassScore;
00430
00431     // I want to iterate through the output as a flat 1D array, so that i can use a pointer instead of
copying values
00432     // When doing this, I need the output to have detections as rows and classes as columns so that I
can evaluate the bounding box of each detection.
00433     // class1_detection1      class1_detection2      ...      class1_detectionN
TRANPOSE      detection1_class1      detection1_class2      ...      detection1_class57
00434     // .....
--->      ....

```



```

00435 // class57_detection1 class57_detectionN
00436 detectionN_class1 detectionN_class57
00437 // This is why a Mat is constructed from the pointer outputData and its transposition t() is taken
00438 // in the order we needed! [Det1Class1 Det1Class2 ... Det1Class57 Det2Class1 ...]
00439 // -> This Mat constructor with the fourth paramter is defined as:
00440 // Mat(int rows, int cols, int type, void* data, size_t step=AUTO_STEP);
00441 // The fourth parameter is just a pointer to the data, it doesn't need to copy the values!
00442 // So this constructor just wraps pointer outputData in a matrix header, resulting a lot more
00443 // efficient.
00444 for(int i = 0; i < numDetections; i++){
00445     float* currentDetection = outputDataMat.ptr<float>(i); // 1 detection corresponds to a 1D
00446     // array of the 57 elements of the detection
00447     float* firstClassScore = currentDetection + 4;
00448     classScores = Mat(1, numChannels-4, CV_32FC1, firstClassScore); // using efficient constructor
00449     // Also, minMaxLoc accepts as input a Mat and as output a Point, so this is why classScores
00450     // and classId are defined as Mat and Point.
00451     minMaxLoc(classScores, 0, &maxClassScore, 0, &classId);
00452     if(maxClassScore > CLASS_CONFIDENCE_THRESHOLD){
00453         detectedClassConfidences.push_back(maxClassScore);
00454         classIds.push_back(classId.x);
00455         float xCenter = currentDetection[0];
00456         float yCenter = currentDetection[1];
00457         float boxWidth = currentDetection[2];
00458         float boxHeight = currentDetection[3];
00459         /* removing coordinates translation caused by the padding */
00460         if(enable_letterbox_padding == true){
00461             xCenter -= xPaddedImgCorner;
00462             yCenter -= yPaddedImgCorner;
00463         }
00464         /* computing the bounding box of the detected object and scaling it to the original image
00465         size */
00466         int xPosScaled = int((xCenter - 0.5f*boxWidth) * resizeScalingFactor[0]);
00467         int yPosScaled = int((yCenter - 0.5f*boxHeight) * resizeScalingFactor[1]);
00468         int bWScaled = int(boxWidth * resizeScalingFactor[0]);
00469         int bHScaled = int(boxHeight * resizeScalingFactor[1]);
00470         Rect box = Rect(xPosScaled, yPosScaled, bWScaled, bHScaled);
00471         /* Clamping bounding box to image boundaries */
00472         box.x = max(0, box.x);
00473         box.y = max(0, box.y);
00474         box.width = min(box.width, img.cols - box.x);
00475         box.height = min(box.height, img.rows - box.y);
00476         boundingBoxes.push_back(box);
00477     }
00478 }
00479 goodDetections.boundingBoxes = boundingBoxes;
00480 goodDetections.classConfidences = detectedClassConfidences;
00481 goodDetections.classIds = classIds;
00482 return goodDetections;
00483 /* Non-Maxima Suppression and construction of the final array of detections */
00484 }
00485 std::vector<Detection> YOLO_model::filterDetectionsNMS(Detections goodDetections){
00486     vector<int> resultNMS;
00487     cv::dnn::NMSBoxes(goodDetections.boundingBoxes, goodDetections.classConfidences,
00488         CLASS_CONFIDENCE_THRESHOLD, NMS_THRESHOLD, resultNMS);
00489     std::vector<Detection> finalDetections;
00490     for(auto finalBoxIndex : resultNMS){
00491         Detection finalDetection;
00492         finalDetection.classId = goodDetections.classIds[finalBoxIndex];
00493         finalDetection.classConfidence = goodDetections.classConfidences[finalBoxIndex];
00494         finalDetection.boundingBox = goodDetections.boundingBoxes[finalBoxIndex];
00495         finalDetections.push_back(finalDetection);
00496     }
00497     return finalDetections;
00498 }
00499 vector<string> YOLO_model::getDataClasses(string labelsFilename){
00500     vector<string> dataClasses;
00501     ifstream ifs(labelsFilename);
00502     string line;
00503     while (getline(ifs, line))
00504         dataClasses.push_back(line);
00505     this->classNames = dataClasses;
00506     return dataClasses;

```

```

00527 }
00528
00537 vector<string> YOLO_model::getDetectionsClassNames(Detections detections){
00538     std::vector<string> detectionsClassNames;
00539     for(int id: detections.classIds){
00540         detectionsClassNames.push_back(this->classNames[id]);
00541     }
00542     return detectionsClassNames;
00543 }
00544
00552 std::string YOLO_model::getClassName(int classId){
00553     return this->classNames[classId];
00554 }
00555
00565 cv::Mat YOLO_model::drawBoundingBoxes(cv::Mat &img, std::vector<Detection> &detections, cv::Scalar
color){
00566     Mat resultImg = img.clone();
00567     for (auto detection : detections)
00568     {
00569         int thickness = max(1, int(max(img.rows, img.cols) / 640));
00570         rectangle(resultImg, detection.boundingBox, color, 2 * thickness);
00571         string label = this->getClassName(detection.classId) + " " +
to_string(static_cast<int>(detection.classConfidence * 100)) + "%";
00572         putText(resultImg, label, Point(detection.boundingBox.x, detection.boundingBox.y - 5 *
thickness), FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 * thickness);
00573     }
00574     // std::string windowTitle = getModelName() + " - " + std::to_string(detections.size()) + "
detections";
00575     // namedWindow(windowTitle, WINDOW_NORMAL);
00576     // imshow(windowTitle, resultImg);
00577     // waitKey(0);
00578     return resultImg;
00579 }
00580
00589 cv::Mat YOLO_model::drawBoundingBoxes(cv::Mat &img, cv::Scalar color){
00590     return drawBoundingBoxes(img, this->detections, color);
00591 }
00592
00596 void YOLO_model::setModelName(string modelName){
00597     this->modelName = modelName;
00598 }
00599
00603 string YOLO_model::getModelName(){
00604     return this->modelName;
00605 }
00606
00610 std::vector<Detection> YOLO_model::getDetections(){
00611     return this->detections;
00612 }

```

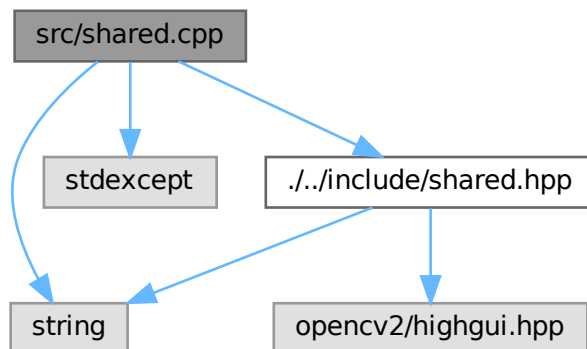
6.15 src/shared.cpp File Reference

```

#include <string>
#include <stdexcept>
#include "../include/shared.hpp"

```

Include dependency graph for shared.cpp:



Namespaces

- namespace `Shared`

Functions

- `Choice Shared::parseChoice` (`std::string s`)

6.16 shared.cpp

[Go to the documentation of this file.](#)

```

00001 #include <string>
00002 #include <stdexcept>
00003
00004 #include "../include/shared.hpp"
00005
00006 namespace Shared{
00007
00008     Choice parseChoice(std::string s) {
00009         //NOTE: Manual Trim
00010         while (!s.empty() && std::isspace((unsigned char)s.front())) s.erase(s.begin());
00011         while (!s.empty() && std::isspace((unsigned char)s.back())) s.pop_back();
00012
00013         if (s.size() != 1) return Choice::Invalid;
00014
00015         char c = (char)std::toupper((unsigned char)s[0]);
00016         if (c == 'C' || c == 'c') return Choice::Camera;
00017         if (c == 'F' || c == 'f') return Choice::File;
00018         if (c == 'H' || c == 'h') return Choice::Help;
00019         return Choice::Invalid;
00020     }
00021 }

```



```
YOLO_model & model,
Mat & frame )
```

Processes detections to group card symbols and apply the Hi-Lo counting system.

This function analyzes the detections and performs several steps:

- Groups symbols of the same class that are physically close into a single "full card" box.
- Ensures each card box meets a minimum size and remains centered.
- Assigns a color to each card based on the Hi-Lo system.
- Creates a visual overlay on the frame with colored boxes and the game status.

Parameters

<i>detections</i>	is a vector of all objects found by the YOLO model.
<i>model</i>	is the reference to the YOLO model.
<i>frame</i>	is the current video frame to be processed.

Returns

Returns a new Mat frame with all visual detections and the running count drawn on it.

Author

Sveva Turola

Definition at line 267 of file [sveva_turola.cpp](#).

```
00267
00268     vector<Detection> green, blue, red;
00269
00270     // groups the detections by value
00271     map<int, vector<Detection>> classGroups;
00272     for(const Detection& d : detections){
00273         classGroups[d.classId].push_back(d);
00274     }
00275
00276     // iterates on each group of classGroups
00277     for(auto& [id, dets] : classGroups){
00278         // used to mark the symbols already processed
00279         vector<bool> processed(dets.size(), false);
00280
00281         for (size_t i = 0; i < dets.size(); i++){
00282             if (processed[i]){
00283                 continue;
00284             }
00285
00286             // takes the first detection not already processed
00287             Detection fullCard = dets[i];
00288             processed[i] = true;
00289
00290             // searches among the other symbols of the same group
00291             for (size_t j = i + 1; j < dets.size(); j++){
00292                 // sets a dynamic threshold based on the dimension of the diagonal
00293                 float diag = sqrt(pow(fullCard.boundingBox.width, 2) +
00294                                     pow(fullCard.boundingBox.height, 2));
00295                 float threshold = min(diag * 4.5f, 350.0f);
00296
00297                 // if it finds another identical and very close symbol, then they belong to the same
00298                 card
00299                 if (!processed[j] && getDistance(dets[i].boundingBox, dets[j].boundingBox) <
00300                 threshold){
00301                     // draws the bounding box of the entire card based on the coordinates of the
00302                     detections
00303                     int minX = min(fullCard.boundingBox.x, dets[j].boundingBox.x);
00304                     int minY = min(fullCard.boundingBox.y, dets[j].boundingBox.y);
00305                     int maxX = max(fullCard.boundingBox.x + fullCard.boundingBox.width,
00306                                     dets[j].boundingBox.x + dets[j].boundingBox.width);
00307                     int maxY = max(fullCard.boundingBox.y + fullCard.boundingBox.height,
00308                                     dets[j].boundingBox.y + dets[j].boundingBox.height);
```

```

00303         Rect mergedBox(minX, minY, maxX - minX, maxY - minY);
00304
00305         // normalization of the dimension of the bounding box
00306         int minSize = 100;
00307
00308         // used when the width of the bounding box is less than minSize
00309         if (mergedBox.width < minSize) {
00310             int diff = minSize - mergedBox.width;
00311             // updates the x to the left to preserve the center
00312             mergedBox.x -= diff / 2;
00313             mergedBox.width = minSize;
00314         }
00315
00316         // used when the height of the bounding box is less than minSize
00317         if (mergedBox.height < minSize) {
00318             int diff = minSize - mergedBox.height;
00319             // updates the y up to preserve the center
00320             mergedBox.y -= diff / 2;
00321             mergedBox.height = minSize;
00322         }
00323
00324         // updates the bounding box of fullCard
00325         fullCard.boundingBox = mergedBox;
00326         processed[j] = true;
00327     }
00328 }
00329
00330 // Hi-Lo system
00331 char cardNumber = model.getClassName(id)[0];
00332
00333 if(cardNumber >= '2' && cardNumber <= '6'){
00334     green.push_back(fullCard);
00335 } else if(cardNumber >= '7' && cardNumber <= '9'){
00336     blue.push_back(fullCard);
00337 } else {
00338     red.push_back(fullCard);
00339 }
00340 }
00341 }
00342
00343 Mat outputFrame = frame.clone();
00344
00345 // draws original bounding boxes using white
00346 if(!detections.empty()){
00347     Mat detectionOverlay = outputFrame.clone();
00348     detectionOverlay = model.drawBoundingBoxes(detectionOverlay, detections, Scalar(255, 255,
00349 255));
00350
00351     addWeighted(detectionOverlay, 0.7, outputFrame, 0.3, 0, outputFrame);
00352 }
00353
00354 // draws the bounding box for the complete card
00355 drawCardGroup(outputFrame, green, Scalar(0, 255, 0), false, model);
00356 drawCardGroup(outputFrame, blue, Scalar(255, 0, 0), false, model);
00357 drawCardGroup(outputFrame, red, Scalar(0, 0, 255), false, model);
00358
00359 // fills the bounding box with the respective color
00360 Mat overlay = outputFrame.clone();
00361
00362 drawCardGroup(overlay, green, Scalar(0, 255, 0), true, model);
00363 drawCardGroup(overlay, blue, Scalar(255, 0, 0), true, model);
00364 drawCardGroup(overlay, red, Scalar(0, 0, 255), true, model);
00365
00366 addWeighted(overlay, 0.3, outputFrame, 0.7, 0, outputFrame);
00367
00368 // computes and shows the running count
00369 drawGameStatus(outputFrame, green, red);
00370
00371 return outputFrame;
00372 }

```

6.17.1.2 drawCardGroup()

```

void drawCardGroup (
    Mat & img,
    const vector< Detection > & list,
    Scalar color,
    bool filled,
    YOLO_model & model )

```

Draws a group of detected cards on the image.

This function draws bounding boxes and labels for a list of detections. It automatically adjusts the thickness of lines and the size of the text based on the image resolution. The labels are centered and placed below each box.

Parameters

<i>img</i>	is the image where the bounding boxes will be drawn.
<i>list</i>	is the list of card detections to show.
<i>color</i>	is the color used for the boxes and text.
<i>filled</i>	is a boolean and if is true, the boxes will be filled with color, otherwise only the outline is drawn.
<i>model</i>	is the reference to the YOLO model.

Author

Sveva Turola

Definition at line 386 of file [sveva_turola.cpp](#).

```

00386 {
00387     // dynamic thickness
00388     int thickness = max(1, int(max(img.rows, img.cols) / 640));
00389
00390     // defines the thickness of the box: filled or thick line
00391     int boxThickness;
00392     if(filled == true){
00393         boxThickness = FILLED;
00394     } else {
00395         boxThickness = 2 * thickness;
00396     }
00397
00398     // iterates through all detections in list
00399     for(size_t i = 0; i < list.size(); i++) {
00400         Rect box = list[i].boundingBox;
00401         rectangle(img, box, color, boxThickness);
00402
00403         string label = model.getClassName(list[i].classId);
00404
00405         double fontScale = 0.5 * thickness;
00406         int fontThickness = max(1, int(1.5 * thickness));
00407         int baseLine = 0;
00408
00409         // measures text dimension
00410         Size textSize = getTextSize(label, FONT_HERSHEY_SIMPLEX, fontScale, fontThickness, &baseLine);
00411
00412         // defines text position: centered w.r.t. x and below the bottom edge
00413         int textX = box.x + (box.width - textSize.width) / 2;
00414         int textY = box.y + box.height + textSize.height + (10 * thickness);
00415
00416         Point textPosition(textX, textY);
00417
00418         putText(img, label, textPosition, FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 *
00419             thickness);
00420     }
00421 }
```

6.17.1.3 drawGameStatus()

```

void drawGameStatus (
    Mat & frame,
    const vector< Detection > & green,
    const vector< Detection > & red )
```

Calculates and displays the blackjack running count and game status.

This function computes the "Running Count" by comparing green (low value) and red (high value) cards. It draws a semi-transparent dashboard on the frame showing the current count and color-coded status (Green for player advantage, Red for house advantage). If the count is high (+2 or more), it displays an additional alert message to monitor the game.

Parameters

<i>frame</i>	is the video frame where the status will be drawn.
<i>green</i>	is the vector of detections for low-value cards.
<i>red</i>	is the vector of detections for high-value cards.

Author

Sveva Turola

Definition at line 433 of file [sveva_turola.cpp](#).

```

00433
00434     // computes running count
00435     int runningCount = (int)green.size() - (int)red.size();
00436
00437     // setup to visualize the running count
00438     int xPos = 20;
00439     int yPos = 20;
00440     int width = 300;
00441     int height = 70;
00442
00443     // draws the background for the running count
00444     Mat overlay;
00445     frame.copyTo(overlay);
00446     rectangle(overlay, Rect(xPos, yPos, width, height), Scalar(0, 0, 0), FILLED);
00447     addWeighted(overlay, 0.5, frame, 0.5, 0, frame);
00448
00449     // draws the border for the background
00450     rectangle(frame, Rect(xPos, yPos, width, height), Scalar(255, 255, 255), 2);
00451
00452     // defines the color and the string
00453     Scalar countColor = Scalar(255, 255, 255); // using white for running count = 0
00454     string prefix = "";
00455     string statusText = "Neutral";
00456
00457     if (runningCount > 0) {
00458         // using green for player's advantage
00459         countColor = Scalar(0, 255, 0);
00460         prefix = "+";
00461         statusText = "Player Advantage";
00462     } else if (runningCount < 0) {
00463         // using red for house's advantage
00464         countColor = Scalar(0, 0, 255);
00465         statusText = "House Advantage";
00466     }
00467
00468     string countStr = "Count: " + prefix + to_string(runningCount);
00469
00470     // writes running count
00471     putText(frame, countStr, Point(xPos + 20, yPos + 45), FONT_HERSHEY_SIMPLEX, 1.2, countColor, 2);
00472
00473     if (runningCount >= 2) {
00474         // in this case, the count is high and this indicates an advantage for the players
00475         int alertY = yPos + height + 10;
00476
00477         // defines the alert rectangle
00478         Rect alertBox;
00479         alertBox.x = xPos;
00480         alertBox.y = alertY;
00481         alertBox.width = width;
00482         alertBox.height = 50;
00483
00484         Mat overlay;
00485         frame.copyTo(overlay);
00486
00487         rectangle(overlay, alertBox, Scalar(0, 0, 255), FILLED);
00488
00489         addWeighted(overlay, 0.7, frame, 0.3, 0, frame);
00490
00491         Point textPosition;
00492         textPosition.x = alertBox.x + 10;
00493         textPosition.y = alertBox.y + 35;
00494
00495         // warns to keep an eye on the players' moves
00496         putText(frame, "Monitor players' moves!", textPosition, FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255,
255, 255), 1);
00497     }
00498 }

```


6.17.1.4 frameCapture()

```
int frameCapture (
    string data_path,
    string labels_path )
```

Initializes the video source and manages the frame extraction process.

This function sets up the environment for video analysis. It creates a fresh output directory for frames, removing any previous content. Depending on the input, it opens either a camera (if "0") or a video file. For video files, it also initializes specific objects for tracking metrics before starting the stream processing pipeline.

Parameters

<i>data_path</i>	is the path to the video file or "0" to use the camera.
<i>labels_path</i>	is the path to the file containing class labels.

Returns

Returns 0 on success, or -1 if the video source fails to open.

Author

Sveva Turola

Definition at line 35 of file [sveva_turola.cpp](#).

```
00035                                     {
00036     fs::path p(data_path);
00037     string name = p.stem().string() + "_frames/";
00038     string outputDir = "../output/" + name; //output directory
00039     Mat frame;
00040     int savedCount = 0;
00041     VideoCapture cap;
00042     VideoWriter out;
00043
00044     // creation of output directory: at every run, creates a new directory, in order to have only the
00045     frames for that run
00046     if (fs::exists(outputDir)){
00047         fs::remove_all(outputDir);
00048     }
00049     fs::create_directory(outputDir);
00050
00051     if(data_path == "0"){
00052         // if camera was selected
00053         cap.open(0);
00054         if (!cap.isOpened()) {
00055             cerr << "Error opening camera!" << endl;
00056             return -1;
00057         }
00058
00059         cout << "Loading camera video..." << endl;
00060         savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00061     } else {
00062         // if video file was selected
00063         cap.open(data_path);
00064         if (!cap.isOpened()) {
00065             cerr << "Error opening video!" << endl;
00066             return -1;
00067         }
00068
00069         cout << "Loading video..." << endl;
00070         initObjectsForVideoMetrics(data_path);
00071         savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00072         printFinalVideoMetrics();
00073     }
00074
00075     cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00076     cout << "Video saved in output directory" << endl;
00077
00078     return 0;
00079 }
00080 }
```

6.17.1.5 `getDistance()`

```
double getDistance (
    const Rect & r1,
    const Rect & r2 )
```

Calculates the distance between the centers of two rectangles.

This function finds the center point (x, y) for both input rectangles. It then calculates the Euclidean distance between these two centers.

Parameters

<i>r1</i>	is the first rectangle (Rect object).
<i>r2</i>	is the second rectangle (Rect object).

Returns

Returns the distance between the two centers as a double.

Author

Sveva Turola

Definition at line 244 of file [sveva_turola.cpp](#).

```
00244 {
00245     // computes center x and center y for both rectangles
00246     double cx1 = r1.x + (r1.width / 2.0);
00247     double cy1 = r1.y + (r1.height / 2.0);
00248     double cx2 = r2.x + (r2.width / 2.0);
00249     double cy2 = r2.y + (r2.height / 2.0);
00250
00251     // returns euclidean distance
00252     return sqrt(pow(cx2 - cx1, 2) + pow(cy2 - cy1, 2));
00253 }
```

6.17.1.6 `playOutputVideo()`

```
void playOutputVideo (
    string path,
    double fps )
```

Opens and reproduces the processed video file.

This function opens the video located at the given path and displays it in a fullscreen window. It calculates the correct playback speed based on the original FPS. Users can control the playback using the keyboard:

- Press 'q' to quit the video.
- Press 'p' to pause the video.

Parameters

<i>path</i>	is the file path of the video to be played.
<i>fps</i>	is the number of frames per second used to set the playback speed.

Author

Sveva Turola

Definition at line 195 of file sveva_turola.cpp.

```

00195 {
00196     // opens the output video detections.mp4
00197     VideoCapture playback;
00198     playback.open(path);
00199
00200     if (!playback.isOpened()) {
00201         cerr << "Error in opening " << path << endl;
00202         return;
00203     }
00204
00205     namedWindow("Final video", WINDOW_NORMAL);
00206     setWindowProperty("Final video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00207
00208     Mat playFrame;
00209
00210     cout << "Reproduction of the output video. Press 'q' to close it or 'p' to pause it." << endl;
00211     while (playback.read(playFrame)) {
00212         // shows the output video
00213         imshow("Final video", playFrame);
00214
00215         // computes the delay for the playback of the video
00216         int delay = 30;
00217
00218         if (fps > 0) {
00219             delay = 1000 / (int) fps;
00220         }
00221
00222         int key = waitKey(delay);
00223         // used to stop the playback of the output video
00224         if (key == 'q') {
00225             break;
00226         } else if (key == 'p') {
00227             // used to pause the playback of the output video
00228             waitKey(0);
00229         }
00230     }
00231
00232     playback.release();
00233     destroyWindow("Final video");
00234 }

```

6.17.1.7 processStream()

```

int processStream (
    string data_path,
    VideoCapture cap,
    VideoWriter out,
    Mat frame,
    int savedCount,
    string labels_path )

```

Manages the main video processing loop, including detection and frame saving.

This function initializes the YOLO11s model and processes the video frame by frame. It performs several key tasks:

- Detects objects at fixed intervals.
- Computes video metrics.
- Identifies card values and draws boxes around them.
- Saves frames as images only when the number of detected cards changes.
- Saves all processed frames into a final MP4 video file.

Parameters

<i>cap</i>	is the VideoCapture object already opened in frameCapture.
<i>out</i>	is the VideoWriter object used to save the output video.
<i>frame</i>	is a Mat object used to store each frame during reading.
<i>savedCount</i>	is the initial number of saved frames.
<i>labels_path</i>	is the path to the configuration file for class names.

Returns

Returns the total number of saved frames, or -1 if the initial frame is empty or the output video cannot be open.

Author

Sveva Turola

Definition at line 99 of file [sveva_turola.cpp](#).

```

00099 {
00100     // initialization of YOLO model
00101     YOLO_model model;
00102     model.setModelName("YOLO11s");
00103     vector<string> dataClasses = model.getDataClasses(labels_path);
00104
00105     // setup of video variables
00106     int frameCount = 0;
00107     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00108     int fps = cap.get(CAP_PROP_FPS);
00109
00110     vector<Detection> detections;
00111
00112     // checks if input is the camera
00113     bool isCamera = (cap.get(CAP_PROP_FRAME_COUNT) <= 0);
00114
00115     cap.read(frame);
00116     if (frame.empty()){
00117         return -1;
00118     }
00119
00120     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00121     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00122
00123     // opens output video
00124     fs::path p(data_path);
00125     string name = p.stem().string();
00126     out.open("../output/" + name + "_detections.mp4", codec, fps, Size(frame_width, frame_height),
00127         true);
00128     if(!out.isOpened()){
00129         cerr << "Could not open the output video file for write\n";
00130         return -1;
00131     }
00132
00133     int cardCount = -1;
00134
00135     cout << "Starting video analysis... Results will be available once the computation is complete." <<
00136     endl;
00137
00138     while (true){
00139         cap.read(frame);
00140
00141         if (frame.empty()){
00142             break;
00143         }
00144
00145         savedCount++;
00146         int key = waitKey(1) & 0xFF;
00147
00148         if (frameCount % (fps/2) == 0){
00149             detections = model.detectionPipeline(frame);
00150             computeVideoMetrics(detections, frameCount+1);
00151         }
00152
00153         frameCount++;
00154         Mat outputFrame = cardValues(detections, model, frame);
00155
00156         int currentCardCount = detections.size();

```

```

00155
00156 // saving only relevant frames, that are the frames in which the number of detections changes
00157 if(currentCardCount != cardCount){
00158     string filename = "../output/" + name + "_frames/frame_" + to_string(savedCount) + ".jpg";
00159     imwrite(filename, outputFrame);
00160     cardCount = currentCardCount;
00161 }
00162
00163 // adds each frame to the video
00164 out.write(outputFrame);
00165
00166 if(isCamera == true){
00167     // stops the camera
00168     if (key == 'q') {
00169         cout << "Closed camera!\n";
00170         break;
00171     }
00172 }
00173 }
00174
00175 cap.release();
00176 out.release();
00177 destroyAllWindows();
00178
00179 // calls the method to show the video
00180 playOutputVideo("../output/" + name + "_detections.mp4", fps);
00181
00182 return savedCount;
00183 }

```

6.18 sveva_turola.cpp

[Go to the documentation of this file.](#)

```

00001 // AUTHOR: Sveva Turola
00002 #include <opencv2/features2d.hpp>
00003 #include <opencv2/highgui.hpp>
00004 #include <opencv2/opencv.hpp>
00005 #include <opencv2/core.hpp>
00006 #include <opencv2/imgproc.hpp>
00007 #include <opencv2/calib3d.hpp>
00008 #include <opencv2/videoio.hpp>
00009 #include <filesystem>
00010 #include <iostream>
00011 #include <string>
00012 #include <vector>
00013 #include <map>
00014 #include <algorithm>
00015 #include <cmath>
00016
00017 #include "../include/sveva_turola.hpp"
00018 #include "../include/marco_annunziata.hpp"
00019 #include "../include/hermann_serain.hpp"
00020
00021 using namespace std;
00022 using namespace cv;
00023 namespace fs = std::filesystem;
00024
00035 int frameCapture(string data_path, string labels_path) {
00036     fs::path p(data_path);
00037     string name = p.stem().string() + "_frames/";
00038     string outputDir = "../output/" + name; //output directory
00039     Mat frame;
00040     int savedCount = 0;
00041     VideoCapture cap;
00042     VideoWriter out;
00043
00044     // creation of output directory: at every run, creates a new directory, in order to have only the
    frames for that run
00045     if (fs::exists(outputDir)){
00046         fs::remove_all(outputDir);
00047     }
00048     fs::create_directory(outputDir);
00049
00050     if(data_path == "0"){
00051         // if camera was selected
00052         cap.open(0);
00053
00054         if (!cap.isOpened()) {
00055             cerr << "Error opening camera!" << endl;
00056             return -1;
00057         }

```

```

00058
00059     cout << "Loading camera video..." << endl;
00060     savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00061 } else {
00062     // if video file was selected
00063     cap.open(data_path);
00064
00065     if (!cap.isOpened()) {
00066         cerr << "Error opening video!" << endl;
00067         return -1;
00068     }
00069
00070     cout << "Loading video..." << endl;
00071     initObjectsForVideoMetrics(data_path);
00072     savedCount = processStream(data_path, cap, out, frame, savedCount, labels_path);
00073     printFinalVideoMetrics();
00074 }
00075
00076 cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00077 cout << "Video saved in output directory" << endl;
00078
00079 return 0;
00080 }
00081
00099 int processStream(string data_path, VideoCapture cap, VideoWriter out, Mat frame, int savedCount,
00100 string labels_path){
00101     // initialization of YOLO model
00102     YOLO_model model;
00103     model.setModelName("YOLO11s");
00104     vector<string> dataClasses = model.getDataClasses(labels_path);
00105
00106     // setup of video variables
00107     int frameCount = 0;
00108     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00109     int fps = cap.get(CAP_PROP_FPS);
00110
00111     vector<Detection> detections;
00112
00113     // checks if input is the camera
00114     bool isCamera = (cap.get(CAP_PROP_FRAME_COUNT) <= 0);
00115
00116     cap.read(frame);
00117     if (frame.empty()){
00118         return -1;
00119     }
00120
00121     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00122     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00123
00124     // opens output video
00125     fs::path p(data_path);
00126     string name = p.stem().string();
00127     out.open("../output/" + name + "_detections.mp4", codec, fps, Size(frame_width, frame_height),
00128 true);
00129     if(!out.isOpened()){
00130         cerr << "Could not open the output video file for write\n";
00131         return -1;
00132     }
00133
00134     int cardCount = -1;
00135
00136     cout << "Starting video analysis... Results will be available once the computation is complete." <<
endl;
00137
00138     while (true){
00139         cap.read(frame);
00140
00141         if (frame.empty()){
00142             break;
00143         }
00144
00145         savedCount++;
00146         int key = waitKey(1) & 0xFF;
00147
00148         if (frameCount % (fps/2) == 0){
00149             detections = model.detectionPipeline(frame);
00150             computeVideoMetrics(detections, frameCount+1);
00151         }
00152
00153         frameCount++;
00154         Mat outputFrame = cardValues(detections, model, frame);
00155
00156         int currentCardCount = detections.size();
00157
00158         // saving only relevant frames, that are the frames in which the number of detections changes
00159         if(currentCardCount != cardCount){
00160             string filename = "../output/" + name + "_frames/frame_" + to_string(savedCount) + ".jpg";

```

```

00159         imwrite(filename, outputFrame);
00160         cardCount = currentCardCount;
00161     }
00162
00163     // adds each frame to the video
00164     out.write(outputFrame);
00165
00166     if(isCamera == true){
00167         // stops the camera
00168         if (key == 'q') {
00169             cout << "Closed camera!\n";
00170             break;
00171         }
00172     }
00173 }
00174
00175 cap.release();
00176 out.release();
00177 destroyAllWindows();
00178
00179 // calls the method to show the video
00180 playOutputVideo("../output/" + name + "_detections.mp4", fps);
00181
00182 return savedCount;
00183 }
00184
00195 void playOutputVideo(string path, double fps){
00196     // opens the output video detections.mp4
00197     VideoCapture playback;
00198     playback.open(path);
00199
00200     if (!playback.isOpened()) {
00201         cerr << "Error in opening " << path << endl;
00202         return;
00203     }
00204
00205     namedWindow("Final video", WINDOW_NORMAL);
00206     setWindowProperty("Final video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00207
00208     Mat playFrame;
00209
00210     cout << "Reproduction of the output video. Press 'q' to close it or 'p' to pause it." << endl;
00211     while (playback.read(playFrame)) {
00212         // shows the output video
00213         imshow("Final video", playFrame);
00214
00215         // computes the delay for the playback of the video
00216         int delay = 30;
00217
00218         if(fps > 0){
00219             delay = 1000 / (int) fps;
00220         }
00221
00222         int key = waitKey(delay);
00223         // used to stop the playback of the output video
00224         if (key == 'q') {
00225             break;
00226         } else if (key == 'p'){
00227             // used to pause the playback of the output video
00228             waitKey(0);
00229         }
00230     }
00231
00232     playback.release();
00233     destroyWindow("Final video");
00234 }
00235
00244 double getDistance(const Rect& r1, const Rect& r2){
00245     // computes center x and center y for both rectangles
00246     double cx1 = r1.x + (r1.width / 2.0);
00247     double cy1 = r1.y + (r1.height / 2.0);
00248     double cx2 = r2.x + (r2.width / 2.0);
00249     double cy2 = r2.y + (r2.height / 2.0);
00250
00251     // returns euclidean distance
00252     return sqrt(pow(cx2 - cx1, 2) + pow(cy2 - cy1, 2));
00253 }
00254
00267 Mat cardValues(vector<Detection> detections, YOLO_model &model, Mat &frame){
00268     vector<Detection> green, blue, red;
00269
00270     // groups the detections by value
00271     map<int, vector<Detection> > classGroups;
00272     for(const Detection& d : detections){
00273         classGroups[d.classId].push_back(d);
00274     }
00275

```

```

00276     // iterates on each group of classGroups
00277     for(auto& [id, dets] : classGroups){
00278         // used to mark the symbols already processed
00279         vector<bool> processed(dets.size(), false);
00280
00281         for (size_t i = 0; i < dets.size(); i++){
00282             if (processed[i]){
00283                 continue;
00284             }
00285
00286             // takes the first detection not already processed
00287             Detection fullCard = dets[i];
00288             processed[i] = true;
00289
00290             // searches among the other symbols of the same group
00291             for (size_t j = i + 1; j < dets.size(); j++){
00292                 // sets a dynamic threshold based on the dimension of the diagonal
00293                 float diag = sqrt(pow(fullCard.boundingBox.width, 2) +
pow(fullCard.boundingBox.height, 2));
00294                 float threshold = min(diag * 4.5f, 350.0f);
00295
00296                 // if it finds another identical and very close symbol, then they belong to the same
00297                 card
if (!processed[j] && getDistance(dets[i].boundingBox, dets[j].boundingBox) <
threshold){
00298                     // draws the bounding box of the entire card based on the coordinates of the
00299                     detections
int minX = min(fullCard.boundingBox.x, dets[j].boundingBox.x);
00300                     int minY = min(fullCard.boundingBox.y, dets[j].boundingBox.y);
00301                     int maxX = max(fullCard.boundingBox.x + fullCard.boundingBox.width,
dets[j].boundingBox.x + dets[j].boundingBox.width);
00302                     int maxY = max(fullCard.boundingBox.y + fullCard.boundingBox.height,
dets[j].boundingBox.y + dets[j].boundingBox.height);
00303
00304                     Rect mergedBox(minX, minY, maxX - minX, maxY - minY);
00305
00306                     // normalization of the dimension of the bounding box
00307                     int minSize = 100;
00308
00309                     // used when the width of the bounding box is less than minSize
00310                     if (mergedBox.width < minSize) {
00311                         int diff = minSize - mergedBox.width;
00312                         // updates the x to the left to preserve the center
00313                         mergedBox.x -= diff / 2;
00314                         mergedBox.width = minSize;
00315                     }
00316
00317                     // used when the height of the bounding box is less than minSize
00318                     if (mergedBox.height < minSize) {
00319                         int diff = minSize - mergedBox.height;
00320                         // updates the y up to preserve the center
00321                         mergedBox.y -= diff / 2;
00322                         mergedBox.height = minSize;
00323                     }
00324
00325                     // updates the bounding box of fullCard
00326                     fullCard.boundingBox = mergedBox;
00327                     processed[j] = true;
00328                 }
00329             }
00330
00331             // Hi-Lo system
00332             char cardNumber = model.getClassName(id)[0];
00333
00334             if(cardNumber >= '2' && cardNumber <= '6'){
00335                 green.push_back(fullCard);
00336             } else if(cardNumber >= '7' && cardNumber <= '9'){
00337                 blue.push_back(fullCard);
00338             } else {
00339                 red.push_back(fullCard);
00340             }
00341         }
00342     }
00343
00344     Mat outputFrame = frame.clone();
00345
00346     // draws original bounding boxes using white
00347     if(!detections.empty()){
00348         Mat detectionOverlay = outputFrame.clone();
00349         detectionOverlay = model.drawBoundingBoxes(detectionOverlay, detections, Scalar(255, 255,
255));
00350
00351         addWeighted(detectionOverlay, 0.7, outputFrame, 0.3, 0, outputFrame);
00352     }
00353
00354     // draws the bounding box for the complete card
00355     drawCardGroup(outputFrame, green, Scalar(0, 255, 0), false, model);

```



```

00356     drawCardGroup(outputFrame, blue, Scalar(255, 0, 0), false, model);
00357     drawCardGroup(outputFrame, red, Scalar(0, 0, 255), false, model);
00358
00359     // fills the bounding box with the respective color
00360     Mat overlay = outputFrame.clone();
00361
00362     drawCardGroup(overlay, green, Scalar(0, 255, 0), true, model);
00363     drawCardGroup(overlay, blue, Scalar(255, 0, 0), true, model);
00364     drawCardGroup(overlay, red, Scalar(0, 0, 255), true, model);
00365
00366     addWeighted(overlay, 0.3, outputFrame, 0.7, 0, outputFrame);
00367
00368     // computes and shows the running count
00369     drawGameStatus(outputFrame, green, red);
00370
00371     return outputFrame;
00372 }
00373
00386 void drawCardGroup(Mat& img, const vector<Detection>& list, Scalar color, bool filled, YOLO_model&
model){
00387     // dynamic thickness
00388     int thickness = max(1, int(max(img.rows, img.cols) / 640));
00389
00390     // defines the thickness of the box: filled or thick line
00391     int boxThickness;
00392     if(filled == true){
00393         boxThickness = FILLED;
00394     } else {
00395         boxThickness = 2 * thickness;
00396     }
00397
00398     // iterates through all detections in list
00399     for(size_t i = 0; i < list.size(); i++) {
00400         Rect box = list[i].boundingBox;
00401         rectangle(img, box, color, boxThickness);
00402
00403         string label = model.getClassName(list[i].classId);
00404
00405         double fontScale = 0.5 * thickness;
00406         int fontThickness = max(1, int(1.5 * thickness));
00407         int baseLine = 0;
00408
00409         // measures text dimension
00410         Size textSize = getTextSize(label, FONT_HERSHEY_SIMPLEX, fontScale, fontThickness, &baseLine);
00411
00412         // defines text position: centered w.r.t. x and below the bottom edge
00413         int textX = box.x + (box.width - textSize.width) / 2;
00414         int textY = box.y + box.height + textSize.height + (10 * thickness);
00415
00416         Point textPosition(textX, textY);
00417
00418         putText(img, label, textPosition, FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 *
thickness);
00419     }
00420 }
00421
00433 void drawGameStatus(Mat& frame, const vector<Detection>& green, const vector<Detection>& red){
00434     // computes running count
00435     int runningCount = (int)green.size() - (int)red.size();
00436
00437     // setup to visualize the running count
00438     int xPos = 20;
00439     int yPos = 20;
00440     int width = 300;
00441     int height = 70;
00442
00443     // draws the background for the running count
00444     Mat overlay;
00445     frame.copyTo(overlay);
00446     rectangle(overlay, Rect(xPos, yPos, width, height), Scalar(0, 0, 0), FILLED);
00447     addWeighted(overlay, 0.5, frame, 0.5, 0, frame);
00448
00449     // draws the border for the background
00450     rectangle(frame, Rect(xPos, yPos, width, height), Scalar(255, 255, 255), 2);
00451
00452     // defines the color and the string
00453     Scalar countColor = Scalar(255, 255, 255); // using white for running count = 0
00454     string prefix = "";
00455     string statusText = "Neutral";
00456
00457     if (runningCount > 0) {
00458         // using green for player's advantage
00459         countColor = Scalar(0, 255, 0);
00460         prefix = "+";
00461         statusText = "Player Advantage";
00462     } else if (runningCount < 0) {
00463         // using red for house's advantage

```

```
00464         countColor = Scalar(0, 0, 255);
00465         statusText = "House Advantage";
00466     }
00467
00468     string countStr = "Count: " + prefix + to_string(runningCount);
00469
00470     // writes running count
00471     putText(frame, countStr, Point(xPos + 20, yPos + 45), FONT_HERSHEY_SIMPLEX, 1.2, countColor, 2);
00472
00473     if (runningCount >= 2) {
00474         // in this case, the count is high and this indicates an advantage for the players
00475         int alertY = yPos + height + 10;
00476
00477         // defines the alert rectangle
00478         Rect alertBox;
00479         alertBox.x = xPos;
00480         alertBox.y = alertY;
00481         alertBox.width = width;
00482         alertBox.height = 50;
00483
00484         Mat overlay;
00485         frame.copyTo(overlay);
00486
00487         rectangle(overlay, alertBox, Scalar(0, 0, 255), FILLED);
00488
00489         addWeighted(overlay, 0.7, frame, 0.3, 0, frame);
00490
00491         Point textPosition;
00492         textPosition.x = alertBox.x + 10;
00493         textPosition.y = alertBox.y + 35;
00494
00495         // warns to keep an eye on the players' moves
00496         putText(frame, "Monitor players' moves!", textPosition, FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255,
255, 255), 1);
00497     }
00498 }
```

Index

- ~TerminalRawMode
 - TerminalRawMode, [12](#)
- allocator
 - YOLO_model, [29](#)
- boundingBox
 - Detection, [9](#)
- boundingBoxes
 - Detections, [10](#)
- calculateloU
 - hermann_serain.cpp, [53](#)
- Camera
 - Shared, [7](#)
- canComputeImageMetrics
 - hermann_serain.cpp, [53](#)
- canComputeMetrics
 - hermann_serain.cpp, [53](#)
- canComputeVideoMetrics
 - hermann_serain.cpp, [53](#)
- cardValues
 - sveva_turola.cpp, [78](#)
 - sveva_turola.hpp, [39](#)
- checkPath
 - hermann_serain.cpp, [53](#)
- Choice
 - Shared, [7](#)
- choice
 - UserData, [13](#)
- CLASS_CONFIDENCE_THRESHOLD
 - YOLO_model, [29](#)
- classConfidence
 - Detection, [9](#)
- classConfidences
 - Detections, [10](#)
- classId
 - Detection, [9](#)
- classIds
 - Detections, [10](#)
- classNames
 - YOLO_model, [29](#)
- clearDetections
 - YOLO_model, [16](#)
- computeImageMetrics
 - hermann_serain.cpp, [54](#)
 - hermann_serain.hpp, [32](#)
- computeVideoMetrics
 - hermann_serain.cpp, [54](#)
 - hermann_serain.hpp, [32](#)
- data_path
 - UserData, [13](#)
- DEFAULT_IMAGE_PATH
 - hermann_serain.cpp, [61](#)
- DEFAULT_VIDEO_PATH
 - hermann_serain.cpp, [61](#)
- detect
 - YOLO_model, [16](#)
- Detection, [9](#)
 - boundingBox, [9](#)
 - classConfidence, [9](#)
 - classId, [9](#)
- detectionPipeline
 - YOLO_model, [20](#)
- Detections, [10](#)
 - boundingBoxes, [10](#)
 - classConfidences, [10](#)
 - classIds, [10](#)
- detections
 - YOLO_model, [29](#)
- drawBoundingBoxes
 - YOLO_model, [22](#)
- drawCardGroup
 - sveva_turola.cpp, [80](#)
 - sveva_turola.hpp, [41](#)
- drawGameStatus
 - sveva_turola.cpp, [81](#)
 - sveva_turola.hpp, [42](#)
- env
 - YOLO_model, [29](#)
- File
 - Shared, [7](#)
- filterDetectionsNMS
 - YOLO_model, [23](#)
- finalProject.cpp
 - main, [49](#)
- findMatches
 - hermann_serain.cpp, [55](#)
- FLAG_IMAGE
 - hermann_serain.cpp, [61](#)
- FLAG_VIDEO
 - hermann_serain.cpp, [61](#)
- fn
 - MetricsCounter, [11](#)
- fp
 - MetricsCounter, [11](#)
- frameCapture
 - sveva_turola.cpp, [82](#)

- sveva_turola.hpp, 43
- g_cumulativeVideoMetrics
 - hermann_serain.cpp, 61
- g_lastLine
 - hermann_serain.cpp, 61
- g_motFile
 - hermann_serain.cpp, 62
- g_videoMetricsEnabled
 - hermann_serain.cpp, 62
- getClassName
 - YOLO_model, 24
- getDataClasses
 - YOLO_model, 24
- getDetections
 - YOLO_model, 25
- getDetectionsClassNames
 - YOLO_model, 25
- getDistance
 - sveva_turola.cpp, 83
 - sveva_turola.hpp, 44
- getModelName
 - YOLO_model, 26
- handleCamera
 - hermann_serain.cpp, 55
- handleFile
 - hermann_serain.cpp, 55
- Help
 - Shared, 7
- hermann_serain.cpp
 - calculateIoU, 53
 - canComputeImageMetrics, 53
 - canComputeMetrics, 53
 - canComputeVideoMetrics, 53
 - checkPath, 53
 - computeImageMetrics, 54
 - computeVideoMetrics, 54
 - DEFAULT_IMAGE_PATH, 61
 - DEFAULT_VIDEO_PATH, 61
 - findMatches, 55
 - FLAG_IMAGE, 61
 - FLAG_VIDEO, 61
 - g_cumulativeVideoMetrics, 61
 - g_lastLine, 61
 - g_motFile, 62
 - g_videoMetricsEnabled, 62
 - handleCamera, 55
 - handleFile, 55
 - initObjectsForVideoMetrics, 56
 - parseYoloLabels, 57
 - printFinalMetrics, 57
 - printFinalVideoMetrics, 58
 - printHelp, 58
 - readInput, 58
 - redrawLine, 59
 - splitPath, 60
 - updateMetricsCounters, 60
- hermann_serain.hpp
 - computeImageMetrics, 32
 - computeVideoMetrics, 32
 - initObjectsForVideoMetrics, 33
 - printFinalVideoMetrics, 33
 - readInput, 33
- include/hermann_serain.hpp, 31, 34
- include/marco_annunziata.hpp, 35, 36
- include/shared.hpp, 37, 38
- include/sveva_turola.hpp, 38, 48
- initObjectsForVideoMetrics
 - hermann_serain.cpp, 56
 - hermann_serain.hpp, 33
- Invalid
 - Shared, 7
- isAvailableGPU
 - YOLO_model, 26
- logger
 - YOLO_model, 26
- main
 - finalProject.cpp, 49
- marco_annunziata.cpp
 - UNUSED, 69
- mergeDetections
 - YOLO_model, 27
- MetricsCounter, 11
 - fn, 11
 - fp, 11
 - tp, 11
- modelName
 - YOLO_model, 29
- NMS_THRESHOLD
 - YOLO_model, 29
- oldt
 - TerminalRawMode, 12
- parseChoice
 - Shared, 7
- parseYoloLabels
 - hermann_serain.cpp, 57
- playOutputVideo
 - sveva_turola.cpp, 84
 - sveva_turola.hpp, 45
- printFinalMetrics
 - hermann_serain.cpp, 57
- printFinalVideoMetrics
 - hermann_serain.cpp, 58
 - hermann_serain.hpp, 33
- printHelp
 - hermann_serain.cpp, 58
- processStream
 - sveva_turola.cpp, 85
 - sveva_turola.hpp, 46
- readInput
 - hermann_serain.cpp, 58

- hermann_serain.hpp, 33
- redrawLine
 - hermann_serain.cpp, 59
- session
 - YOLO_model, 29
- sessionOptions
 - YOLO_model, 30
- setModelName
 - YOLO_model, 28
- Shared, 7
 - Camera, 7
 - Choice, 7
 - File, 7
 - Help, 7
 - Invalid, 7
 - parseChoice, 7
- splitPath
 - hermann_serain.cpp, 60
- src/finalProject.cpp, 48, 50
- src/hermann_serain.cpp, 51, 62
- src/marco_annunziata.cpp, 69
- src/shared.cpp, 76, 77
- src/sveva_turola.cpp, 78, 87
- sveva_turola.cpp
 - cardValues, 78
 - drawCardGroup, 80
 - drawGameStatus, 81
 - frameCapture, 82
 - getDistance, 83
 - playOutputVideo, 84
 - processStream, 85
- sveva_turola.hpp
 - cardValues, 39
 - drawCardGroup, 41
 - drawGameStatus, 42
 - frameCapture, 43
 - getDistance, 44
 - playOutputVideo, 45
 - processStream, 46
- TerminalRawMode, 11
 - ~TerminalRawMode, 12
 - oldt, 12
 - TerminalRawMode, 12
- tp
 - MetricsCounter, 11
- UNUSED
 - marco_annunziata.cpp, 69
- updateMetricsCounters
 - hermann_serain.cpp, 60
- UserData, 12
 - choice, 13
 - data_path, 13
 - UserData, 13
- usingGPU
 - YOLO_model, 30
- YOLO_model, 13
 - allocator, 29
 - CLASS_CONFIDENCE_THRESHOLD, 29
 - classNames, 29
 - clearDetections, 16
 - detect, 16
 - detectionPipeline, 20
 - detections, 29
 - drawBoundingBoxes, 22
 - env, 29
 - filterDetectionsNMS, 23
 - getClassName, 24
 - getDataClasses, 24
 - getDetections, 25
 - getDetectionsClassNames, 25
 - getModelName, 26
 - isAvailableGPU, 26
 - logger, 26
 - mergeDetections, 27
 - modelName, 29
 - NMS_THRESHOLD, 29
 - session, 29
 - sessionOptions, 30
 - setModelName, 28
 - usingGPU, 30
 - YOLO_model, 15
 - YOLO_TARGET_INPUT_SIZE, 30
- YOLO_TARGET_INPUT_SIZE
 - YOLO_model, 30